UVM Introduction and UVM Hierarchy

UVM Introduction and UVM Hierarchy

1. Topic Overview

This section introduces **UVM**, which stands for **Universal Verification Methodology**.

UVM is used to build structured, reusable, and scalable verification environments for complex digital designs.

2. Why UVM Is Needed

SystemVerilog is a powerful language for hardware design and verification.

It supports:

- Object-oriented programming
- Constrained random verification
- Functional coverage
- Assertions
- Advanced verification features

However, SystemVerilog alone does not provide a standard testbench methodology.

Different teams may create different testbench structures, which makes reuse and scalability difficult.

UVM solves this problem by providing a standard verification framework.

3. What Is UVM?

UVM is not a separate programming language.

It is a verification methodology built on top of SystemVerilog.

UVM provides:

- Standard testbench architecture
- Reusable verification components
- Scalable verification structure
- Built-in base classes
- Common phases and reporting mechanism
- Standard communication methods between components

4. Evolution of UVM

Verilog-Based Testbenches

In early verification, testbenches were written using Verilog.

These testbenches usually applied direct input stimulus and checked output responses.

Issues with Verilog Testbenches

Directed Verilog testbenches had several limitations:

- Time-consuming to write manually
- Difficult to scale for complex designs
- Low reusability
- New testbench needed for every project
- Limited structure for large verification environments

5. SystemVerilog Improvements

SystemVerilog introduced several features that improved verification.

Important features include:

- Classes
- Polymorphism
- Randomization
- Constrained random stimulus generation
- Functional coverage
- Assertions

Benefits of SystemVerilog

SystemVerilog made testbenches more structured and powerful.

It allowed:

- Random stimulus generation to improve coverage
- Functional coverage to track tested scenarios
- Assertions to check design properties

Limitations of SystemVerilog Alone

Even though SystemVerilog is powerful, it still has limitations:

- No standard testbench structure
- Each team may build its own framework
- No fixed guidelines for stimulus generation
- No fixed guidelines for checking and coverage collection
- Limited reuse across projects

6. UVM Standardization

UVM was introduced as an industry-wide standardized methodology.

It combines the best features of earlier methodologies such as:

- OVM
- VMM

UVM was developed by **Accellera** and became an **IEEE standard in 2017**.

Today, major EDA vendors support UVM, which improves compatibility across tools.

7. Basic UVM Testbench Idea

A UVM testbench is more structured than a simple SystemVerilog testbench.

In a basic SystemVerilog testbench, we may create:

- Driver class
- Monitor class
- Scoreboard
- Reference model
- Environment
- Test

In UVM, these components are created using predefined UVM base classes.

The user extends these base classes to build custom verification components.

8. UVM Class Hierarchy

The main parent class in UVM is:

- `uvm_object`

From `uvm_object`, two main branches are formed:

1. Component branch
2. Sequence branch

9. UVM Component Branch

The component branch represents structural elements of the testbench.

The hierarchy includes:

- `uvm_object`
- `uvm_report_object`
- `uvm_component`

From `uvm_component`, several important classes are derived:

- `uvm_driver`
- `uvm_monitor`
- `uvm_sequencer`
- `uvm_scoreboard`

A user-defined driver, monitor, sequencer, or scoreboard must extend these UVM base classes.

Example:

- User driver extends `uvm_driver`

- User monitor extends `uvm_monitor`
- User sequencer extends `uvm_sequencer`
- User scoreboard extends `uvm_scoreboard`

This allows the user to reuse built-in UVM features.

10. UVM Sequence Branch

The sequence branch is used for transaction and stimulus generation.

Important classes include:

- `uvm_transaction`
- `uvm_sequence_item`
- `uvm_sequence`

These classes are mainly used to define transactions and generate stimulus for the DUT.

11. UVM Components

UVM components represent the structural parts of a testbench.

Examples:

- Driver
- Monitor
- Sequencer
- Scoreboard
- Agent
- Environment
- Test

Important Features of UVM Components

UVM components:

- Have hierarchy
- Have hierarchical names
- Are useful for reporting and debugging
- Participate automatically in UVM phases
- Are created during the build phase
- Are usually created using the factory `create()` method
- Communicate using TLM ports, exports, and analysis ports

12. UVM Objects

UVM objects are mainly used as data containers or non-hierarchical utilities.

Examples:

- Sequence items

- Transactions
- Sequences

Important Features of UVM Objects

UVM objects:

- Do not have hierarchy
- Do not have parent-child structure like components
- Are mainly used for data and stimulus generation
- Do not directly represent structural testbench blocks

13. Difference Between UVM Component and UVM Object

Feature	UVM Component	UVM Object
Purpose	Structural testbench element	Data or utility object
Hierarchy	Has hierarchy	No hierarchy
Parent-child relationship	Present	Not present
Examples	Driver, monitor, sequencer, scoreboard	Transaction, sequence item, sequence
UVM phases	Participates in phases	Does not participate like components
Main use	Build testbench structure	Carry/generate stimulus data

14. Typical UVM Testbench Structure

A typical UVM testbench contains:

- Test
- Environment
- Agent
- Sequencer
- Driver
- Monitor
- Scoreboard
- Coverage collector
- DUT

Basic Hierarchy

`Test`

→ `Environment`

→ `Agent`

→ `Sequencer`

→ `Driver`

→ `Monitor`

The environment may also include:

- Scoreboard
- Reference model
- Coverage collector

15. UVM Test

The test is the top-level class in the UVM testbench.

It controls:

- Which environment is created
- Which sequences are run
- Which configuration is applied

The environment is instantiated inside the test.

16. UVM Environment

The environment groups major verification components.

It may contain:

- Agent
- Scoreboard
- Coverage collector
- Reference model

The environment connects different verification components together.

17. UVM Agent

An agent is a protocol-specific verification component.

It usually contains:

- Sequencer
- Driver
- Monitor

The agent communicates with the DUT.

The agent is designed according to the protocol being verified.

For example, if the DUT uses a specific bus protocol, the agent is built to generate, drive, and monitor that protocol.

18. Driver

The driver converts transaction-level items into pin-level signals.

It receives transactions from the sequencer and drives them to the DUT interface.

Driver Role

- Gets sequence items
- Converts transactions into signal activity
- Drives signals to the DUT

19. Monitor

The monitor observes DUT signals.

It collects responses from the DUT and converts pin-level activity back into transaction-level information.

Monitor Role

- Observes DUT interface
- Collects signal activity
- Converts signals into transactions
- Sends observed data to scoreboard or coverage collector

20. Sequencer

The sequencer controls the flow of sequence items.

It sends transactions from sequences to the driver.

The sequencer is an important UVM component that is not normally present in a simple SystemVerilog testbench.

21. Scoreboard

The scoreboard checks correctness.

It compares:

- Expected output
- Actual DUT output

The scoreboard helps decide whether the DUT behavior is correct.

22. Coverage Collector

The coverage collector tracks which verification scenarios have been tested.

It is used to measure functional coverage.

This helps verify whether important cases have been exercised.

23. Active and Passive Agents

A UVM agent can be configured as:

1. Active agent
2. Passive agent

This configurability is an important feature of UVM.

24. Active Agent

An active agent contains:

- Sequencer
- Driver
- Monitor

It can generate and drive stimulus to the DUT.

Active Agent Use

Use an active agent when the verification environment must control and drive the DUT interface.

25. Passive Agent

A passive agent contains only:

- Monitor

In passive mode, the sequencer and driver are disabled.

Passive Agent Use

Use a passive agent when the environment only needs to observe the DUT interface without driving it.

26. Agent Configurability

The same agent can be configured as active or passive during runtime.

This is useful because the same agent can be reused in different verification environments.

For example:

- In one testbench, the agent may actively drive stimulus.
- In another testbench, the same agent may only monitor activity.

27. Key Points to Remember

- UVM stands for Universal Verification Methodology.

- UVM is not a language; it is a methodology built on SystemVerilog.
- SystemVerilog provides verification features, but not a standard methodology.
- UVM provides a standard, reusable, and scalable testbench structure.
- UVM was developed by Accellera and became an IEEE standard in 2017.
- `uvm_object` is the main parent class in UVM.
- UVM has two major branches: component branch and sequence branch.
- UVM components have hierarchy and participate in UVM phases.
- UVM objects are mainly used as data containers and do not have hierarchy.
- A typical UVM testbench contains test, environment, agent, sequencer, driver, monitor, scoreboard, and coverage collector.
- An agent is protocol-specific.
- An active agent has sequencer, driver, and monitor.
- A passive agent has only a monitor.
- The driver converts transactions into pin-level signals.
- The monitor collects DUT activity and converts it into transactions.
- The scoreboard checks correctness.
- The coverage collector tracks tested scenarios.

28. Final Summary

UVM provides a standardized way to build verification testbenches using SystemVerilog.

It improves:

- Reusability
- Scalability
- Modularity
- Debugging
- Coverage collection
- Testbench structure

The main idea of UVM is to build verification environments using reusable components such as agents, drivers, monitors, sequencers, scoreboards, and coverage collectors.

This makes UVM suitable for verifying complex digital designs.

Need for UVM Factory

Why We Need UVM Factory

1. Topic Overview

This section explains why the **UVM factory** is needed.

The main idea is to understand how one packet or transaction class can be replaced by another without manually changing the testbench code everywhere.

2. Basic SystemVerilog Testbench Setup

Consider a simple SystemVerilog testbench with the following structure:

- Test
- Environment
- Generator
- Driver
- Mailbox
- DUT

The generator creates packets and sends them to the driver through a mailbox.

The driver receives the packet and drives the corresponding stimulus to the DUT.

In this example, the monitor is not considered.

3. Initial Packet Requirement

Assume there is a transaction class called:

`write_txn`

This class contains:

- `data` field
- Data width: 8 bits

Example:

```
logic [7:0] data;
```

The driver uses this packet to drive 8-bit data to the DUT.

4. New Packet Requirement

Now assume there is another transaction class called:

`write_txn2`

This class contains:

- `data` field
- Data width: 16 bits

Example:

```
logic [15:0] data;
```

The requirement is:

- First, the driver should drive the original 8-bit packet.
- Later, the driver should drive the new 16-bit packet.

This creates the need to replace one transaction type with another.

5. Original SystemVerilog Implementation

The basic implementation contains the following classes:

write_txn Class

This is the base transaction class.

It contains:

- Random 8-bit data field
- Display function to print the transaction data

Generator Class

The generator contains:

- A `write_txn` handle
- A mailbox handle
- A constructor to receive the mailbox handle
- A `send_packet()` task

The `send_packet()` task:

1. Randomizes the transaction.
2. Displays the transaction data.
3. Sends the transaction to the driver through the mailbox.

Driver Class

The driver contains:

- A mailbox handle
- A constructor to receive the mailbox handle
- A `drive_packet()` task

The `drive_packet()` task:

1. Gets the transaction from the mailbox.
2. Displays the received transaction.
3. Drives the packet to the DUT.

The actual DUT driving logic is not included in this example.

Environment Class

The environment contains:

- Generator handle
- Driver handle
- Mailbox handle

Inside the constructor:

- Mailbox object is created.
- Generator object is created.
- Driver object is created.
- The same mailbox is passed to both generator and driver.

The environment also contains a `run()` task.

The `run()` task:

1. Calls `send_packet()` from the generator.
2. Calls `drive_packet()` from the driver.

Test Module

The test module:

1. Creates the environment object.
2. Calls the environment `run()` task.
3. Prints a test completion message.

6. Initial Output

When the original `write_txn` packet is used, the output shows the base transaction data.

Example output:

- Base transaction data is displayed.
- Test completed message is displayed.

This confirms that the generator successfully sends the transaction and the driver receives it.

7. Replacing the Packet Manually

To use the new transaction class `write_txn2`, one direct method is to replace every occurrence of `write_txn` with `write_txn2`.

However, this is not a good approach because:

- Code must be edited manually.
- Multiple files may need changes.
- It is error-prone.
- It reduces reusability.
- It is not scalable for large verification environments.

8. Using Inheritance

A better SystemVerilog approach is to derive the new transaction class from the original transaction class.

Example concept:

```
write_txn2 extends write_txn
```

Here:

- `write_txn` is the base class.
- `write_txn2` is the derived class.

The derived class can modify or extend the behaviour of the base class.

In this example:

- `write_txn` contains 8-bit data.
- `write_txn2` contains 16-bit data.
- `write_txn2` has its own display function.

9. Passing Transaction Handles

To allow flexibility, the transaction handle can be passed through constructors.

The generator constructor can receive:

- Mailbox handle
- Transaction handle

The environment constructor can also receive a transaction handle and pass it to the generator.

The test creates the required transaction object and passes it into the environment.

This allows the test to decide which transaction object should be used.

10. Method Overriding

To override the display function of the base class, the base class function must be declared as `virtual`.

Without `virtual`, the base class display function is called.

With `virtual`, the derived class display function is called when a derived object is passed through a base class handle.

This enables runtime polymorphism.

11. Improved Inheritance-Based Method

Using inheritance and virtual methods:

1. Create base class `write_txn`.
2. Create derived class `write_txn2 extends write_txn`.
3. Declare the base class display method as `virtual`.
4. Create a `write_txn2` object in the test.
5. Pass it through the environment to the generator.
6. The generator and driver can now work with the derived transaction.
7. The derived display function is called.

This allows one transaction type to be replaced by another using object-oriented programming.

12. Limitation of Manual Inheritance-Based Replacement

Even with inheritance, some manual work is still required.

The user may need to:

- Add new handles
- Create new objects
- Modify constructors
- Pass objects manually
- Change testbench code
- Carefully manage base and derived class handles

This is still not ideal for a large verification environment.

13. Why UVM Factory Is Needed

The UVM factory solves this problem.

Using the UVM factory, one class can be replaced with another class without manually changing the testbench code everywhere.

For example:

- `write_txn` can be overridden by `write_txn2`.

This can be done during runtime using factory override.

14. Main Advantage of UVM Factory

The UVM factory allows:

- Runtime class replacement
- Better reusability
- Less manual code modification

- Easier test customization
- Scalable verification environment
- Cleaner object creation
- Flexible packet or component override

Instead of directly creating objects using `new`, UVM uses the factory-based `create()` method.

This allows UVM to decide which class object should actually be created.

15. Factory Override Concept

Factory override means replacing one class type with another.

Example concept:

- Original class: `write_txn`
- Override class: `write_txn2`

When the testbench asks for a `write_txn` object, the factory can create a `write_txn2` object instead.

This avoids changing the driver, generator, or environment code manually.

16. Compile-Time vs Runtime Replacement

Manual SystemVerilog Method

In normal SystemVerilog:

- Class replacement usually requires code edits.
- Changes are mostly compile-time changes.
- The user must manually update the testbench.

UVM Factory Method

In UVM:

- Class replacement can be configured at runtime.
- The original code does not need to be edited everywhere.
- Overrides can be controlled from the test.

This is one of the main reasons UVM factory is powerful.

17. Key Points to Remember

- The generator creates packets and sends them to the driver through a mailbox.
- The driver receives packets and drives stimulus to the DUT.
- `write_txn` contains 8-bit data.
- `write_txn2` contains 16-bit data.
- Manually replacing `write_txn` with `write_txn2` is not scalable.
- Inheritance allows `write_txn2` to extend `write_txn`.
- Virtual methods are required for proper method overriding.

- Passing derived objects through base class handles enables polymorphism.
- Manual object passing still requires code changes.
- UVM factory avoids manual replacement.
- UVM factory allows one class to be overridden by another.
- Factory overrides are useful for runtime test customization.
- UVM factory improves reusability, flexibility, and scalability.

18. Final Summary

In a normal SystemVerilog testbench, replacing one packet type with another requires manual code changes or extra object-passing logic.

Using inheritance and virtual methods improves flexibility, but it still needs manual changes in the testbench.

The UVM factory provides a cleaner solution. It allows the testbench to request one class and create another overridden class instead.

This makes verification environments more reusable, configurable, and scalable.

UVM Factory

Introduction to UVM Factory

1. Topic Overview

This section explains the **UVM factory**, including:

- What the UVM factory is
- Why it is needed
- Factory registration
- Object and component creation
- Factory overrides
- Global override
- Instance override

The previous topic explained why factory-based creation is useful compared to manual object creation in SystemVerilog.

2. What Is UVM Factory?

The **UVM factory** is a core feature of UVM.

It is responsible for dynamically creating:

- UVM objects
- UVM components

Instead of directly creating objects using the traditional SystemVerilog `new` method, UVM uses the factory `create()` method.

This provides:

- Flexibility
- Reusability
- Configurability
- Runtime replacement
- Polymorphism support
- Factory override support

3. Why UVM Factory Is Needed

In a traditional SystemVerilog testbench, objects are usually created using:

`new`

This approach has limitations in complex verification environments because:

- Object replacement requires manual code changes.
- Testbench flexibility is reduced.
- Reuse becomes difficult.

- Large environments become hard to maintain.

The UVM factory solves these issues by allowing objects and components to be created dynamically.

4. Benefits of UVM Factory

The UVM factory helps in:

- Creating reusable testbenches
- Replacing one class with another without editing source code
- Supporting polymorphism
- Enabling runtime factory overrides
- Building configurable verification environments
- Improving scalability for complex designs

5. Factory Registration

Before a class can be created or overridden using the UVM factory, it must be registered with the factory.

This applies to both:

- UVM component classes
- UVM object classes

Factory registration is mandatory for factory-based creation and overriding.

6. Registering a UVM Component Manually

Consider a user-defined component class:

```
my_component extends uvm_component
```

To register this component with the factory, UVM provides the class:

```
uvm_component_registry
```

A manual registration method uses:

```
typedef uvm_component_registry #(my_component) type_id;
```

This creates a factory registration type called `type_id`.

A static function called `get_type()` is also used to return the factory registry instance.

Conceptually:

typedef uvm_component_registry #(my_component) type_id;

static function type_id get_type();
  return type_id::get();
endfunction

Here:

- `type_id` is a wrapper around the factory registration class.
- `get_type()` returns the factory registration instance.
- `::` is the scope resolution operator used to call static methods.
- `type_id::get()` gets the singleton factory registry instance.

7. Using UVM Registration Macros

Instead of writing manual registration code, UVM provides macros.

Macros make factory registration shorter and cleaner.

8. Component Registration Macros

For a normal UVM component class:

`uvm_component_utils(my_component)

For a parameterized component class:

`uvm_component_param_utils(my_component_t)

If the parameterized class name is long, a `typedef` can be used first.

Example concept:

typedef my_param_component #(ADDR_WIDTH, DATA_WIDTH) my_component_t;
`uvm_component_param_utils(my_component_t)

This avoids repeatedly writing the full parameterized class name.

9. Object Registration Macros

For a normal UVM object class:

`uvm_object_utils(my_item)

For a parameterized UVM object class:

`uvm_object_param_utils(my_item_t)

Objects such as sequence items and transactions should also be registered if they need factory-based creation or overriding.

10. Component and Object Constructors

UVM components and objects must follow standard constructor prototypes.

This allows the factory to create them correctly.

11. UVM Component Constructor

A UVM component constructor usually has two arguments:

function new(string name = "my_component", uvm_component parent = null);

The arguments are:

- `name` – component name
- `parent` – parent component in the UVM hierarchy

Components have hierarchy, so they require a parent argument.

12. UVM Object Constructor

A UVM object constructor usually has only one argument:

function new(string name = "my_object");

The argument is:

- `name` – object name

Objects do not have hierarchy, so they do not require a parent argument.

13. Deferred Construction

UVM supports deferred construction.

This means objects or components are not necessarily constructed immediately when declared.

They are created when required, commonly during the UVM build phase.

To support this, constructors should use default arguments.

14. Factory-Based Creation

In UVM, instances are created using the factory `create()` method instead of directly calling `new`.

The `create()` method allows UVM to decide which class should actually be constructed.

This is important for factory overrides.

15. Creating a UVM Component

Example concept:

my_driver drv;

drv = my_driver::type_id::create("drv", this);

Here:

- `my_driver` is the class name.
- `drv` is the handle.
- `type_id` is created during factory registration.
- `create()` creates the component instance.
- `"drv"` is the instance name.
- `this` refers to the parent component.

This kind of code is commonly written inside another component such as an agent.

For example, an agent creates its driver, monitor, and sequencer using the factory.

16. Creating a UVM Object

Example concept:

my_sequence_item tx;

tx = my_sequence_item::type_id::create("tx");

Here:

- `my_sequence_item` is the object class.
- `tx` is the handle.
- `create()` creates the object instance.
- Objects do not require a parent argument.

17. Why create() Is Used Instead of new

Using `new` directly creates the exact class written in the code.

Using `create()` allows the factory to check whether any override exists.

If an override exists, the factory creates the substitute class instead of the original class.

This enables runtime flexibility.

18. Factory Override

Factory override means replacing one registered class with another registered class.

Example idea:

- Original class: `base_driver`
- Substitute class: `driver_1`

When the testbench requests `base_driver`, the factory can create `driver_1` instead.

This replacement can happen without changing the source code where the object or component is created.

19. Types of Factory Override

There are two main types of factory override:

1. Global override
2. Instance override

20. Global Override

A global override replaces all instances of a particular type.

If `base_driver` is globally overridden by `driver_1`, then every place that requests `base_driver` will receive `driver_1`.

Global Override Syntax

```
set_type_override_by_type(
  original_type::get_type(),
  substitute_type::get_type(),
  replace
);
```

Example concept:

```
base_driver::type_id::set_type_override(
  driver_1::get_type()
);
```

Or using factory-style syntax:

```
factory.set_type_override_by_type(
  base_driver::get_type(),
  driver_1::get_type(),
  1
);
```

Here:

- `original_type` is the class to be replaced.
- `substitute_type` is the class used as replacement.
- `get_type()` returns the factory registry instance.
- `replace` decides whether an existing override can be replaced.

21. Instance Override

An instance override replaces only a specific instance of a class.

This is useful when the same class is used in multiple places, but only one specific instance must be replaced.

Example:

- `agent1` has `base_driver`
- `agent2` also has `base_driver`

If only the driver inside `agent1` needs to be replaced, instance override is used.

22. Instance Override Syntax

```
set_inst_override_by_type(
  original_type::get_type(),
  substitute_type::get_type(),
  "path.to.instance"
);
```

Example concept:

```
factory.set_inst_override_by_type(
  base_driver::get_type(),
  driver_1::get_type(),
  "env.agent_h.my_driver"
);
```

Here:

- `base_driver` is the original type.
- `driver_1` is the substitute type.
- `"env.agent_h.my_driver"` is the hierarchical path of the specific instance to override.

23. Difference Between Global and Instance Override

Override Type	Meaning
Global override	Replaces all instances of a type
Instance override	Replaces only a specific instance at a given hierarchy path

24. Example Override Scenario

Assume there is a base agent class.

From it, two agents are derived:

- `agent1`
- `agent2`

A new class called `child_agent` is derived from `agent1`.

Possible override cases:

- Replace only `agent1` with `child_agent`
- Replace `agent1` and `agent2` with `child_agent`
- Replace a specific instance of `agent1` with `child_agent`

Global override is used when all matching types must be replaced.

Instance override is used when only one selected instance must be replaced.

25. get_type() Function

The `get_type()` function returns the factory registry instance of a class.

This instance is used by the factory to identify the class during creation and override.

The registry instance is a singleton, meaning each registered class has one factory registration instance.

26. replace Argument

Factory override methods can include a `replace` argument.

This is usually a bit value.

If replace = 1

- Existing override can be replaced.
- New override takes effect.

If replace = 0

- Existing override is not replaced.
- If an override already exists, the new override is ignored.

27. Important Rules for Factory Override

To use factory override correctly:

- The original class must be registered with the factory.
- The substitute class must also be registered with the factory.
- Objects and components should be created using `create()`, not `new`.
- The substitute class should be compatible with the original class.
- For instance override, the correct hierarchy path must be specified.

28. Key Points to Remember

- UVM factory dynamically creates objects and components.
- Factory creation uses `create()` instead of `new`.
- Classes must be registered before using factory features.
- Registration can be done manually or using macros.
- Components use `uvm_component_utils`.
- Objects use `uvm_object_utils`.
- Parameterized classes use parameterized registration macros.
- Components have constructors with `name` and `parent`.
- Objects have constructors with only `name`.
- Factory override allows one class to be replaced by another.
- Global override replaces all instances of a type.
- Instance override replaces only one specific instance.
- `get_type()` returns the factory registration instance.
- The `replace` bit controls whether previous overrides can be replaced.

29. Final Summary

The UVM factory provides a standard and flexible way to create verification components and objects.

Instead of directly using `new`, UVM classes are registered with the factory and created using `create()`.

This allows the testbench to replace one class with another through factory overrides.

Global override replaces all instances of a type, while instance override replaces only a selected instance using its hierarchy path.

This makes UVM testbenches more reusable, configurable, and scalable.

UVM Factory Override

UVM Factory Override with Coding Example

1. Topic Overview

This section explains a practical example of **UVM factory override**.

The example shows how to override:

- A base agent with a child agent
- A base driver with a child driver
- A specific driver instance using instance override
- Driver selection using compile-time defines

2. Factory Override Goal

Assume a UVM environment contains:

- `base_agent`
- `child_agent`
- `base_driver`
- `driver_one`
- `driver_two`

The goal is to replace one class with another using the UVM factory.

Examples:

- Replace `base_agent` with `child_agent`
- Replace `base_driver` with `driver_one`
- Replace `base_driver` with `driver_two`
- Replace only a specific instance of `base_driver`

3. EDA Playground Setup

When running UVM code in EDA Playground:

- Select the **UVM** option.
- Include the UVM macro file.
- Import the UVM package.

Required setup:

```
`include "uvm_macros.svh"
import uvm_pkg::*;
```

4. Base Agent Class

Create a class called `base_agent`.

It extends:

uvm_agent

The class must be registered with the UVM factory.

`uvm_component_utils(base_agent)

The constructor should follow the standard UVM component constructor format:

```
function new(string name = "base_agent", uvm_component parent = null);
  super.new(name, parent);
endfunction
```

5. Child Agent Class

Create another class called `child_agent`.

It extends:

base_agent

This means `child_agent` is derived from `base_agent`.

It must also be registered with the UVM factory:

`uvm_component_utils(child_agent)

Its constructor also calls the parent constructor:

```
function new(string name = "child_agent", uvm_component parent = null);
  super.new(name, parent);
endfunction
```

6. Environment Class

Create an environment class called `environment`.

It extends:

uvm_env

Register it with the factory:

`uvm_component_utils(environment)

The environment contains a handle for the base agent:

base_agent b_agent_h;

The agent instance is created inside the `build_phase()` using the factory `create()` method:

```
function void build_phase(uvm_phase phase);
  super.build_phase(phase);

  b_agent_h = base_agent::type_id::create("b_agent_h", this);
endfunction
```

This is important because factory overrides work only when objects/components are created using `create()`, not directly using `new`.

7. Test Class

Create a test class called `test`.

It extends:

uvm_test

Register it with the factory:

`uvm_component_utils(test)

The test contains:

- Environment handle
- UVM factory handle

```
environment env_h;
uvm_factory factory;
```

The environment is created in the `build_phase()`:

env_h = environment::type_id::create("env_h", this);

The factory singleton instance is obtained using:

factory = uvm_factory::get();

8. Global Override of Agent

To override `base_agent` with `child_agent`, use global type override.

```
set_type_override_by_type(
  base_agent::get_type(),
  child_agent::get_type()
);
```

This means whenever the factory is asked to create `base_agent`, it creates `child_agent` instead.

9. Printing Factory Information

The factory contents and override information can be printed using:

```
factory.print();
```

The factory print output shows:

- Registered classes
- Requested type
- Override type

Example interpretation:

- Requested type: `base_agent`
- Override type: `child_agent`

This confirms that `base_agent` is overridden by `child_agent`.

10. Top Module

The top module calls `run_test()`.

```
module top;
  initial begin
    run_test("test");
  end
endmodule
```

`run_test()` starts the UVM simulation and executes the UVM phases.

11. Base Driver Class

Create a driver class called `base_driver`.

It extends:

uvm_driver

Register it with the factory:

`uvm_component_utils(base_driver)

The constructor is:

```
function new(string name = "base_driver", uvm_component parent = null);
  super.new(name, parent);
endfunction
```

12. Driver One Class

Create another driver class called `driver_one`.

It extends:

base_driver

Register it with the factory:

`uvm_component_utils(driver_one)

The constructor is:

```
function new(string name = "driver_one", uvm_component parent = null);
  super.new(name, parent);
endfunction
```

13. Creating Driver Inside Base Agent

The `base_agent` contains a handle for `base_driver`.

base_driver bdh;

The driver is created inside the agent's `build_phase()`:

```
function void build_phase(uvm_phase phase);
  super.build_phase(phase);

  bdh = base_driver::type_id::create("bdh", this);
endfunction
```

Since the driver is created using `create()`, it can be overridden using the factory.

14. Global Override of Driver

To override `base_driver` with `driver_one`, use:

```
set_type_override_by_type(
  base_driver::get_type(),
  driver_one::get_type()
);
```

This globally replaces every requested `base_driver` with `driver_one`.

The factory print output shows:

- Requested type: `base_driver`
- Override type: `driver_one`

This confirms that the override is working.

15. Instance Override of Driver

Instance override is used when only one specific instance must be replaced.

Example path:

```
env_h.b_agent_h.bdh
```

This path points to the driver inside the agent inside the environment.

Instance override syntax:

```
set_inst_override_by_type(
  base_driver::get_type(),
  driver_one::get_type(),
  "env_h.b_agent_h.bdh"
);
```

This replaces only that specific `base_driver` instance with `driver_one`.

Other instances of `base_driver` are not affected.

16. Difference Between Global and Instance Override

Override Type	Meaning
Global override	Replaces all instances of a class type
Instance override	Replaces only one specific instance using hierarchy path

17. Adding Driver Two

Create another driver class called `driver_two`.

It extends:

base_driver

Register it with the factory:

`uvm_component_utils(driver_two)

Its constructor is similar to the previous driver classes:

```
function new(string name = "driver_two", uvm_component parent = null);
  super.new(name, parent);
endfunction
```

Now the factory can override `base_driver` with either:

- `driver_one`
- `driver_two`

18. Conditional Override Using Compile-Time Defines

Compile-time defines can be used to select which driver override should be applied.

Example:

```
`ifdef DRV1
  set_type_override_by_type(
    base_driver::get_type(),
    driver_one::get_type()
  );
`elsif DRV2
  set_type_override_by_type(
    base_driver::get_type(),
    driver_two::get_type()
  );
`endif
```

This allows the user to choose the override from compile options.

19. Compile Options

To select `driver_one`, use:

+define+DRV1

To select `driver_two`, use:

+define+DRV2

If `DRV1` is defined, `base_driver` is replaced by `driver_one`.

If `DRV2` is defined, `base_driver` is replaced by `driver_two`.

20. Important Rules

Factory override works correctly only when:

- Classes are registered with the UVM factory.
- Components are created using `type_id::create()`.
- The original and override classes are compatible.
- The override is applied before the component is created.
- For instance override, the correct hierarchy path is used.

21. Key Points to Remember

- `base_agent` can be overridden by `child_agent`.
- `base_driver` can be overridden by `driver_one` or `driver_two`.
- `uvm_component_utils` registers classes with the factory.
- `type_id::create()` enables factory-based construction.
- `uvm_factory::get()` returns the factory singleton instance.
- `factory.print()` displays registered classes and overrides.
- `set_type_override_by_type()` performs global override.
- `set_inst_override_by_type()` performs instance override.
- `run_test("test")` starts the UVM test.
- Compile-time defines can select which override is applied.

22. Final Summary

This example demonstrates how UVM factory override is applied in practical code.

First, classes such as agents, drivers, environment, and test are registered with the factory. Then, components are created using the factory `create()` method.

A global override replaces all instances of a class type. An instance override replaces only a specific object in the UVM hierarchy.

By using compile-time defines such as `DRV1` and `DRV2`, different driver implementations can be selected without changing the main testbench code.

This makes the UVM testbench more reusable, flexible, and configurable.

UVM copy() vs clone()

UVM `copy()` vs `clone()`

1. Topic Overview

This section explains two important UVM methods:

- `copy()`
- `clone()`

Both methods are used to copy data from one UVM object to another.

The example uses a transaction class derived from `uvm_object`.

2. What Is `copy()`?

`copy()` is a public virtual function of the `uvm_object` base class.

It is used to copy the contents of one object into another existing object.

Example:

t_h2.copy(t_h1);

Here:

- `t_h1` is the source object.
- `t_h2` is the destination object.
- Data from `t_h1` is copied into `t_h2`.

3. Role of `copy()`

The `copy()` method:

1. Is called by the user or external code.
2. Checks whether the source object is of the correct type.
3. Internally calls `do_copy()`.
4. Uses `do_copy()` to perform the actual field-by-field copy.

So, `copy()` starts the copying process, but `do_copy()` defines what exactly gets copied.

4. What Is `do_copy()`?

`do_copy()` is a protected virtual function.

It must be overridden by the user inside the transaction class.

The actual copying logic is written inside `do_copy()`.

Example role:

- Copy address field
- Copy data field
- Copy any other class members

Without overriding `do_copy()`, the user-defined fields are not copied.

5. Example Transaction Class

A transaction class is created by extending `uvm_object`.

class transaction extends uvm_object;

The class contains two fields:

bit [3:0] addr;
bit [7:0] data;

The class is registered with the UVM factory using:

`uvm_object_utils(transaction)

The constructor is written as:

```
function new(string name = "transaction");
  super.new(name);
endfunction
```

A display function is also added to print the values of `addr` and `data`.

6. Test Module Setup

Inside the test module, two transaction handles are declared:

transaction t_h1, t_h2;

Objects are created using factory creation:

```
t_h1 = transaction::type_id::create("t_h1");
t_h2 = transaction::type_id::create("t_h2");
```

Values are assigned to the first transaction:

```
t_h1.addr = 10;
t_h1.data = 50;
```

Then `copy()` is called:

t_h2.copy(t_h1);

The display function is called for both handles.

7. Behaviour Without `do_copy()`

If `do_copy()` is not written, the copy operation does not copy the user-defined fields.

Output result:

- `t_h1.addr = 10`
- `t_h1.data = 50`
- `t_h2.addr = 0`
- `t_h2.data = 0`

This shows that `copy()` alone is not enough for user-defined fields.

The user must override `do_copy()`.

8. Writing `do_copy()`

The `do_copy()` function receives an argument of type `uvm_object`.

Example:

virtual function void do_copy(uvm_object rhs);

Here:

- `rhs` is the source object passed to `copy()`.
- In `t_h2.copy(t_h1)`, `t_h1` is passed as `rhs`.

Since `rhs` is of type `uvm_object`, it cannot directly access transaction fields such as `addr` and `data`.

Therefore, it must be type-cast to the transaction class.

9. Using `$cast` in `do_copy()`

A local transaction handle is created:

transaction t_h;

Then `$cast` is used:

$cast(t_h, rhs);

After casting, fields can be copied:

this.addr = t_h.addr;
this.data = t_h.data;

So the complete idea is:

```
virtual function void do_copy(uvm_object rhs);
  transaction t_h;

  $cast(t_h, rhs);

  this.addr = t_h.addr;
  this.data = t_h.data;
endfunction
```

10. Behaviour With `do_copy()`

After overriding `do_copy()`, the copy works correctly.

Output result:

- `t_h1.addr = 10`
- `t_h1.data = 50`
- `t_h2.addr = 10`
- `t_h2.data = 50`

Now the contents of `t_h1` are successfully copied into `t_h2`.

11. Important Point About `copy()`

The `copy()` method requires both objects to already exist.

For example:

```
t_h1 = transaction::type_id::create("t_h1");
t_h2 = transaction::type_id::create("t_h2");
```

Both object creations are required before calling:

```
t_h2.copy(t_h1);
```

If `t_h2` is not created and `copy()` is called, a null pointer error occurs.

Example issue:

```
// t_h2 object is not created
t_h2.copy(t_h1);
```

This causes an object null pointer access error.

12. What Is `clone()`?

`clone()` is another UVM method used to copy objects.

Like `copy()`, it also uses `do_copy()` internally.

The main difference is that `clone()` creates a new object automatically and copies the contents into it.

13. Using `clone()`

Example:

```
$cast(t_h2, t_h1.clone());
```

Here:

- `t_h1.clone()` creates a new object.
- It copies the contents of `t_h1`.
- `$cast` converts the returned `uvm_object` handle into a `transaction` handle.
- The copied object is assigned to `t_h2`.

14. Behaviour of `clone()`

Unlike `copy()`, `clone()` does not require the destination object to be created beforehand.

For example, even if `t_h2` is not created using `create()` or `new`, this still works:

```
$cast(t_h2, t_h1.clone());
```

After cloning:

- `t_h2` points to a new copied object.
- `t_h2.addr` contains the copied address value.
- `t_h2.data` contains the copied data value.

15. Difference Between `copy()` and `clone()`

Feature	`copy()`	`clone()`
Purpose	Copies data into an existing object	Creates a new object and copies data

Destination object required?	Yes	No
Calls `do_copy()` internally?	Yes	Yes
Null destination issue?	Yes, if destination is not created	No, because clone creates a new object
Typical syntax	`t_h2.copy(t_h1);`	`$cast(t_h2, t_h1.clone());`

16. Key Points to Remember

- `copy()` is used to copy one object's contents into another existing object.
- `copy()` is a public virtual function from `uvm_object`.
- `copy()` internally calls `do_copy()`.
- `do_copy()` must be overridden by the user.
- Actual field-by-field copy logic is written inside `do_copy()`.
- Without `do_copy()`, user-defined fields are not copied.
- Since `rhs` is of type `uvm_object`, `$cast` is needed before accessing fields.
- `copy()` requires both source and destination objects to be created.
- If the destination object is null, `copy()` causes a null pointer error.
- `clone()` creates a new object automatically.
- `clone()` also calls `do_copy()` internally.
- `clone()` does not require the destination object to be created beforehand.
- `$cast` is commonly used with `clone()` because `clone()` returns a `uvm_object` handle.

17. Final Summary

`copy()` and `clone()` are used to duplicate UVM objects.

Use `copy()` when the destination object already exists.

Use `clone()` when a new copied object should be created automatically.

In both cases, user-defined fields are copied correctly only when `do_copy()` is properly overridden.

UVM compare() and print()

UVM `compare()` vs `do_compare()` and `print()` vs `do_print()`

1. Topic Overview

This section explains important UVM methods used for object comparison and printing:

- `compare()`
- `do_compare()`
- `print()`
- `do_print()`
- `sprint()`

It also explains the difference between:

- `$display`
- `$sformatf`

These concepts are commonly used when working with UVM objects such as packets, transactions, and sequence items.

2. UVM `compare()` Method

The `compare()` method is used to compare two UVM objects field by field.

It is declared in the `uvm_object` base class.

Methods such as `copy()`, `clone()`, and `compare()` are all available through `uvm_object`.

Syntax

```
virtual function bit compare(
  uvm_object rhs,
  uvm_comparer comparer = null
);
```

Arguments

- `rhs` is the object to compare with the current object.
- `comparer` is a handle of type `uvm_comparer`.
- By default, `comparer` is `null`.

Return Value

- Returns `1` if both objects are equal.

- Returns `0` if both objects are not equal.

3. Relationship Between `compare()` and `do_compare()`

The `compare()` method internally calls `do_compare()`.

This is similar to how:

- `copy()` internally calls `do_copy()`
- `print()` internally calls `do_print()`

The user must define which fields should be compared inside `do_compare()`.

If the required fields are not included in `do_compare()`, those fields will not be compared.

4. Need for `do_compare()`

Assume a class has two fields:

int a;
int b;

If two objects of this class are compared, UVM will compare only the fields written inside `do_compare()`.

For example:

- If only `a` is compared, then differences in `b` are ignored.
- If both `a` and `b` are compared, then both fields affect the comparison result.
- If no comparison logic is written, the comparison may fail or not behave as expected.

5. Example Class for Comparison

A class called `base_packet` is created by extending `uvm_object`.

class base_packet extends uvm_object;

The class is registered with the UVM factory:

`uvm_object_utils(base_packet)

It contains two integer fields:

int a;
int b;

The constructor is written as:

```
function new(string name = "base_packet");
  super.new(name);
endfunction
```

6. Writing `do_compare()`

The `do_compare()` method should be written inside the class.

```
function bit do_compare(
  uvm_object rhs,
  uvm_comparer comparer
);
```

Since `rhs` is of type `uvm_object`, it cannot directly access fields such as `a` and `b`.

So, it must be cast to the correct class type.

```
base_packet bph;

if (!$cast(bph, rhs))
  return 0;
```

After casting, fields can be compared.

```
return ((this.a == bph.a) && (this.b == bph.b));
```

7. Complete `do_compare()` Idea

```
function bit do_compare(
  uvm_object rhs,
  uvm_comparer comparer
);
  base_packet bph;

  if (!$cast(bph, rhs))
    return 0;

  return ((this.a == bph.a) && (this.b == bph.b));
endfunction
```

8. Test Setup for Comparison

Inside the top module, two handles are created:

```
base_packet bph1;
base_packet bph2;
```

Objects are created using factory creation:

```
bph1 = base_packet::type_id::create("bph1");
bph2 = base_packet::type_id::create("bph2");
```

Values are assigned:

```
bph1.a = 10;
bph1.b = 20;

bph2.a = 10;
bph2.b = 30;
```

Then comparison is performed:

```
if (bph1.compare(bph2))
  $display("Comparison success");
else
  $display("Comparison failed");
```

9. Comparison Result

In this example:

```
bph1.a = 10;
bph2.a = 10;

bph1.b = 20;
bph2.b = 30;
```

If both `a` and `b` are compared:

- `a` matches
- `b` does not match

So the result is:

Comparison failed

If only `a` is compared inside `do_compare()`, then the comparison succeeds because both objects have the same value for `a`.

10. Important Points About `compare()`

- `compare()` compares two UVM objects.
- It returns `1` for match and `0` for mismatch.
- It internally calls `do_compare()`.

- The user must write comparison logic inside `do_compare()`.
- Only the fields mentioned in `do_compare()` are compared.
- `$cast` is needed because `rhs` is received as a `uvm_object`.

11. `$display` vs `$sformatf`

Before understanding UVM printing methods, it is useful to know the difference between `$display` and `$sformatf`.

12. `$display`

`$display` directly prints formatted text to the simulation console.

It behaves like `printf`.

Example:

int a = 10;

$display("The value of a is %0d", a);

Output:

The value of a is 10

Key Point

`$display` prints directly. It does not return a formatted string.

13. `$sformatf`

`$sformatf` creates and returns a formatted string.

It does not directly print to the console.

Example:

int b = 20;
string message;

message = $sformatf("The value of b is %0d", b);
$display("%s", message);

Output:

The value of b is 20

Key Point

`$sformatf` is useful when the formatted message must be stored in a string or passed to another function.

14. Difference Between `$display` and `$sformatf`

Feature	`$display`	`$sformatf`
Purpose	Prints directly to console	Returns a formatted string
Return value	No useful return value	Returns a string
Storage	Cannot store output directly	Can store output in a string variable
Use case	Direct debug printing	Formatted messages for reuse

15. UVM `print()` Method

The `print()` method is used to display the contents of a UVM object or component.

It prints to the standard simulation output or console log.

The `print()` method internally calls:

do_print()

This is similar to:

- `copy()` calling `do_copy()`
- `compare()` calling `do_compare()`

16. UVM `do_print()` Method

`do_print()` is a virtual method that should be overridden by the user.

It defines which fields should be printed and how they should be printed.

The method receives a `uvm_printer` argument.

function void do_print(uvm_printer printer);

Inside `do_print()`, the user can call:

printer.print_field();

17. `print_field()` Method

The `print_field()` method is used to print individual fields.

It expects arguments such as:

- Field name
- Field value
- Field size
- Radix/base

Example

```
printer.print_field(
  "value of a",
  a,
  $bits(a),
  UVM_DEC
);
```

Another example:

```
printer.print_field(
  "value of b",
  b,
  $bits(b),
  UVM_HEX
);
```

18. Radix Options

Common radix options include:

- `UVM_BIN` for binary
- `UVM_DEC` for decimal
- `UVM_HEX` for hexadecimal
- `UVM_OCT` for octal

19. Example `do_print()` Method

```
function void do_print(uvm_printer printer);

  printer.print_field(
    "value of a",
    a,
    $bits(a),
    UVM_DEC
  );

  printer.print_field(
    "value of b",
    b,
```

```
    $bits(b),
    UVM_HEX
  );

endfunction
```

20. Calling `print()`

The `print()` method can be called using the object handle.

```
bph1.print();
bph2.print();
```

By default, UVM prints the object in table format.

The output shows details such as:

- Object name
- Field name
- Field type
- Field size
- Field value

21. UVM Print Formats

UVM supports different print formats.

The common printer formats are:

1. Table printer
2. Tree printer
3. Line printer

22. Table Printer

Table printer is the default format.

If no printer is specified, UVM prints in table format.

Example:

```
bph1.print();
```

The output appears in a table showing name, type, size, and value.

23. Tree Printer

Tree printer displays the object details in a tree-like format.

Example:

bph1.print(uvm_default_tree_printer);

It prints the object details first, followed by its field values in a structured tree format.

24. Line Printer

Line printer displays all object information in a single line.

Example:

bph2.print(uvm_default_line_printer);

This is useful when compact output is preferred.

25. UVM `sprint()` Method

`sprint()` is similar to `$sformatf`.

It formats object information into a string instead of printing directly to the console.

Like `print()`, it also internally calls:

do_print()

26. Difference Between `print()` and `sprint()`

Feature	`print()`	`sprint()`
Purpose	Prints object contents directly	Returns object contents as a string
Console output	Yes	No, unless the returned string is printed
Internally calls	`do_print()`	`do_print()`
Similar to	`$display`	`$sformatf`

27. Important Points About UVM Printing

- `print()` is used to directly display object contents.
- `sprint()` returns the printed contents as a formatted string.
- Both `print()` and `sprint()` internally call `do_print()`.
- The user should override `do_print()` to control which fields are printed.
- `printer.print_field()` is used to print individual fields.
- UVM supports table, tree, and line printer formats.
- Table format is the default print format.

28. Key Points to Remember

- `compare()` is used to compare two UVM objects.
- `compare()` internally calls `do_compare()`.
- `do_compare()` must include the fields that need comparison.
- `$cast` is needed inside `do_compare()` because `rhs` is a `uvm_object`.
- `$display` directly prints to the console.
- `$sformatf` returns a formatted string.
- `print()` directly prints UVM object contents.
- `sprint()` returns UVM object contents as a string.
- `print()` and `sprint()` internally call `do_print()`.
- `do_print()` defines the fields to print.
- `uvm_printer` controls print formatting.
- `uvm_default_tree_printer` prints in tree format.
- `uvm_default_line_printer` prints in line format.
- Default UVM printing uses table format.

29. Final Summary

UVM provides built-in methods for comparing and printing objects.

Use `compare()` when two UVM objects need to be checked field by field. The actual comparison logic must be written inside `do_compare()`.

Use `print()` when object contents should be directly printed to the console.

Use `sprint()` when object contents should be formatted as a string and reused later.

For printing, both `print()` and `sprint()` depend on `do_print()`, where the user defines which fields are displayed and in which format.

UVM Field Macros

UVM Field Macros

1. Topic Overview

This section explains **UVM field macros**.

Field macros provide an automatic way to perform common UVM object operations such as:

- `copy()`
- `compare()`
- `print()`
- `clone()`

They reduce the need to manually write methods like:

- `do_copy()`
- `do_compare()`
- `do_print()`

2. Why Field Macros Are Used

In previous UVM methods:

- `copy()` internally calls `do_copy()`.
- `compare()` internally calls `do_compare()`.
- `print()` internally calls `do_print()`.

If the user does not write these `do_*` methods, the required operation may not work correctly for user-defined fields.

Field macros provide an alternate method.

They automatically generate the required code for copying, comparing, and printing class properties.

3. What Are UVM Field Macros?

UVM field macros are macros used inside the UVM utility macro block.

They operate on class properties and provide automatic implementation of common UVM methods.

They are placed between:

`uvm_object_utils_begin(class_name)
  // field macros
`uvm_object_utils_end

Field macros are called “field macros” because they operate on fields or properties of a class.

4. Benefit of Field Macros

Field macros help the user avoid manually writing:

- `do_copy()`
- `do_compare()`
- `do_print()`

This saves time, especially for classes with many fields.

5. Important Limitation

Although field macros are convenient, they are generally not recommended for high-performance environments.

Reason:

- Field macros expand into general-purpose code.
- This can affect simulation performance.
- Manually writing `do_copy()`, `do_compare()`, and `do_print()` gives better control and efficiency.

So, field macros are useful for learning and quick development, but manual methods are preferred in serious projects.

6. Field Macro Selection Based on Data Type

The field macro must match the data type of the variable.

Examples:

Variable Type	Field Macro
`int`	`uvm_field_int`
`bit`	`uvm_field_int` or suitable bit-field macro
`string`	`uvm_field_string`
Object handle	`uvm_field_object`

Example:

`uvm_field_int(data, UVM_ALL_ON)

Here, `data` is an integer-type field.

7. Field Macro Arguments

Most UVM field macros take at least two arguments:

`uvm_field_int(argument, flag)

Argument

The first argument is the variable name.

Example:

data

The variable type should match the macro being used.

Flag

The second argument is the flag.

The flag controls which operations are enabled or disabled for that field.

8. Common UVM Field Macro Flags

`UVM_ALL_ON`

Enables all supported operations for that field.

This includes operations such as:

- Copy
- Compare
- Print

`UVM_DEFAULT`

Enables the default set of operations.

It is equivalent to `UVM_ALL_ON`.

`UVM_NOCOPY`

Disables copy operation for that field.

If this flag is used, the field will not be copied during `copy()` or `clone()`.

`UVM_NOCOMPARE`

Disables compare operation for that field.

If this flag is used, the field will not be compared during `compare()`.

`UVM_NOPRINT`

Disables print operation for that field.

If this flag is used, the field will not be printed during `print()` or `sprint()`.

9. Example Packet Class

Create a class called `packet`.

It extends:

uvm_object

The class has three properties:

rand int data;
rand reg [3:0] address;
rand logic enable;

These fields are registered using field macros.

10. Registering the Packet Class with Field Macros

The packet class is registered using:

`uvm_object_utils_begin(packet)

  `uvm_field_int(data, UVM_ALL_ON)
  `uvm_field_int(address, UVM_NOCOPY)
  `uvm_field_int(enable, UVM_NOCOMPARE)

`uvm_object_utils_end

Meaning

- `data` uses `UVM_ALL_ON`, so copy, compare, and print are enabled.
- `address` uses `UVM_NOCOPY`, so it will not be copied.
- `enable` uses `UVM_NOCOMPARE`, so it will not be compared.

11. Constructor

The constructor follows the standard UVM object constructor format:

function new(string name = "packet");
  super.new(name);
endfunction

Since `packet` is a UVM object, it only needs a `name` argument.

It does not require a parent argument.

12. Display Function

A display function is written to print the values of the packet fields.

It displays:

- `data`
- `address`
- `enable`

Example idea:

```
function void display(string handle);
  $display("%s data=%0d address=%0d enable=%0d",
        handle, data, address, enable);
endfunction
```

13. Top Module Setup

Inside the top module, two packet handles are declared:

```
packet p1_h;
packet p2_h;
```

Objects are created using factory creation:

```
p1_h = packet::type_id::create("p1_h");
p2_h = packet::type_id::create("p2_h");
```

Then `p1_h` is randomized:

```
p1_h.randomize();
```

After randomization, copy is performed:

```
p2_h.copy(p1_h);
```

Finally, both packets are displayed:

```
p1_h.display("p1_h");
p2_h.display("p2_h");
```

14. Required UVM Setup

The UVM macro file must be included:

`include "uvm_macros.svh"

The UVM package must be imported:

import uvm_pkg::*;

In EDA Playground or simulator setup, the UVM library option must also be enabled.

15. Copy Behaviour in the Example

Since `data` is registered with:

`uvm_field_int(data, UVM_ALL_ON)

The `data` field is copied from `p1_h` to `p2_h`.

Since `address` is registered with:

`uvm_field_int(address, UVM_NOCOPY)

The `address` field is not copied from `p1_h` to `p2_h`.

So after calling:

p2_h.copy(p1_h);

The expected behaviour is:

- `data` is copied.
- `address` is not copied.
- `enable` is copied, unless another flag disables copy.

16. Compare Behaviour in the Example

Since `enable` is registered with:

`uvm_field_int(enable, UVM_NOCOMPARE)

The `enable` field is ignored during comparison.

So during `compare()`:

- `data` is compared.
- `address` is compared unless comparison is disabled.
- `enable` is not compared.

17. How Field Macros Control UVM Methods

Field macros allow the user to control object operations without manually writing `do_*` methods.

For each field, the selected flag decides whether that field participates in:

- Copy
- Compare
- Print

Example:

`uvm_field_int(address, UVM_NOCOPY)

This means `address` is included in field registration, but excluded from copy operation.

18. Key Points to Remember

- UVM field macros are used to register class properties.
- They provide automatic implementations for methods such as copy, compare, print, and clone.
- Field macros are written inside `uvm_object_utils_begin` and `uvm_object_utils_end`.
- The field macro must match the variable type.
- Each field macro takes a variable name and a flag.
- `UVM_ALL_ON` enables all operations.
- `UVM_DEFAULT` is equivalent to `UVM_ALL_ON`.
- `UVM_NOCOPY` disables copy for that field.
- `UVM_NOCOMPARE` disables compare for that field.
- `UVM_NOPRINT` disables print for that field.
- Field macros reduce coding effort.
- Field macros may impact simulation performance.
- Manual `do_copy()`, `do_compare()`, and `do_print()` methods are generally preferred for better control.

19. Final Summary

UVM field macros provide a quick way to automate common operations on UVM object fields.

Instead of manually writing `do_copy()`, `do_compare()`, and `do_print()`, fields can be registered using macros such as:

`uvm_field_int(data, UVM_ALL_ON)

Flags such as `UVM_NOCOPY`, `UVM_NOCOMPARE`, and `UVM_NOPRINT` allow specific operations to be disabled for selected fields.

Field macros are useful for quick implementation and learning, but for large professional verification environments, manually written `do_*` methods are usually preferred for better performance and control.

UVM Phases

UVM Phases: `build_phase`, `connect_phase`, and `end_of_elaboration_phase`

1. Topic Overview

This section explains **UVM phases**, why they are used, and how they control the execution order of a UVM testbench.

The main phases discussed are:

- `build_phase`
- `connect_phase`
- `end_of_elaboration_phase`

These phases are part of the UVM phase mechanism.

2. What Are UVM Phases?

Every UVM testbench component is derived from `uvm_component`.

Examples:

- Driver
- Monitor
- Agent
- Environment
- Scoreboard
- Test

The `uvm_component` base class already contains predefined phase methods.

These phase methods act like callback methods. The user may override them if required.

Implementing phases is optional, but they help organize and synchronize the testbench execution.

3. Why UVM Phases Are Used

UVM phases provide synchronization between all testbench components.

A component cannot move to the next phase until the current phase is completed for all components.

This ensures that all parts of the testbench execute in a proper order.

For example:

- Components are created first.
- Connections are made next.
- Final adjustments are done before simulation starts.

4. Important Points About UVM Phases

- UVM phases execute in a fixed order.
- All phases are virtual methods.
- Users can override these phase methods.
- Some phases are functions and consume zero simulation time.
- Some phases are tasks and consume simulation time.
- Tasks are used when simulation time is required.
- Functions are used when execution must happen in zero simulation time.

5. Types of UVM Phases

UVM phases can be grouped into three categories:

1. Build phases
2. Runtime phases
3. Cleanup phases

6. Build Phases

Build phases are used to construct, configure, and connect the testbench.

They are necessary for:

- Creating component objects
- Building the testbench hierarchy
- Connecting components
- Making final structural adjustments

The build-phase category includes:

- `build_phase`
- `connect_phase`
- `end_of_elaboration_phase`

7. Runtime Phases

Runtime phases consume simulation time.

They are implemented as tasks.

These phases are used when actual test execution happens, such as driving stimulus and monitoring DUT activity.

8. Cleanup Phases

Cleanup phases are used after simulation activity is completed.

They are mainly used for:

- Collecting results

- Reporting simulation status
- Final checks and summaries

9. `build_phase`

The `build_phase` is used to create testbench components.

It is a function, so it executes in zero simulation time.

Purpose of `build_phase`

The `build_phase` is used for:

- Creating component instances
- Building the testbench hierarchy
- Creating objects using the UVM factory
- Passing or receiving configuration information through the configuration database

Execution Order

`build_phase` executes from top to bottom in the hierarchy.

Order:

1. Test
2. Environment
3. Agent
4. Driver
5. Monitor

This means the parent component is built before its child components.

10. `connect_phase`

The `connect_phase` is used to connect different testbench components.

It is also a function, so it executes in zero simulation time.

Purpose of `connect_phase`

The `connect_phase` is used for:

- Connecting TLM ports
- Connecting exports
- Connecting analysis ports
- Linking components after they are created

TLM port mechanisms are discussed separately.

Execution Order

`connect_phase` executes from bottom to top in the hierarchy.

Order:

1. Driver
2. Monitor
3. Agent
4. Environment
5. Test

This means lower-level components complete their connection phase before higher-level components.

11. `end_of_elaboration_phase`

The `end_of_elaboration_phase` occurs before the simulation starts.

It is also a function and executes in zero simulation time.

Purpose of `end_of_elaboration_phase`

This phase is used for:

- Final structural checks
- Final configuration adjustments
- Final connectivity adjustments
- Printing the UVM testbench topology

Execution Order

`end_of_elaboration_phase` executes from bottom to top.

Order:

1. Driver
2. Monitor
3. Agent
4. Environment
5. Test

12. Example Testbench Hierarchy

The example testbench contains the following hierarchy:

```
test
└── environment
    └── agent
        ├── driver
        └── monitor
```

In this structure:

- The test creates the environment.
- The environment creates the agent.
- The agent creates the driver and monitor.

13. Required UVM Setup

The UVM macro file must be included:

`include "uvm_macros.svh"

The UVM package must be imported:

import uvm_pkg::*;

14. Driver Class

The driver class extends `uvm_driver`.

class driver extends uvm_driver;

It is registered with the factory:

`uvm_component_utils(driver)

The constructor follows the standard component format:

```
function new(string name = "driver", uvm_component parent = null);
  super.new(name, parent);
endfunction
```

The `build_phase` can be overridden to print a message:

```
function void build_phase(uvm_phase phase);
  super.build_phase(phase);
  `uvm_info("BUILD_PHASE", "build_phase called from driver component", UVM_LOW)
endfunction
```

15. Monitor Class

The monitor class extends `uvm_monitor`.

class monitor extends uvm_monitor;

It is registered with the factory:

`uvm_component_utils(monitor)

It also has a standard constructor and can override phases like `build_phase`, `connect_phase`, and `end_of_elaboration_phase`.

16. Agent Class

The agent class extends `uvm_agent`.

class agent extends uvm_agent;

It is registered using:

`uvm_component_utils(agent)

The agent contains handles for:

driver drv_h;
monitor mon_h;

Since the agent contains the driver and monitor, it creates them inside its `build_phase`.

drv_h = driver::type_id::create("drv_h", this);
mon_h = monitor::type_id::create("mon_h", this);

17. Environment Class

The environment class extends `uvm_env`.

class environment extends uvm_env;

It is registered using:

`uvm_component_utils(environment)

The environment contains an agent handle:

agent agent_h;

The agent is created inside the environment `build_phase`:

agent_h = agent::type_id::create("agent_h", this);

18. Test Class

The test class extends `uvm_test`.

class test extends uvm_test;

It is registered using:

`uvm_component_utils(test)

The test contains an environment handle:

environment env_h;

The environment is created inside the test `build_phase`:

env_h = environment::type_id::create("env_h", this);

19. Top Module and `run_test()`

The top module calls `run_test()` inside an initial block.

```
module top;
  initial begin
    run_test("test");
  end
endmodule
```

Purpose of `run_test()`

`run_test()` is called from the static part of the testbench.

It performs two main actions:

1. Constructs the UVM root component and testbench hierarchy.
2. Starts the UVM phase mechanism.

Without calling `run_test()`, the UVM phases will not execute.

20. Build Phase Execution Order

After running the example, the `build_phase` messages appear in this order:

```
build_phase called from test component
build_phase called from environment component
build_phase called from agent component
build_phase called from driver component
build_phase called from monitor component
```

This confirms that `build_phase` executes from top to bottom.

21. Connect Phase Template

The `connect_phase` has the following format:

```
function void connect_phase(uvm_phase phase);
  super.connect_phase(phase);
  `uvm_info("CONNECT_PHASE", "connect_phase called", UVM_LOW)
endfunction
```

In a real testbench, TLM connections are written inside this phase.

22. Connect Phase Execution Order

The `connect_phase` executes after all build phases are completed.

The observed order is:

```
connect_phase called from driver component
connect_phase called from monitor component
connect_phase called from agent component
connect_phase called from environment component
connect_phase called from test component
```

This confirms that `connect_phase` executes from bottom to top.

23. End of Elaboration Phase Template

The `end_of_elaboration_phase` has the following format:

```
function void end_of_elaboration_phase(uvm_phase phase);
  super.end_of_elaboration_phase(phase);
endfunction
```

This phase can be used to print the UVM topology.

24. Printing UVM Topology

To print the UVM topology, call:

```
uvm_top.print_topology();
```

This is usually written in the `end_of_elaboration_phase` of the top-level test class.

Example:

```
function void end_of_elaboration_phase(uvm_phase phase);
  super.end_of_elaboration_phase(phase);
  uvm_top.print_topology();
endfunction
```

25. End of Elaboration Execution Order

The `end_of_elaboration_phase` executes after the `connect_phase`.

The observed order is:

```
end_of_elaboration_phase called from driver component
end_of_elaboration_phase called from monitor component
end_of_elaboration_phase called from agent component
end_of_elaboration_phase called from environment component
end_of_elaboration_phase called from test component
```

This confirms that `end_of_elaboration_phase` also executes from bottom to top.

26. UVM Topology Output

The topology output shows the testbench hierarchy.

Example structure:

```
uvm_test_top
 └── env_h
     └── agent_h
         ├── drv_h
         └── mon_h
```

The printed topology can also show ports such as analysis ports.

27. Execution Order Summary

Phase	Type	Purpose	Execution Order
`build_phase`	Function	Create components and build hierarchy	Top to bottom
`connect_phase`	Function	Connect components using TLM	Bottom to top
`end_of_elaboration_phase`	Function	Final adjustments and topology printing	Bottom to top

28. Key Points to Remember

- UVM phases are predefined callback methods in `uvm_component`.
- Users can override phases when needed.
- Phases provide synchronization between testbench components.
- A component cannot move to the next phase until all components complete the current phase.
- `build_phase` is used to create components.
- `connect_phase` is used to connect components.
- `end_of_elaboration_phase` is used for final checks and topology printing.
- `build_phase` executes from top to bottom.
- `connect_phase` executes from bottom to top.
- `end_of_elaboration_phase` executes from bottom to top.
- Build-related phases are functions and consume zero simulation time.
- Runtime phases are tasks because they consume simulation time.
- `run_test()` starts the UVM test and phase mechanism.
- `uvm_top.print_topology()` prints the UVM hierarchy.

29. Final Summary

UVM phases organize the complete verification flow.

The `build_phase` creates the testbench hierarchy, the `connect_phase` connects the components, and the `end_of_elaboration_phase` performs final checks before simulation starts.

These phases ensure that every UVM component executes in a synchronized and predictable order.

UVM Phases Part 2: `run_phase`, Post-Run Phases, and Objections

1. Topic Overview

This section continues the discussion on **UVM phases**.

The previous section covered:

- `build_phase`
- `connect_phase`
- `end_of_elaboration_phase`

This section explains:

- `run_phase`
- Runtime sub-phases
- Reset-related phases
- Configuration phases
- Cleanup phases
- UVM objections

2. `run_phase`

The `run_phase` is a task-based UVM phase.

Since it is a task, it can consume simulation time.

The main simulation activity happens in the `run_phase`.

Typical activities include:

- Stimulus generation
- Sequence execution
- Driving transactions to the DUT
- Monitoring DUT signals
- Sending observed data to the scoreboard
- Checking DUT behaviour during simulation

3. Concurrent Execution of `run_phase`

All UVM components execute their `run_phase` concurrently.

For example:

- The sequencer starts sending sequences.
- The driver receives sequence items and drives them to the DUT.
- The monitor observes the DUT.
- The scoreboard receives monitored data and checks correctness.

These activities happen in parallel during the simulation.

4. `run_phase` Template

The `run_phase` is written as a task.

```
task run_phase(uvm_phase phase);
  super.run_phase(phase);

  `uvm_info("RUN_PHASE", "run_phase called from component", UVM_LOW)
endtask
```

The argument is:

uvm_phase phase

This phase handle is used later for objections.

5. Example `run_phase` Execution

When `run_phase` is added to different components such as:

- Test
- Environment
- Agent
- Driver
- Monitor

The earlier phases execute first:

1. `build_phase`
2. `connect_phase`
3. `end_of_elaboration_phase`

After these complete, the `run_phase` starts.

The `run_phase` messages are printed from multiple components because all components execute this phase in parallel.

6. Runtime Phases Parallel to `run_phase`

Apart from `run_phase`, UVM also has several runtime sub-phases.

These phases run in parallel with the main runtime phase flow.

The runtime phases include:

- `pre_reset_phase`
- `reset_phase`

- `post_reset_phase`
- `pre_configure_phase`
- `configure_phase`
- `post_configure_phase`
- `pre_main_phase`
- `main_phase`
- `post_main_phase`
- `pre_shutdown_phase`
- `shutdown_phase`
- `post_shutdown_phase`

These phases organize simulation activity into smaller, meaningful stages.

7. `pre_reset_phase`

The `pre_reset_phase` is used for activities that must happen before reset is applied.

It is useful when some signals must be stable before reset starts.

Example use cases:

- Power-up signal setup
- Supply-related control signals
- Clock enable setup
- Boot mode selection
- Configuration pins
- Isolation controls
- Bias controls
- Reset supervisor signals

8. Power-Up Example

Some chips require power to be present before reset is applied.

Example signal:

- `power_up`
- `VDD`

The reset line is meaningful only when the chip is powered.

So, power-related signals should be enabled before applying reset.

This type of activity is handled in `pre_reset_phase`.

9. Clock Enable Example

Some DUTs expect the clock to be running before reset starts.

In such cases:

- Clock enable must be asserted before reset.
- The DUT must receive a valid clock before reset is applied.

This setup can be done in `pre_reset_phase`.

10. Boot Mode Selection Example

Some DUTs require boot mode pins to be configured before reset.

The boot mode must be selected before reset is applied so that the DUT starts in the required mode after reset.

This can also be handled in `pre_reset_phase`.

11. `reset_phase`

The `reset_phase` is used to apply reset to the DUT.

During this phase:

- Reset is generated.
- DUT interfaces are placed in default state.
- Required reset behaviour is applied.

This phase ensures the DUT starts from a known state.

12. `post_reset_phase`

The `post_reset_phase` happens after reset is deasserted.

It is used for activities required immediately after reset.

Typical use cases:

- Wait for DUT stabilization
- Perform basic DUT initialization
- Check whether the DUT is ready
- Prepare the DUT to accept transactions

13. Configuration Phases

Configuration means setting up all required values, parameters, and connections before driving transactions to the DUT.

In UVM, configuration refers to preparing the environment or DUT before the actual test execution begins.

14. What Configuration Includes

Configuration may include:

- Setting data width
- Setting burst size
- Setting address range
- Passing virtual interfaces
- Setting environment parameters
- Preparing DUT/testbench settings before transactions begin

These activities can be handled in configuration-related phases.

15. Configuration-Related Phases

The configuration phase group includes:

- `pre_configure_phase`
- `configure_phase`
- `post_configure_phase`

These phases help organize setup activity before the main test starts.

16. Cleanup Phases

Cleanup phases are executed after the runtime phases are completed.

They are used to collect and report simulation results.

Cleanup phases are useful for:

- Collecting functional coverage information
- Checking scoreboard results
- Verifying whether coverage goals are reached
- Determining whether the test passed or failed
- Reporting final simulation status

17. Cleanup Phase List

The cleanup phases are:

- `extract_phase`
- `check_phase`
- `report_phase`
- `final_phase`

These phases are functions, so they execute in zero simulation time.

18. Cleanup Phase Execution Order

The cleanup phases generally execute from bottom to top.

However:

- `final_phase` executes from top to bottom.

These phases are usually explored in detail while building a complete UVM project.

19. UVM Objections

UVM objections are used to control phase completion.

They provide a mechanism for coordinating status between multiple components or objects.

The UVM objection class is derived from:

uvm_report_object

Objections are mainly used in time-consuming phases such as `run_phase`.

20. Why Objections Are Needed

In UVM, different components execute the `run_phase` concurrently.

Example:

- Driver is running its `run_phase`.
- Agent is also running its `run_phase`.

If the driver still has work to do, the simulation should not end early.

The driver can raise an objection to tell UVM:

“I am still active. Do not end this phase yet.”

Once the work is complete, the driver drops the objection.

21. Objection Rule

A UVM phase ends only when all raised objections are dropped.

If any component raises an objection and does not drop it, the phase will not end.

This ensures that simulation does not finish before all important activities are completed.

22. Raising an Objection

An objection is raised using:

phase.raise_objection(this);

Optional arguments can also be passed, such as:

- Object handle
- Description string
- Count

The description and count are optional.

23. Dropping an Objection

After the component completes its work, the objection must be dropped using:

phase.drop_objection(this);

This tells UVM that the component has finished its phase activity.

24. Objection Example in Driver

Inside the driver `run_phase`:

```
task run_phase(uvm_phase phase);
  super.run_phase(phase);

  phase.raise_objection(this);
  `uvm_info("RUN_PHASE", "run_phase objection raised from driver", UVM_LOW)

  #10;

  phase.drop_objection(this);
  `uvm_info("RUN_PHASE", "run_phase objection dropped from driver", UVM_LOW)
endtask
```

Here:

- Driver raises an objection.
- Driver waits for 10 time units.
- Driver drops the objection.

25. Objection Example in Monitor

Inside the monitor `run_phase`:

```
task run_phase(uvm_phase phase);
  super.run_phase(phase);

  phase.raise_objection(this);
  `uvm_info("RUN_PHASE", "run_phase objection raised from monitor", UVM_LOW)

  #50;

  phase.drop_objection(this);
  `uvm_info("RUN_PHASE", "run_phase objection dropped from monitor", UVM_LOW)
endtask
```

Here:

- Monitor raises an objection.
- Monitor waits for 50 time units.
- Monitor drops the objection.

Since the monitor waits longer than the driver, the `run_phase` continues until the monitor also drops its objection.

26. Behaviour with Multiple Objections

If both driver and monitor raise objections:

- Driver drops objection after 10 time units.
- Monitor drops objection after 50 time units.
- The `run_phase` ends only after the monitor drops its objection.

So the phase continues until all objections are cleared.

27. What Happens If an Objection Is Not Dropped

If a component raises an objection but does not drop it:

- The `run_phase` does not end.
- Later phases do not start.
- The simulation may appear stuck.

Therefore, every raised objection must be dropped.

28. Purpose of UVM Objections

UVM objections are used to:

- Prevent premature phase completion
- Extend time-consuming phases
- Synchronize components
- Coordinate driver, monitor, sequencer, and scoreboard activity
- Ensure simulation ends only after all required work is complete

29. Key Points to Remember

- `run_phase` is a task-based phase.
- It consumes simulation time.
- Main simulation activity happens in `run_phase`.
- All components execute `run_phase` concurrently.
- Driver, monitor, sequencer, and scoreboard commonly work during `run_phase`.
- Runtime phases include reset, configure, main, and shutdown-related phases.
- `pre_reset_phase` is used for setup before reset.
- `reset_phase` applies reset.
- `post_reset_phase` handles activities after reset deassertion.
- Configuration phases prepare the DUT and environment before transactions.
- Cleanup phases collect coverage, check results, and report simulation status.

- Cleanup phases include `extract_phase`, `check_phase`, `report_phase`, and `final_phase`.
- UVM objections control when a phase can end.
- A phase ends only after all objections are dropped.
- `raise_objection()` keeps a phase active.
- `drop_objection()` allows the phase to complete.
- Every raised objection must be dropped.

30. Final Summary

The `run_phase` is the main time-consuming phase in UVM where stimulus generation, driving, monitoring, and checking occur.

UVM also provides reset, configuration, main, shutdown, and cleanup phases to organize simulation flow.

UVM objections are used to control phase completion. Components raise objections when they are still active and drop them after finishing their work. The phase ends only when all objections are dropped.

UVM Report Functions and Macros

UVM Report Functions and Macros

1. Topic Overview

This section explains UVM reporting methods and macros used to print messages during simulation.

In normal SystemVerilog, messages are printed using:

- `$display`
- `$write`
- `$monitor`

In UVM, reporting is usually done using UVM report functions and macros.

2. UVM Report Types

UVM provides report methods for different message severities:

- Info
- Warning
- Error
- Fatal

The general function format is:

uvm_report_*(id, message, verbosity);

Here, `*` can be replaced by:

- `info`
- `warning`
- `error`
- `fatal`

Examples:

uvm_report_info(...)
uvm_report_warning(...)
uvm_report_error(...)
uvm_report_fatal(...)

3. Report Function Arguments

A UVM report function usually contains:

- `id`

- `message`
- `verbosity level`
- optional file name
- optional line number

ID

The `id` is also called the tag or type ID.

It identifies the source or category of the message.

Usually, `get_type_name()` is used to get the class name as the report ID.

Message

The message is the text to be printed.

Verbosity Level

Verbosity controls how much detail should be printed in the simulation log.

4. UVM Verbosity Levels

UVM verbosity is used to control the amount of information printed during simulation.

The common verbosity levels are:

- `UVM_NONE`
- `UVM_LOW`
- `UVM_MEDIUM`
- `UVM_HIGH`
- `UVM_FULL`
- `UVM_DEBUG`

These are defined as an enum type in UVM.

5. Meaning of Verbosity

Verbosity works like a filter.

Only messages with verbosity level equal to or lower than the current verbosity setting are printed.

Messages with higher verbosity are suppressed.

Example

If the current verbosity is set to:

UVM_MEDIUM

Then messages with the following levels are printed:

- `UVM_NONE`
- `UVM_LOW`
- `UVM_MEDIUM`

Messages with the following levels are suppressed:

- `UVM_HIGH`
- `UVM_FULL`
- `UVM_DEBUG`

6. Runtime Verbosity Setting

Verbosity can be set at runtime using plus arguments.

General format:

+UVM_SET_VERBOSITY=*,_ALL_,UVM_HIGH,time

Using `*` applies the verbosity setting to all components.

Verbosity can be set to any UVM verbosity level, such as:

- `UVM_LOW`
- `UVM_MEDIUM`
- `UVM_HIGH`
- `UVM_FULL`

7. UVM Report Functions

UVM provides four main report functions:

uvm_report_info
uvm_report_warning
uvm_report_error
uvm_report_fatal

These are virtual functions available in the UVM library.

8. `uvm_report_info`

`uvm_report_info` is used to print informational messages.

Example:

```
uvm_report_info(
  get_type_name(),
  "Printing from build phase",
  UVM_MEDIUM,
  "test.sv",
```

```
  18
);
```

Arguments:

- `get_type_name()` gives the report ID.
- `"Printing from build phase"` is the message.
- `UVM_MEDIUM` is the verbosity level.
- `"test.sv"` is the file name.
- `18` is the line number.

File name and line number are optional.

9. Example UVM Test Class

A simple UVM test class can be created as:

```
`include "uvm_macros.svh"
import uvm_pkg::*;

class test extends uvm_test;

  `uvm_component_utils(test)

  function new(string name = "test", uvm_component parent = null);
    super.new(name, parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);

    uvm_report_info(
      get_type_name(),
      "Printing from build phase",
      UVM_MEDIUM,
      "test.sv",
      18
    );
  endfunction

endclass
```

The top module runs the test using:

```
module top;
  initial begin
    run_test("test");
  end
endmodule
```

10. Default Verbosity Behaviour

By default, UVM verbosity is usually set to:

UVM_MEDIUM

So, if a message has verbosity:

UVM_HIGH

it will not be printed unless the simulation verbosity is increased.

If the message verbosity is changed to:

UVM_MEDIUM

then it will be printed.

11. Setting Verbosity in Code

Verbosity can also be changed inside the testbench using:

uvm_top.set_report_verbosity_level(UVM_LOW);

If the verbosity is set to `UVM_LOW`, only messages with `UVM_LOW` or lower will be printed.

So, a message with `UVM_MEDIUM` will be suppressed.

12. UVM Report Macros

Instead of using report functions, UVM macros are commonly used.

The info macro is:

`uvm_info(id, message, verbosity)

Example:

```
`uvm_info(
  get_type_name(),
  "Printing from build phase",
  UVM_NONE
)
```

13. Difference Between Report Function and Macro

Report Function

Example:

uvm_report_info(id, message, verbosity, file, line);

It can take optional file name and line number arguments.

If file and line are not provided, they may not be printed.

Report Macro

Example:

`uvm_info(id, message, verbosity)

The macro automatically prints useful information such as:

- file name
- line number
- severity
- report ID
- message

This makes macros shorter and more convenient.

14. Printing Variables in UVM Messages

UVM report macros expect only three main arguments:

`uvm_info(id, message, verbosity)

So, a variable cannot be passed as an extra argument like `$display`.

Incorrect style:

`uvm_info(get_type_name(), "value of a = %0d", a, UVM_LOW)

This adds too many arguments.

To print variable values, use `$sformatf`.

Correct style:

int a = 50;

```
`uvm_info(
  get_type_name(),
  $sformatf("The value of a is %0d", a),
  UVM_LOW
```

)

15. Why `$sformatf` Is Used

`$sformatf` creates a formatted string.

It does not directly print to the console.

It is useful because UVM macros expect the message as a single string argument.

Example:

$sformatf("The value of a is %0d", a)

This returns a formatted string that can be passed to `uvm_info`.

16. UVM Warning Report

`uvm_report_warning` is used to print warning messages.

Example:

```
uvm_report_warning(
  get_type_name(),
  "This is a warning message"
);
```

The default verbosity level for warning reports is usually `UVM_MEDIUM`.

A warning indicates that something unusual happened, but simulation can continue.

17. UVM Error Report

`uvm_report_error` is used to print error messages.

Example:

```
uvm_report_error(
  get_type_name(),
  "This is an error message"
);
```

The default verbosity level for error reports is usually `UVM_LOW`.

An error indicates a problem in the test or design behaviour, but simulation may still continue depending on the configured action.

18. UVM Fatal Report

`uvm_report_fatal` is used to print fatal error messages.

Example:

```
uvm_report_fatal(
  get_type_name(),
  "This is a fatal error message"
);
```

A fatal report usually prints the message and exits the simulation.

It has the highest severity.

19. Severity Order

UVM report severities increase in this order:

UVM_INFO < UVM_WARNING < UVM_ERROR < UVM_FATAL

Meaning:

- `UVM_INFO` is the lowest severity.
- `UVM_WARNING` is more serious than info.
- `UVM_ERROR` is more serious than warning.
- `UVM_FATAL` is the most serious and usually stops simulation.

20. Report Summary Counts

At the end of simulation, UVM reports a summary count.

It shows how many messages occurred for each severity:

- Number of info messages
- Number of warnings
- Number of errors
- Number of fatal messages

This helps quickly understand the simulation result.

21. Severity Actions

UVM allows actions to be assigned to different severity levels.

For example, an action can define whether a message should be:

- displayed
- counted
- ignored

- used to exit simulation

22. Disabling an Action

Example:

set_report_severity_action(UVM_INFO, UVM_NO_ACTION);

This disables action for `UVM_INFO`.

As a result, info messages may not be displayed or counted, depending on the action setting.

23. Forcing Exit on a Severity

Example:

set_report_severity_action(UVM_INFO, UVM_EXIT);

This forces the simulation to exit when an info message is reported.

This is normally not recommended for info messages because info messages are usually meant only for display.

24. Default Severity Actions

Typical default actions are:

Severity	Default Behaviour
`UVM_INFO`	Display message
`UVM_WARNING`	Display message
`UVM_ERROR`	Display message and count error
`UVM_FATAL`	Display message and exit simulation

25. Key Points to Remember

- UVM uses report functions and macros instead of plain `$display`.
- Main report types are info, warning, error, and fatal.
- UVM report functions include `uvm_report_info`, `uvm_report_warning`, `uvm_report_error`, and `uvm_report_fatal`.
- UVM report macros include `uvm_info`, `uvm_warning`, `uvm_error`, and `uvm_fatal`.
- Verbosity controls how much information is printed.
- Only messages at or below the current verbosity level are printed.

- Higher verbosity messages are suppressed.
- `UVM_MEDIUM` is usually the default verbosity.
- `get_type_name()` is commonly used as the message ID.
- `$sformatf` is used to format variables into UVM messages.
- Report macros automatically include file and line information.
- Report functions can optionally include file and line information.
- Severity actions control what happens when a report message occurs.
- `UVM_FATAL` usually terminates the simulation.

26. Final Summary

UVM reporting provides a structured way to print simulation messages.

Instead of using `$display`, UVM uses report functions and macros such as:

`uvm_info
`uvm_warning
`uvm_error
`uvm_fatal

Verbosity levels control which messages are shown, while severity levels indicate how serious each message is.

Use `$sformatf` when variable values need to be included inside UVM report messages.

UVM report macros are commonly preferred because they are shorter and automatically provide useful debug information such as file name and line number.

UVM TLM Ports: put and put_imp

UVM TLM Ports: `put` and `put_imp`

1. Topic Overview

This section explains **UVM TLM ports**, especially:

- TLM communication
- Producer-consumer model
- Difference between mailbox and TLM
- Port, export, and implementation port
- Blocking `put` port
- Blocking `put_imp`
- Generator-to-driver transaction transfer

2. What Is TLM?

TLM stands for **Transaction-Level Modeling**.

In UVM, TLM ports are used to transfer transactions between verification components.

Examples of transactions:

- Sequence items
- Packets
- Data objects

TLM communication is commonly used between:

- Sequence and driver
- Generator and driver
- Monitor and scoreboard
- Monitor and coverage collector
- Scoreboard and other checking components

3. Why TLM Ports Are Used

TLM ports provide a standard communication mechanism between UVM components.

They help transfer transactions without tightly coupling the components.

This improves:

- Reusability
- Maintainability
- Modularity
- Separation of concerns

4. Producer-Consumer Model

UVM TLM follows a producer-consumer model.

One component produces or sends a transaction.

Another component consumes or receives that transaction.

Example:

- Generator produces a transaction.
- Driver consumes the transaction and drives it to the DUT.

5. Blocking and Non-Blocking Communication

UVM TLM supports two communication styles:

- Blocking communication
- Non-blocking communication

Blocking Communication

A blocking method waits until the transaction is successfully transferred.

Example:

- `put()`
- `get()`

Non-Blocking Communication

A non-blocking method does not wait. It returns immediately with success or failure status.

Non-blocking versions are discussed separately.

6. Common Types of TLM Ports

Common UVM TLM interfaces include:

- `put` port and `put` export
- `get` port and `get` export
- `peek` port and `peek` export
- `transport` port and `transport` export
- Analysis port
- TLM FIFO

7. TLM FIFO

A **TLM FIFO** is similar to a mailbox.

It is implemented using TLM ports and exports.

In SystemVerilog, mailboxes are often used directly.

In UVM, TLM FIFO provides a cleaner API over mailbox-like communication.

8. Mailbox Communication in SystemVerilog

In a SystemVerilog testbench, two components may communicate using a mailbox.

Example components:

- Component A
- Component B

To use a mailbox:

1. Mailbox handles must be declared in both components.
2. The mailbox object is usually created in a top-level component such as the environment.
3. The same mailbox instance is passed to both components through constructors.
4. One component uses `put()`.
5. The other component uses `get()`.

Example idea:

```
mbx.put(transaction);
mbx.get(transaction);
```

9. Limitation of Mailbox-Based Communication

Mailbox-based communication creates dependency between components.

Both components must know about the mailbox type.

If the communication mechanism changes later, the component code may also need changes.

This reduces reusability.

10. How TLM Solves This

TLM removes direct mailbox dependency.

A component only needs to know what method it can call.

For example:

- Component A has a `put` port.
- It only requires the connected component to implement a `put()` method.

The connected component can be:

- Driver
- FIFO
- Arbiter
- Any other compatible component

This makes the design more flexible.

11. TLM Terminologies

Important TLM terms:

- Port
- Export
- Implementation port

12. Port

A **port** is used by the initiator.

It is the side that calls a TLM method.

The port does not contain the actual method logic.

It only calls methods such as:

- `put()`
- `get()`
- `write()`

Port Symbol

A port is usually represented using a square box.

Port Declaration Syntax

Example for a blocking put port:

uvm_blocking_put_port #(transaction) put_port;

Here:

- `uvm_blocking_put_port` is the TLM port type.
- `transaction` is the transaction type.
- `put_port` is the port handle.

13. Export

An **export** is an intermediate connection point.

It passes the method call to another component or implementation.

Export is usually represented using a circle.

Exports are useful when a component forwards TLM calls to another lower-level component.

14. Implementation Port

An **implementation port**, or `imp`, is used by the target.

It contains the actual implementation of the TLM method.

For example, if the initiator calls `put()`, the target implements the body of the `put()` task.

Implementation Port Symbol

Implementation port is also represented using a circle.

Implementation Port Declaration Syntax

Example:

uvm_blocking_put_imp #(transaction, driver) put_imp;

Here:

- `transaction` is the transaction type.
- `driver` is the class where the `put()` method is implemented.
- `put_imp` is the implementation port handle.

15. Initiator and Target

In TLM communication:

Initiator

The initiator calls the method.

Example:

put_port.put(txn);

The initiator does not contain the actual method body.

Target

The target implements the method body.

Example:

```
task put(transaction txn);
  // method implementation
endtask
```

16. Generator and Driver Example

Consider two components:

- Generator
- Driver

The generator creates a transaction and sends it to the driver.

In this example:

- Generator is the initiator.
- Driver is the target.
- Generator has a `put_port`.
- Driver has a `put_imp`.
- Generator calls `put()`.
- Driver implements `put()`.

17. Transaction Class

A transaction class is created to pass data between generator and driver.

The class extends:

uvm_object

Example:

class transaction extends uvm_object;

The class is registered with the factory:

`uvm_object_utils(transaction)

It contains a randomized data field:

rand int data;

It also contains a display function to print the transaction data.

18. Required UVM Setup

The UVM macros file must be included:

`include "uvm_macros.svh"

The UVM package must be imported:

import uvm_pkg::*;

19. Transaction Class Structure

Basic transaction class:

```
class transaction extends uvm_object;

  `uvm_object_utils(transaction)

  rand int data;

  function new(string name = "transaction");
    super.new(name);
  endfunction

  function void display(string message);
    $display("%s: data = %0d", message, data);
  endfunction

endclass
```

20. Generator Class

The generator extends:

uvm_component

It is registered with:

`uvm_component_utils(uvm_generator)

The generator is the initiator, so it contains a blocking put port.

uvm_blocking_put_port #(transaction) put_port;

21. Creating the Put Port

The put port is created inside the generator constructor.

put_port = new("put_port", this);

This creates the TLM port object.

22. Generator `run_phase`

Inside the generator `run_phase`:

1. Create a transaction object.

2. Randomize the transaction.
3. Call `put()` using the port.
4. Pass the transaction to the connected target.

Example:

```
task run_phase(uvm_phase phase);
  transaction txn;

  txn = transaction::type_id::create("txn");
  txn.randomize();

  put_port.put(txn);
endtask
```

23. Meaning of `put_port.put(txn)`

The generator calls:

put_port.put(txn);

This does not execute logic inside the generator.

Instead:

- The call travels through the TLM connection.
- It reaches the driver's implementation port.
- The driver's `put()` task is executed.

So, the generator calls the method, but the driver implements it.

24. Driver Class

The driver extends:

uvm_driver

It is registered with:

`uvm_component_utils(driver)

The driver is the target, so it contains a blocking put implementation port.

uvm_blocking_put_imp #(transaction, driver) put_imp;

25. Creating the Implementation Port

The implementation port is created in the driver constructor.

put_imp = new("put_imp", this);

26. Driver `put()` Task

The driver must implement the `put()` task because it owns the `put_imp`.

Example:

```
task put(transaction txn);
  txn.display("Driver received transaction");
endtask
```

The driver can also call another task, such as `drive()`, to drive the transaction to the DUT.

Example idea:

```
task put(transaction txn);
  drive(txn);
endtask
```

27. Driver Logic

The driver receives the transaction through `put()`.

Then it can:

- Display the transaction
- Decode the transaction
- Convert it to pin-level signals
- Drive it to the DUT

The transcript mainly demonstrates displaying the received transaction.

28. Agent Class

The agent extends:

uvm_agent

It is registered with:

`uvm_component_utils(agent)

The agent contains:

- Generator handle

- Driver handle

Example:

uvm_generator gen_h;
driver drv_h;

29. Agent `build_phase`

Inside the agent `build_phase`, the generator and driver are created using the factory.

gen_h = uvm_generator::type_id::create("gen_h", this);
drv_h = driver::type_id::create("drv_h", this);

30. Agent `connect_phase`

The TLM connection is made in the agent `connect_phase`.

gen_h.put_port.connect(drv_h.put_imp);

This connects:

- Generator's `put_port`
- Driver's `put_imp`

After this connection, the generator can call `put()` and the driver will receive the transaction.

31. Test Class

The test class extends:

uvm_test

It creates the agent inside its `build_phase`.

The test is started from the top module using:

run_test("test");

32. Top Module

The top module calls the UVM test.

```
module top;
  initial begin
    run_test("test");
```

```
  end
endmodule
```

33. Output Behaviour

When the code runs:

1. Generator creates a transaction.
2. Generator randomizes the data.
3. Generator calls `put_port.put(txn)`.
4. Driver receives the transaction through `put_imp`.
5. Driver displays the transaction data.

The displayed output confirms that data has been transferred from generator to driver.

34. Put Port Data Flow

For `put` communication:

Generator → Driver

Meaning:

- Data is pushed from generator to driver.
- Generator initiates the transfer.
- Driver receives and implements the method.

35. Push-Based Communication

`put()` is a push-based method.

The producer pushes data to the consumer.

In the example:

- Generator has the data.
- Generator pushes the data to the driver using `put()`.

36. Pull-Based Communication

`get()` is a pull-based method.

The consumer pulls data from the producer.

Example scenario:

- Driver is the initiator.
- Generator is the target.
- Driver calls `get()`.

- Generator provides the transaction.

This uses:

- `get_port`
- `get_imp`

The `get` method is discussed separately.

37. Put vs Get Concept

Method	Communication Type	Initiator	Target	Data Direction
`put()`	Push-based	Generator	Driver	Generator → Driver
`get()`	Pull-based	Driver	Generator	Generator → Driver

In both cases, the transaction may move from generator to driver.

The difference is who initiates the method call.

38. Key Points to Remember

- TLM stands for Transaction-Level Modeling.
- TLM ports transfer transactions between UVM components.
- TLM improves reusability and reduces dependency compared to mailboxes.
- Mailbox handles must be shared manually in SystemVerilog.
- TLM ports are part of the UVM communication mechanism.
- A port calls a method.
- An implementation port provides the method body.
- The initiator owns the port.
- The target owns the implementation port.
- `put()` is push-based communication.
- `get()` is pull-based communication.
- `uvm_blocking_put_port` is used by the sender or initiator.
- `uvm_blocking_put_imp` is used by the receiver or target.
- In the example, generator calls `put()`.
- Driver implements `put()`.
- The generator's `put_port` is connected to the driver's `put_imp` in `connect_phase`.
- Transaction data is transferred from generator to driver.
- The driver can use the received transaction to drive DUT signals.

39. Final Summary

UVM TLM ports provide a clean and reusable way to communicate between verification components.

In the `put` example, the generator creates and randomizes a transaction, then sends it to the driver using a blocking put port.

The driver receives the transaction through a blocking put implementation port and executes the `put()` task.

This avoids direct mailbox dependency and makes the testbench more modular, reusable, and easier to maintain.

UVM TLM FIFO and Export

UVM TLM FIFO and Export

1. Topic Overview

This section explains:

- TLM export
- Push-based and pull-based communication
- `put` and `get` ports
- `uvm_tlm_fifo`
- How to connect generator and driver using TLM FIFO

This is a continuation of the previous TLM topic, where TLM ports and implementation ports were introduced.

2. Recap of TLM Ports

TLM stands for **Transaction-Level Modeling**.

UVM TLM ports are used to pass transactions between UVM components.

Examples of communicating components:

- Generator to driver
- Sequencer to driver
- Monitor to scoreboard
- Monitor to coverage collector

In the previous topic, two important TLM concepts were discussed:

- **Port**
- **Implementation port**

3. Port

A port is used by the component that initiates a TLM method call.

The component with the port calls methods such as:

- `put()`
- `get()`
- `write()`

Example:

uvm_blocking_put_port #(packet) put_port;

The port only calls the method. It does not contain the actual method implementation.

4. Implementation Port

An implementation port contains the actual implementation of the TLM method.

Example:

uvm_blocking_put_imp #(packet, driver) put_imp;

The target component implements the method body.

Example:

```
task put(packet pkt);
  // implementation logic
endtask
```

So, the port calls the method, and the implementation port executes the method.

5. Export

An export forwards method calls to another connected port or implementation port.

It acts like a relay or pass-through.

Exports are useful in hierarchical connections.

6. Need for Export

Consider this hierarchy:

```
Parent component
  └── Subcomponent with port
Another component
  └── Subcomponent with implementation port
```

If the port inside one hierarchy must connect to an implementation port inside another hierarchy, an export can be used as an intermediate connection.

The export forwards the call from the port to the actual implementation.

7. Export Use Case

Exports are used when:

- A component does not implement the TLM method directly.
- A lower-level subcomponent implements the method.
- The parent component only forwards the connection.
- Hierarchical TLM connections are required.

An export can be connected to:

- A port
- An implementation port
- Another export

8. Push-Based Communication

In push-based communication, the initiator pushes data to the target.

Example:

Initiator → Target

The initiator has a port.

The target has an implementation port.

For push-based transfer, `put()` is used.

Example

Generator → Driver

The generator pushes a transaction to the driver using a `put` port.

9. Pull-Based Communication

In pull-based communication, the initiator requests or pulls data from the target.

Example:

Target → Initiator

The initiator has a port.

The target has an implementation port.

For pull-based transfer, `get()` is used.

Example

Driver pulls transaction from Generator or FIFO

The driver calls `get()` to request a transaction.

10. Put Port vs Get Port

Feature	Put Port	Get Port
Communication type	Push-based	Pull-based

Data movement	Initiator to target	Target to initiator
Method used	`put()`	`get()`
Example initiator	Generator	Driver
Example target	Driver	Generator or FIFO

11. Get Port

A get port is used when a component actively requests data from another component.

It is commonly used in components such as:

- Drivers
- Monitors

Example:

A driver may request a transaction from a sequencer or FIFO.

Get Port Syntax

uvm_blocking_get_port #(packet) get_port;

Get Implementation Port Syntax

uvm_blocking_get_imp #(packet, component_name) get_imp;

12. Main Difference Between Put and Get

The main difference is the direction of method initiation and data movement.

Put

The producer pushes data.

Generator calls put()
Driver receives data

Get

The consumer requests data.

Driver calls get()
Generator or FIFO provides data

13. Need for Implementation Ports

When using only `put` and `get` ports directly, the target usually needs an implementation port.

For example:

- `put_port` requires a connected `put_imp`
- `get_port` requires a connected `get_imp`

The implementation method must be written in the target component.

14. What Is UVM TLM FIFO?

`uvm_tlm_fifo` is a built-in UVM component used for transaction-level FIFO communication.

It can be:

- Bounded
- Unbounded

It simplifies communication between components.

15. Why UVM TLM FIFO Is Useful

When using `uvm_tlm_fifo`, the user does not need to manually write `put_imp` or `get_imp`.

The FIFO already provides the required implementation.

The user only needs to:

- Declare ports in components
- Create the FIFO
- Connect ports to FIFO exports

16. Interfaces Provided by UVM TLM FIFO

A `uvm_tlm_fifo` provides interfaces for:

- `put`
- `get`
- `peek`

It supports both:

- Blocking communication
- Non-blocking communication

The transcript focuses on blocking `put` and `get`.

17. Blocking Communication

Blocking communication waits until the operation can complete.

Examples:

- `put()`
- `get()`

If the FIFO is full, `put()` may wait.

If the FIFO is empty, `get()` may wait.

Non-blocking communication is discussed separately.

18. TLM FIFO-Based Communication Setup

Consider an agent containing:

- Generator
- Driver
- TLM FIFO

Agent
├── Generator
├── TLM FIFO
└── Driver

The generator sends packets into the FIFO.

The driver receives packets from the FIFO.

19. Data Flow Using TLM FIFO

The data flow is:

Generator → TLM FIFO → Driver

The generator uses a `put_port`.

The driver uses a `get_port`.

The FIFO provides:

- `put_export`
- `get_export`

So, no manual implementation port is needed in the generator or driver.

20. Generator Role

The generator:

1. Creates a packet.

2. Randomizes the packet.
3. Sends the packet into the FIFO using `put_port.put()`.

The generator has:

uvm_blocking_put_port #(packet) put_port;

21. Driver Role

The driver:

1. Calls `get_port.get()`.
2. Receives a packet from the FIFO.
3. Displays or drives the packet.

The driver has:

uvm_blocking_get_port #(packet) get_port;

22. FIFO Role

The FIFO acts as the intermediate storage between generator and driver.

It receives data from the generator through `put_export`.

It provides data to the driver through `get_export`.

Example declaration:

uvm_tlm_fifo #(packet) fifo_h;

23. Required UVM Setup

The UVM macro file must be included:

`include "uvm_macros.svh"

The UVM package must be imported:

import uvm_pkg::*;

24. Packet Class

A packet class is created and passed from generator to driver.

The packet extends:

uvm_object

Example structure:

```
class packet extends uvm_object;

  `uvm_object_utils(packet)

  rand int data;

  function new(string name = "packet");
    super.new(name);
  endfunction

  function void display(string s);
    $display("%s: data = %0d", s, data);
  endfunction

endclass
```

The packet contains:

- Random data field
- Constructor
- Display function

25. Generator Class

The generator extends:

uvm_component

It is registered using:

```
`uvm_component_utils(generator)
```

The generator contains:

```
packet pkt;
uvm_blocking_put_port #(packet) put_port;
```

The `put_port` is created in the constructor:

```
put_port = new("put_port", this);
```

26. Generator `build_phase`

Inside the generator `build_phase`, the packet object is created:

pkt = packet::type_id::create("pkt");

27. Generator `run_phase`

Inside the generator `run_phase`:

pkt.randomize();
put_port.put(pkt);

This randomizes the packet and sends it into the FIFO.

28. Driver Class

The driver extends:

uvm_driver

It is registered using:

`uvm_component_utils(driver)

The driver contains:

packet pkt;
uvm_blocking_get_port #(packet) get_port;

The `get_port` is created in the constructor:

get_port = new("get_port", this);

29. Driver `run_phase`

Inside the driver `run_phase`:

get_port.get(pkt);
pkt.display("Driver");

This gets the packet from the FIFO and displays the received data.

30. Agent Class

The agent extends:

uvm_agent

It is registered using:

`uvm_component_utils(agent)

The agent contains:

generator gen_h;
driver drv_h;
uvm_tlm_fifo #(packet) fifo_h;

31. Agent Constructor

The FIFO can be created inside the agent constructor:

fifo_h = new("fifo_h", this);

32. Agent `build_phase`

Inside the agent `build_phase`, generator and driver objects are created:

gen_h = generator::type_id::create("gen_h", this);
drv_h = driver::type_id::create("drv_h", this);

33. Agent `connect_phase`

The generator and driver are connected to the FIFO.

Generator connection:

gen_h.put_port.connect(fifo_h.put_export);

Driver connection:

drv_h.get_port.connect(fifo_h.get_export);

This connects:

- Generator `put_port` to FIFO `put_export`
- Driver `get_port` to FIFO `get_export`

34. Test Class

The test class extends:

uvm_test

It is registered using:

`uvm_component_utils(test)

The test creates the agent in its `build_phase`:

agent_h = agent::type_id::create("agent_h", this);

35. Top Module

The top module starts the UVM test:

```
module top;
  initial begin
    run_test("test");
  end
endmodule
```

36. Output Behaviour

When the simulation runs:

1. Generator creates and randomizes a packet.
2. Generator puts the packet into the FIFO.
3. Driver gets the packet from the FIFO.
4. Driver displays the packet data.

Example output meaning:

Driver: data = <random_value>

This confirms that the packet was successfully transferred through the TLM FIFO.

37. TLM FIFO vs Direct Implementation Port

Method	Direct Port/Imp	TLM FIFO
Target implementation needed	Yes	No
Manual `put()`/`get()` body required	Yes	No
Intermediate storage	No	Yes

Built-in exports	No	Yes
Common use	Direct component-to-component method call	Buffered transaction transfer

38. Key Points to Remember

- Export forwards TLM method calls.
- Export acts as a relay in hierarchical connections.
- Push-based communication uses `put()`.
- Pull-based communication uses `get()`.
- `put()` sends data from initiator to target.
- `get()` allows initiator to pull data from target.
- Direct `put/get` communication needs implementation ports.
- `uvm_tlm_fifo` already provides the required implementation.
- With TLM FIFO, generator uses a `put_port`.
- With TLM FIFO, driver uses a `get_port`.
- FIFO provides `put_export` and `get_export`.
- Generator connects to `fifo_h.put_export`.
- Driver connects to `fifo_h.get_export`.
- TLM FIFO simplifies communication and avoids manually writing implementation ports.
- TLM FIFO can be bounded or unbounded.
- TLM FIFO supports put, get, and peek interfaces.
- TLM FIFO supports blocking and non-blocking methods.

39. Final Summary

`uvm_tlm_fifo` provides a simple way to transfer transactions between UVM components.

Instead of directly connecting a port to an implementation port, the generator puts transactions into the FIFO, and the driver gets transactions from the FIFO.

This removes the need to manually implement `put_imp` or `get_imp`, provides buffering, and makes the testbench cleaner, more reusable, and easier to maintain.

UVM TLM Blocking and Non-Blocking Ports

UVM TLM Blocking and Non-Blocking Ports

1. Topic Overview

This section explains UVM TLM ports in more detail, especially:

- Blocking methods
- Non-blocking methods
- Put, get, and peek methods
- Analysis write method
- Port, export, and implementation port relationship
- Which TLM port can call which method

This continues from the earlier TLM topics where ports, exports, implementation ports, `put`, `get`, and TLM FIFO were introduced.

2. TLM Interface Methods

UVM TLM interface methods are mainly grouped into:

- Put methods
- Get methods
- Peek methods
- Analysis write method

These methods may be blocking or non-blocking depending on the port type used.

3. Blocking Method

A blocking method waits until the operation is completed.

If a component calls a blocking method through a TLM port, the process does not move forward until the method responds.

Example blocking methods:

- `put()`
- `get()`
- `peek()`

Blocking Example

If `put()` is called, the process waits until the transaction is successfully sent.

If `get()` is called, the process waits until the transaction is successfully received.

4. Non-Blocking Method

A non-blocking method does not wait for the operation to complete.

It returns immediately with a status.

Return value:

- `1` → operation successful
- `0` → operation not completed

Example non-blocking methods:

- `try_put()`
- `can_put()`
- `try_get()`
- `can_get()`
- `try_peek()`
- `can_peek()`
- `write()`

5. Put Port Types

UVM provides several put-related ports and implementation ports.

Put Ports

- `uvm_put_port`
- `uvm_blocking_put_port`
- `uvm_nonblocking_put_port`

Put Implementation Ports

- `uvm_put_imp`
- `uvm_blocking_put_imp`
- `uvm_nonblocking_put_imp`

Put Exports

- `uvm_put_export`
- `uvm_blocking_put_export`
- `uvm_nonblocking_put_export`

6. Get Port Types

UVM provides several get-related ports and implementation ports.

Get Ports

- `uvm_get_port`
- `uvm_blocking_get_port`

- `uvm_nonblocking_get_port`

Get Implementation Ports

- `uvm_get_imp`
- `uvm_blocking_get_imp`
- `uvm_nonblocking_get_imp`

Get Exports

- `uvm_get_export`
- `uvm_blocking_get_export`
- `uvm_nonblocking_get_export`

7. Peek Port Types

UVM also provides peek-related ports and implementation ports.

Peek Ports

- `uvm_peek_port`
- `uvm_blocking_peek_port`
- `uvm_nonblocking_peek_port`

Peek Implementation Ports

- `uvm_peek_imp`
- `uvm_blocking_peek_imp`
- `uvm_nonblocking_peek_imp`

Peek Exports

- `uvm_peek_export`
- `uvm_blocking_peek_export`
- `uvm_nonblocking_peek_export`

8. Port, Target, and Method Relationship

In UVM TLM:

- The initiator owns the port.
- The target owns the implementation port.
- The initiator calls the method through the port.
- The target implements the method body.

For example:

Initiator port calls method → Target implementation port executes method

The method is declared and implemented in the target component.

9. Put Method

`put()` is a blocking method.

It is used to send a transaction from one component to another.

Purpose

The `put()` method sends a transaction to a consumer component.

Behaviour

Since it is blocking, the caller waits until the transaction is accepted successfully.

Example

port.put(trans_item);

Here, `trans_item` is passed from the initiator to the target.

10. Try Put Method

`try_put()` is a non-blocking method.

It is also used to send a transaction to another component.

Behaviour

- Returns `1` if the consumer is ready and accepts the transaction.
- Returns `0` if the consumer is not ready.

Example

port.try_put(trans_item);

Unlike `put()`, this method does not wait until the transaction is accepted.

11. Can Put Method

`can_put()` is a non-blocking method.

It does not send any transaction.

It only checks whether the target is ready to accept a transaction.

Behaviour

- Returns `1` if the target can accept a transaction.

- Returns `0` if the target cannot accept a transaction.

Important Point

No argument is passed to `can_put()`.

Example:

port.can_put();

12. Put-Based Communication Example

Consider:

Generator → Driver

Here:

- Generator is the initiator.
- Driver is the target.
- Generator owns the put port.
- Driver owns the implementation port.
- Data flows from generator to driver.

If blocking behaviour is needed:

put()

If non-blocking behaviour is needed:

try_put()

If only readiness must be checked:

can_put()

13. Push-Based Transactions

Put methods are used for push-based transactions.

In push-based communication:

Initiator → Target

The initiator pushes the transaction to the target.

Example:

Generator pushes transaction to driver

14. Get Method

`get()` is a blocking method.

It is used to receive or pull a transaction from another component.

Purpose

The initiator requests a transaction from the target.

Behaviour

Since it is blocking, the caller waits until the transaction is successfully retrieved.

Example

port.get(trans_item);

15. Can Get Method

`can_get()` is a non-blocking method.

It checks whether data is available to be retrieved.

Behaviour

- Returns `1` if data is available.
- Returns `0` if data is not available.

No transaction is consumed by `can_get()`.

16. Try Get Method

`try_get()` is a non-blocking method.

It tries to retrieve a transaction.

Behaviour

- Returns `1` if the transaction is successfully retrieved.
- Returns `0` if no transaction is available.

Unlike `get()`, it does not wait.

17. Pull-Based Transactions

Get methods are used for pull-based transactions.

In pull-based communication:

Target → Initiator

The initiator pulls the transaction from the target.

Example:

Driver pulls transaction from generator or FIFO

18. Peek Method

`peek()` is a blocking method.

It returns a transaction without consuming it.

Meaning

When `peek()` is called, the transaction is copied or observed, but it remains available.

A later `peek()` or `get()` can return the same transaction.

Difference from Get

- `get()` retrieves and consumes the transaction.
- `peek()` retrieves a copy or view without consuming the transaction.

19. Try Peek Method

`try_peek()` is a non-blocking method.

It tries to peek at a transaction without consuming it.

Behaviour

- Returns `1` if the transaction is available and copied.
- Returns `0` if no transaction is available.

20. Can Peek Method

`can_peek()` is a non-blocking method.

It checks whether a transaction is available for peeking.

Behaviour

- Returns `1` if a transaction can be peeked.

- Returns `0` if no transaction is available.

It does not consume the transaction.

21. Analysis Write Method

`write()` is used with the analysis interface.

It is a non-blocking method.

Purpose

The analysis write method broadcasts transactions to one or more listeners.

This is useful for one-to-many communication.

Example:

Monitor → Scoreboard
→ Coverage Collector
→ Subscriber

The monitor can broadcast the same transaction to multiple components.

22. Analysis Communication

Unlike put/get communication, analysis communication can have:

- One initiator
- Multiple targets/listeners

This is useful when the same transaction must be sent to several components.

23. Port as a Contract

A TLM port defines which methods can be called.

The connected component must implement the required method.

Example:

If a component has a blocking put port, it expects the connected target to implement:

put()

If the connected component does not implement the required method, a runtime error can occur.

24. Methods Supported by Put Ports

`uvm_blocking_put_port`

Can call only:

put()

It cannot call:

try_put()
can_put()

uvm_nonblocking_put_port

Can call only:

try_put()
can_put()

It cannot call:

put()

uvm_put_port

Can call all put-related methods:

put()
try_put()
can_put()

25. Methods Supported by Get Ports

uvm_blocking_get_port

Can call only:

get()

uvm_nonblocking_get_port

Can call:

try_get()
can_get()

uvm_get_port

Can call all get-related methods:

get()
try_get()
can_get()

26. Methods Supported by Peek Ports

`uvm_blocking_peek_port`

Can call only:

peek()

`uvm_nonblocking_peek_port`

Can call:

try_peek()
can_peek()

`uvm_peek_port`

Can call all peek-related methods:

peek()
try_peek()
can_peek()

27. Port and Method Summary

Port Type	Supported Methods
`uvm_blocking_put_port`	`put()`
`uvm_nonblocking_put_port`	`try_put()`, `can_put()`
`uvm_put_port`	`put()`, `try_put()`, `can_put()`
`uvm_blocking_get_port`	`get()`
`uvm_nonblocking_get_port`	`try_get()`, `can_get()`
`uvm_get_port`	`get()`, `try_get()`, `can_get()`
`uvm_blocking_peek_port`	`peek()`

`uvm_nonblocking_peek_port`	`try_peek()`, `can_peek()`
`uvm_peek_port`	`peek()`, `try_peek()`, `can_peek()`

28. Producer and Consumer Example

Consider an environment with:

- Producer
- Consumer

The producer is the initiator because it owns a TLM port.

The consumer is the target because it owns an implementation port.

Example:

uvm_put_port #(int) put_port;

The consumer has:

uvm_put_imp #(int, consumer) put_imp;

Here:

- `int` is the type of data being transferred.
- `consumer` is the class where the TLM method is implemented.
- The producer calls the method.
- The consumer implements the method.

29. Implementation Port Parameters

An implementation port usually needs:

1. The data type being transferred
2. The class where the method is implemented

Example:

uvm_put_imp #(int, consumer) put_imp;

Meaning:

- The transferred data type is `int`.
- The `put()` method is implemented inside the `consumer` class.

30. Key Points to Remember

- Blocking methods wait until completion.
- Non-blocking methods return immediately.
- `put()`, `get()`, and `peek()` are blocking methods.
- `try_put()`, `try_get()`, and `try_peek()` are non-blocking methods.
- `can_put()`, `can_get()`, and `can_peek()` only check readiness or availability.
- Put methods are used for push-based communication.
- Get methods are used for pull-based communication.
- Peek methods observe transactions without consuming them.
- Analysis `write()` broadcasts transactions to multiple listeners.
- Ports are owned by initiators.
- Implementation ports are owned by targets.
- A port can call only the methods supported by its type.
- The target must implement the methods required by the connected port.
- If the required method is not implemented, a runtime error can occur.

31. Final Summary

UVM TLM provides blocking and non-blocking communication methods for transaction transfer.

Blocking methods such as `put()`, `get()`, and `peek()` wait until the operation completes.

Non-blocking methods such as `try_put()`, `try_get()`, and `try_peek()` return immediately with success or failure.

The `can_*` methods only check readiness or availability.

Choosing the correct port type is important because each port supports only specific methods.

UVM TLM Blocking & Non-Blocking Ports

UVM TLM Blocking and Non-Blocking Ports with Coding Example

1. Topic Overview

This section explains a coding example for UVM TLM blocking and non-blocking ports.

The example uses:

- Producer
- Consumer
- `uvm_put_port`
- `uvm_put_imp`
- `put()`
- `try_put()`
- `can_put()`

The communication is push-based, where data is sent from the producer to the consumer.

2. Example Architecture

The design contains two main components:

Producer → Consumer

Producer

The producer is the initiator.

It contains:

uvm_put_port #(int) tlm_put;

The producer calls TLM methods such as:

- `put()`
- `try_put()`
- `can_put()`

Consumer

The consumer is the target.

It contains:

uvm_put_imp #(int, consumer) tlm_imp;

The consumer implements the methods called by the producer.

3. Communication Type

This example uses push-based communication.

Data direction:

Producer → Consumer

The producer sends an integer value to the consumer.

The consumer receives and processes the value.

4. Required UVM Setup

The UVM macro file must be included:

`include "uvm_macros.svh"

The UVM package must be imported:

import uvm_pkg::*;

5. Producer Class

The producer class extends `uvm_component`.

class producer extends uvm_component;

It is registered with the UVM factory:

`uvm_component_utils(producer)

The constructor follows the standard UVM component format:

```
function new(string name = "producer", uvm_component parent = null);
  super.new(name, parent);
endfunction
```

6. Producer TLM Port

The producer declares a put port:

uvm_put_port #(int) tlm_put;

Since `uvm_put_port` is used, the producer can call all three put-related methods:

- `put()`
- `try_put()`
- `can_put()`

The port is created inside the constructor:

tlm_put = new("tlm_put", this);

7. Producer `run_phase`

Inside the producer `run_phase`, an integer variable is declared and assigned.

Example:

int data;
data = 20;

The value is printed using `uvm_info`.

`uvm_info(
  get_type_name(),
  $sformatf("The value of data is %0d", data),
  UVM_LOW
)

8. Calling `put()`

The producer sends the data using:

tlm_put.put(data);

Behaviour of `put()`

- `put()` is a blocking method.
- It sends data to the consumer.
- It waits until the transaction is accepted.
- The corresponding `put()` task must be implemented in the consumer.

9. Calling `try_put()`

The producer also calls:

tlm_put.try_put(data);

Behaviour of `try_put()`

- `try_put()` is a non-blocking method.
- It tries to send data to the consumer.
- It returns immediately.
- It returns `1` if the data is accepted.
- It returns `0` if the data is not accepted.

10. Calling `can_put()`

The producer also calls:

tlm_put.can_put();

Behaviour of `can_put()`

- `can_put()` is a non-blocking method.
- It does not send data.
- It only checks whether the consumer is ready to accept data.
- It returns `1` if the consumer can accept data.
- It returns `0` if the consumer cannot accept data.

No argument is passed to `can_put()`.

11. Consumer Class

The consumer class extends `uvm_component`.

class consumer extends uvm_component;

It is registered with the UVM factory:

`uvm_component_utils(consumer)

The constructor follows the standard UVM component format:

```
function new(string name = "consumer", uvm_component parent = null);
  super.new(name, parent);
endfunction
```

12. Consumer Implementation Port

The consumer declares an implementation port:

uvm_put_imp #(int, consumer) tlm_imp;

The two parameters are:

- `int` – the data type being transferred
- `consumer` – the class where the implementation methods are written

The implementation port is created inside the constructor:

tlm_imp = new("tlm_imp", this);

13. Implementing `put()` in Consumer

Since the producer calls `put()`, the consumer must implement the `put()` method.

```
task put(int value);
  #10;
  `uvm_info(
    get_type_name(),
    $sformatf("Received the data %0d", value),
    UVM_LOW
  )
endtask
```

Why `put()` Is a Task

`put()` is a blocking method.

It may consume simulation time.

Since functions cannot consume simulation time, `put()` is implemented as a task.

In the example, a delay is added:

#10;

14. Implementing `try_put()` in Consumer

Since the producer calls `try_put()`, the consumer must implement `try_put()`.

```
function bit try_put(int value);
  `uvm_info(
    get_type_name(),
    $sformatf("Received try_put value %0d", value),
    UVM_LOW
  )
  return 1;
endfunction
```

Why `try_put()` Is a Function

`try_put()` is non-blocking.

It does not consume simulation time.

Therefore, it is implemented as a function.

It returns a bit value:

- `1` means success
- `0` means failure

15. Implementing `can_put()` in Consumer

Since the producer calls `can_put()`, the consumer must implement `can_put()`.

```
function bit can_put();
  `uvm_info(
    get_type_name(),
    "Inside can_put",
    UVM_LOW
  )
  return 1;
endfunction
```

Important Point

`can_put()` does not take any argument.

It only checks whether the consumer can accept data.

16. Environment Class

The environment contains both producer and consumer.

class environment extends uvm_env;

It is registered with the factory:

`uvm_component_utils(environment)

The environment declares handles:

```
producer ph;
consumer cr;
```

17. Environment Constructor

The constructor follows the standard UVM component format:

```
function new(string name = "environment", uvm_component parent = null);
  super.new(name, parent);
endfunction
```

18. Environment `build_phase`

Inside `build_phase`, producer and consumer objects are created using the UVM factory.

```
function void build_phase(uvm_phase phase);
  super.build_phase(phase);

  ph = producer::type_id::create("ph", this);
  cr = consumer::type_id::create("cr", this);
endfunction
```

19. Environment `connect_phase`

Inside `connect_phase`, the producer port is connected to the consumer implementation port.

```
function void connect_phase(uvm_phase phase);
  super.connect_phase(phase);

  ph.tlm_put.connect(cr.tlm_imp);
endfunction
```

This connects:

producer.tlm_put → consumer.tlm_imp

After this connection, calls from the producer reach the consumer.

20. Test Class

The test class extends `uvm_test`.

```
class test extends uvm_test;
```

It is registered with the factory:

```
`uvm_component_utils(test)
```

The test declares an environment handle:

environment env;

21. Test `build_phase`

The environment is created inside the test `build_phase`.

```
function void build_phase(uvm_phase phase);
  super.build_phase(phase);

  env = environment::type_id::create("env", this);
endfunction
```

22. Test `run_phase` and Objection

An objection is used to keep the simulation alive long enough for the TLM methods to execute.

```
task run_phase(uvm_phase phase);
  super.run_phase(phase);

  phase.raise_objection(this);

  #50;

  phase.drop_objection(this);
endtask
```

Purpose

- `raise_objection()` prevents the run phase from ending early.
- `#50` allows simulation activity to complete.
- `drop_objection()` allows the run phase to finish.

23. Top Module

The top module starts the UVM test.

```
module top;
  initial begin
    run_test("test");
  end
endmodule
```

24. Expected Output Behaviour

When the simulation runs:

1. Producer creates an integer value.
2. Producer prints the data value.
3. Producer calls `put(data)`.
4. Consumer receives the data through its `put()` task.
5. Producer calls `try_put(data)`.
6. Consumer receives the data through its `try_put()` function.
7. Producer calls `can_put()`.
8. Consumer executes `can_put()` and returns readiness status.

Example behaviour:

Producer: value of data is 20
Consumer: received data 20
Consumer: received try_put value 20
Consumer: inside can_put

If the output is displayed in hexadecimal format, the decimal value `20` may appear as `14`.

25. Why `uvm_put_port` Is Used

In this example, `uvm_put_port` is used instead of only `uvm_blocking_put_port`.

Reason:

uvm_put_port

supports all put-related methods:

- `put()`
- `try_put()`
- `can_put()`

If `uvm_blocking_put_port` were used, only `put()` could be called.

If `uvm_nonblocking_put_port` were used, only `try_put()` and `can_put()` could be called.

26. Method Support Summary

Port Type	Supported Methods
`uvm_blocking_put_port`	`put()` only
`uvm_nonblocking_put_port`	`try_put()`, `can_put()`
`uvm_put_port`	`put()`, `try_put()`, `can_put()`

27. Blocking vs Non-Blocking in This Example

Blocking Method

put()

- Implemented as a task
- Can consume simulation time
- Waits until the transfer completes

Non-Blocking Methods

try_put()
can_put()

- Implemented as functions
- Cannot consume simulation time
- Return immediately with status

28. Key Points to Remember

- The producer is the initiator.
- The consumer is the target.
- The producer has `uvm_put_port #(int)`.
- The consumer has `uvm_put_imp #(int, consumer)`.
- The data type transferred is `int`.
- The communication is push-based.
- `put()` sends data and blocks until completion.
- `try_put()` tries to send data and returns immediately.
- `can_put()` checks readiness and does not send data.
- `put()` is implemented as a task.
- `try_put()` and `can_put()` are implemented as functions.
- The producer port is connected to the consumer implementation port in `connect_phase`.
- `raise_objection()` and `drop_objection()` are used to control simulation duration.
- `uvm_put_port` can call all three put-related methods.

29. Final Summary

This example demonstrates how UVM TLM put communication works using blocking and non-blocking methods.

The producer sends integer data to the consumer using a `uvm_put_port`.

The consumer receives the calls through a `uvm_put_imp` and implements:

- `put()`
- `try_put()`
- `can_put()`

This shows how a single `uvm_put_port` can support blocking transfer, non-blocking transfer, and readiness checking in a push-based TLM communication model.

UVM Analysis Port

UVM Analysis Port

1. Topic Overview

This section explains the **UVM analysis port** and how it is used to broadcast transactions from one component to multiple components.

The example demonstrates:

- Analysis port
- Analysis implementation port
- `write()` method
- One-to-many transaction broadcasting
- Producer-to-multiple-consumer communication

2. What Is a UVM Analysis Port?

A **UVM analysis port** is a TLM-based communication mechanism.

It provides a `write()` method for sending transactions.

Unlike normal `put` or `get` communication, an analysis port can broadcast the same transaction to multiple components.

3. Normal TLM Communication vs Analysis Communication

Normal TLM Communication

Previously discussed TLM communication uses ports such as:

- `put_port`
- `get_port`

These are mainly used for communication between two components.

Example:

Component A → Component B

Analysis Port Communication

Using an analysis port, one component can broadcast a transaction to multiple components.

Example:

Producer → Consumer A
→ Consumer B
→ Consumer C

This is useful when the same transaction must be sent to multiple subscribers, such as:

- Scoreboard
- Coverage collector
- Reference model
- Logger

4. `write()` Method

The analysis interface uses the `write()` method.

The producer calls:

analysis_port.write(transaction);

Each connected consumer implements its own `write()` method.

So, one transaction can be received by multiple components.

5. Important Properties of Analysis Port

A UVM analysis port can be left open without connecting to an implementation port or export.

This means:

- No error is reported if an analysis port has no connected receiver.
- A single analysis port can connect to one or more analysis implementation ports.
- A single analysis port can also connect to an analysis export.
- Multiple analysis ports can connect to a single analysis implementation port or analysis export.

This makes analysis ports flexible for broadcast-style communication.

6. Valid Analysis Connections

Valid connections include:

Connection	Valid?
Port to port	Yes
Port to export	Yes
Port to implementation port	Yes
Export to export	Yes
Export to implementation port	Yes
Implementation port to export	No

7. Analysis Port Syntax

Analysis Port Declaration

uvm_analysis_port #(transaction) producer_put;

Here:

- `transaction` is the type of object being sent.
- `producer_put` is the analysis port handle.

8. Analysis Implementation Port Syntax

uvm_analysis_imp #(transaction, consumer_a) consumer_imp;

Here:

- `transaction` is the transaction type.
- `consumer_a` is the component where the `write()` method is implemented.
- `consumer_imp` is the implementation port handle.

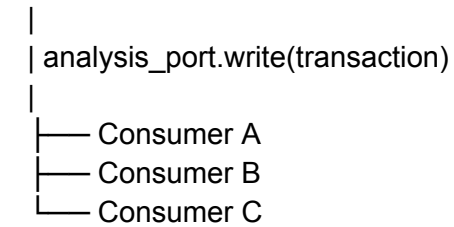
9. Analysis Export Syntax

uvm_analysis_export #(transaction) analysis_export;

An analysis export is used to forward analysis transactions hierarchically.

10. Example Architecture

The example contains:



The producer contains:

- One analysis port

Each consumer contains:

- One analysis implementation port
- One `write()` function

The producer broadcasts the same transaction to all three consumers.

11. Transaction Class

A transaction class is created first.

It extends:

uvm_object

Example:

```
class transaction extends uvm_object;

  `uvm_object_utils(transaction)

  int data = 20;

  function new(string name = "transaction");
    super.new(name);
  endfunction

endclass
```

The transaction contains:

- `data` field initialized to `20`
- Standard UVM object constructor
- Factory registration using `uvm_object_utils`

12. Producer Class

The producer extends:

uvm_component

It contains an analysis port:

uvm_analysis_port #(transaction) producer_put;

The analysis port is created inside the constructor:

producer_put = new("producer_put", this);

13. Producer `run_phase`

Inside the producer `run_phase`:

1. A transaction handle is created.

2. The transaction object is created using the factory.
3. The transaction is randomized.
4. The data value is printed.
5. The transaction is broadcast using `write()`.

Example:

```
task run_phase(uvm_phase phase);
  transaction t_h;

  super.run_phase(phase);

  t_h = transaction::type_id::create("t_h", this);
  assert(t_h.randomize());

  `uvm_info(
    get_type_name(),
    $sformatf("The value of data is %0d", t_h.data),
    UVM_NONE
  )

  producer_put.write(t_h);
endtask
```

14. Meaning of `producer_put.write(t_h)`

This statement broadcasts the transaction object `t_h` to all connected analysis implementation ports.

If three consumers are connected, all three receive the same transaction.

15. Consumer A Class

Consumer A extends:

uvm_component

It contains an analysis implementation port:

uvm_analysis_imp #(transaction, consumer_a) consumer_imp;

The implementation port is created in the constructor:

consumer_imp = new("consumer_imp", this);

Consumer A implements the `write()` function:

```
virtual function void write(transaction t_h);
  `uvm_info(
```

```
    get_type_name(),
    $sformatf("Received value is %0d", t_h.data),
    UVM_LOW
  )
endfunction
```

16. Consumer B Class

Consumer B is similar to Consumer A.

It contains its own analysis implementation port:

```
uvm_analysis_imp #(transaction, consumer_b) b_imp;
```

It also implements the `write()` function:

```
virtual function void write(transaction t_h);
  `uvm_info(
    get_type_name(),
    $sformatf("Received value is %0d", t_h.data),
    UVM_LOW
  )
endfunction
```

17. Consumer C Class

Consumer C also follows the same structure.

It contains:

```
uvm_analysis_imp #(transaction, consumer_c) c_imp;
```

It implements:

```
virtual function void write(transaction t_h);
  `uvm_info(
    get_type_name(),
    $sformatf("Received value is %0d", t_h.data),
    UVM_LOW
  )
endfunction
```

18. Environment Class

The environment extends:

```
uvm_env
```

It contains handles for:

- Producer
- Consumer A
- Consumer B
- Consumer C

Example:

```
producer   p_h;
consumer_a ca_h;
consumer_b cb_h;
consumer_c cc_h;
```

19. Environment `build_phase`

Inside the environment `build_phase`, all components are created using the factory:

```
p_h  = producer::type_id::create("p_h", this);
ca_h = consumer_a::type_id::create("ca_h", this);
cb_h = consumer_b::type_id::create("cb_h", this);
cc_h = consumer_c::type_id::create("cc_h", this);
```

20. Environment `connect_phase`

Inside the environment `connect_phase`, the producer analysis port is connected to each consumer implementation port.

```
p_h.producer_put.connect(ca_h.consumer_imp);
p_h.producer_put.connect(cb_h.b_imp);
p_h.producer_put.connect(cc_h.c_imp);
```

This creates one-to-many communication.

The same producer analysis port is connected to three analysis implementation ports.

21. Test Class

The test class extends:

uvm_test

It contains an environment handle:

environment env;

Inside the test `build_phase`, the environment is created:

env = environment::type_id::create("env", this);

22. Top Module

The top module starts the UVM test.

```
module top;
  initial begin
    run_test("test");
  end
endmodule
```

23. Expected Output

When the simulation runs:

1. Producer creates a transaction.
2. Producer prints the transaction data.
3. Producer broadcasts the transaction using `write()`.
4. Consumer A receives the transaction.
5. Consumer B receives the transaction.
6. Consumer C receives the transaction.

Expected output meaning:

```
Producer: The value of data is 20
Consumer A: Received value is 20
Consumer B: Received value is 20
Consumer C: Received value is 20
```

This confirms that one producer can broadcast the same transaction to multiple consumers.

24. Key Points to Remember

- UVM analysis port is used for broadcast communication.
- It uses the `write()` method.
- One analysis port can connect to multiple analysis implementation ports.
- Analysis port communication is one-to-many.
- Analysis ports can be left unconnected without causing an error.
- Analysis implementation ports must implement the `write()` function.
- The producer owns the analysis port.
- Consumers own analysis implementation ports.
- The producer calls `write()`.
- Each consumer receives the same transaction through its own `write()` function.
- Analysis ports are useful for sending monitor transactions to multiple components.
- Common receivers include scoreboards, coverage collectors, subscribers, and reference models.

25. Final Summary

The UVM analysis port is used when one component needs to broadcast transactions to multiple components.

In the example, the producer sends one transaction through its analysis port. The same transaction is received by Consumer A, Consumer B, and Consumer C through their analysis implementation ports.

This makes analysis ports ideal for monitor-to-scoreboard, monitor-to-coverage, and monitor-to-subscriber communication in UVM testbenches.

UVM Config DB

Introduction to UVM Config DB

1. Topic Overview

This section introduces **UVM Config DB**, which is used to pass configuration information between UVM components.

Configuration data may include:

- Addresses
- Timeout values
- Control signals
- Virtual interface handles
- Setup values
- Object or transaction handles

UVM Config DB allows higher-level components, such as the test or environment, to pass data to lower-level components, such as agents, drivers, and monitors.

2. Why UVM Config DB Is Needed

In a UVM testbench, configuration data often needs to move from top-level components to lower-level components.

Example hierarchy:

```
top
 └── test
     └── environment
         └── driver
```

A common requirement is to pass a **virtual interface handle** from the top module to the driver.

In a normal SystemVerilog testbench, this may be passed using the constructor `new`.

In UVM, this is usually done using:

uvm_config_db

This avoids manual constructor-based passing and makes the testbench more flexible and scalable.

3. What Is UVM Config DB?

`uvm_config_db` is an internal UVM configuration database.

It works like a storage system where:

- A component stores a value using `set()`.

- Another component retrieves the value using `get()`.

Each stored value is associated with a **key**.

The sender and receiver must use the same key to access the same value.

4. Key Concept

Every value stored in the config DB has a corresponding key.

Example:

Value: 50
Key: "control"

Example:

Value: virtual interface handle
Key: "vif"

The component that retrieves the value must use the same key.

If the key does not match, the value cannot be retrieved.

5. Common UVM Config DB Methods

The main methods in `uvm_config_db` are:

- `set()`
- `get()`
- `exists()`
- `unset()`

All these methods are static.

So they are accessed using the scope resolution operator:

uvm_config_db#(type)::method_name(...)

6. `set()` Method

The `set()` method stores a value into the UVM config database.

It is used by the component that sends or provides configuration data.

Example use:

uvm_config_db#(int)::set(...);

If the value is an integer, the type parameter is `int`.

If the value is a virtual interface, the type parameter should be the virtual interface type.

7. `get()` Method

The `get()` method retrieves a value from the UVM config database.

It is used by the component that needs to receive configuration data.

Example use:

uvm_config_db#(int)::get(...);

The type used in `get()` must match the type used in `set()`.

8. `exists()` Method

The `exists()` method checks whether a value exists in the config DB for a given key.

It is useful when a component wants to check if a configuration entry is available before trying to use it.

9. `unset()` Method

The `unset()` method removes a value from the config DB.

It is used when a configuration entry should no longer be available.

10. Syntax of `set()`

General syntax:

```
uvm_config_db#(type)::set(
  context,
  instance_path,
  field_name,
  value
);
```

Arguments

Argument	Meaning
`context`	Starting point of the hierarchy
`instance_path`	Path that controls which components can access the value

`field_name`	Key name
`value`	Actual value stored in the database

11. `set()` Argument: Context

The `context` argument defines the hierarchy starting point.

It tells UVM from where the database entry should be considered.

If `set()` is called from a UVM component, use:

this

If `set()` is called from a SystemVerilog top module, use:

null

Reason:

- UVM components have hierarchy.
- The top module is not a UVM component.

12. `set()` Argument: Instance Path

The `instance_path` limits which components can access the stored value.

Example:

"env.driver"

Only the driver at that path can access the value.

Example:

"env.*"

All components below `env` can access the value.

The instance path is useful for controlling visibility.

13. `set()` Argument: Field Name

The `field_name` is the key.

Example:

"control"

The same key must be used in the `get()` method.

14. `set()` Argument: Value

The `value` is the actual data stored in the config DB.

Example values:

- Integer control value
- Virtual interface handle
- Timeout value
- Configuration object handle

15. Syntax of `get()`

General syntax:

```
uvm_config_db#(type)::get(
  context,
  instance_path,
  field_name,
  variable
);
```

Arguments

Argument	Meaning
`context`	Component that is accessing the database
`instance_path`	Usually left empty on receiver side
`field_name`	Key name
`variable`	Variable where retrieved value is stored

16. `get()` Usage

On the receiver side, the first argument is usually:

this

The second argument is usually an empty string:

""

Example:

```
uvm_config_db#(int)::get(
  this,
  "",
  "control",
  ctrl
);
```

Here:

- The component calls `get()` from its own hierarchy.
- The key is `"control"`.
- The retrieved value is stored in `ctrl`.

17. Top Module vs UVM Component Usage

From Top Module

The top module is not a UVM component.

So, when using `set()` from the top module:

```
uvm_config_db#(vif_type)::set(
  null,
  "uvm_test_top.env.driver",
  "vif",
  vif
);
```

Use `null` as the first argument.

From UVM Component

If `set()` is used inside a UVM component, use:

this

Example:

```
uvm_config_db#(int)::set(
  this,
  "env.*",
  "control",
  ctrl
);
```

18. Example Requirement

Consider this UVM hierarchy:

```
test
 └── environment
     ├── class A
     └── class B
         └── class C
```

Requirement:

- `class C` should be created only if `control = 1`.
- If `control = 0`, `class C` should not be created.
- The `control` value is set by the test.
- `class B` receives the `control` value using UVM Config DB.
- Based on the value, `class B` decides whether to create `class C`.

19. Class A

`class A` extends:

uvm_component

It is registered with the factory using:

`uvm_component_utils(class_a)

It contains a display function that prints a message from inside class A.

20. Class C

`class C` also extends:

uvm_component

It is registered with the factory.

It contains a display function that prints a message from inside class C.

This class is created conditionally from class B.

21. Class B

`class B` extends:

uvm_component

It contains an integer variable:

int ctrl;

Inside its `build_phase`, it retrieves the control value from the config DB.

Example:

```
if (!uvm_config_db#(int)::get(this, "", "control", ctrl)) begin
  `uvm_fatal(get_type_name(), "get function failed")
end
```

If the `get()` fails, a fatal message is printed.

For `uvm_fatal`, no verbosity argument is required.

22. Conditional Creation of Class C

After receiving `ctrl`, class B checks the value.

```
if (ctrl == 1) begin
  c_h = class_c::type_id::create("c_h", this);
  c_h.display();
end
```

Meaning:

- If `ctrl = 1`, class C is created.
- If `ctrl = 0`, class C is not created.

23. Environment Class

The environment extends:

uvm_env

It creates:

- Class A
- Class B

inside its `build_phase`.

Example:

```
a_h = class_a::type_id::create("a_h", this);
b_h = class_b::type_id::create("b_h", this);
```

24. Test Class

The test extends:

uvm_test

It creates the environment inside its `build_phase`.

It also sets the control value into the config DB.

Example:

```
int ctrl = 1;

uvm_config_db#(int)::set(
  this,
  "*",
  "control",
  ctrl
);
```

This allows components matching the path to retrieve the `control` value.

25. Top Module

The top module starts the UVM test using:

```
run_test("test");
```

This starts the UVM phase mechanism and builds the hierarchy.

26. Output When `ctrl = 1`

If `ctrl` is set to `1`:

- Class B retrieves the value successfully.
- Class C is created.
- Class C display message is printed.

This confirms that Config DB successfully passed the control value from the test to class B.

27. Effect of Changing the Path

The path in the `set()` method controls which components can access the config value.

Case 1: Path = `"env"`

If the path is set only to:

"env"

then class B may not be able to retrieve the value.

Result:

- `get()` fails in class B.
- Fatal message is displayed.

Case 2: Path = `"env.*"`

If the path is set to:

"env.*"

then all components under the environment can access the value.

Result:

- Class B can retrieve the value.
- Class C is created if `ctrl = 1`.

28. Importance of Path Control

The instance path controls access to config DB values.

It can be used to:

- Allow only one component to access a value.
- Allow all subcomponents under a hierarchy to access a value.
- Restrict access to unrelated components.
- Avoid accidental configuration sharing.

29. Key Points to Remember

- UVM Config DB is used to pass configuration data through the UVM hierarchy.
- It avoids manual constructor-based passing.
- It is commonly used for virtual interface handles and control values.
- `uvm_config_db` is a parameterized class.
- The type used in `set()` and `get()` must match.
- Config DB methods are static.
- Static methods are accessed using `::`.
- `set()` stores data in the config DB.
- `get()` retrieves data from the config DB.
- `exists()` checks if a key exists.
- `unset()` removes a stored value.
- Each stored value is associated with a key.
- Sender and receiver must use the same key.

- In `set()`, the path controls which components can access the data.
- In `get()`, the path is usually left empty.
- Use `null` as context when calling `set()` from the top module.
- Use `this` as context when calling `set()` or `get()` from a UVM component.
- `run_test()` creates the UVM hierarchy and starts the phase mechanism.
- Config DB can control whether components are created based on configuration values.

30. Final Summary

UVM Config DB is a flexible mechanism for passing configuration information between testbench components.

A higher-level component stores a value using `set()`, and a lower-level component retrieves it using`get()`.

The value is accessed using a matching key, and the instance path controls which components are allowed to retrieve it.

In the example, the test sends a `control` value to class B. Class B uses that value to decide whether class C should be created.

UVM Sequencer and Driver

Introduction to UVM Sequencer and Driver

1. Topic Overview

This section explains the role of the **UVM driver** and **UVM sequencer**, and how they communicate with each other.

Main topics covered:

- Role of a UVM driver
- Difference between SystemVerilog driver flow and UVM driver flow
- Role of sequences and sequence items
- Role of the sequencer
- Driver-sequencer TLM connection
- Driver methods for getting sequence items
- Difference between `get_next_item/item_done` and `get/put`

2. Role of a Driver

A driver is a UVM component that interacts directly with the DUT.

Its main job is to convert transaction-level data into pin-level signal activity.

The driver:

- Receives randomized transactions or sequence items
- Converts them into pin-level activity
- Drives them to the DUT through an interface

In UVM, randomized transactions are usually called **sequence items**.

3. Driver and DUT Interface

The driver communicates with the DUT using an interface handle.

Example DUT signals may include:

- `write_address`
- `write_enable`
- `data_in`

The driver receives these values as transaction fields and drives them onto the interface signals connected to the DUT.

4. Driver in SystemVerilog Testbench

In a basic SystemVerilog testbench, a transaction class contains randomized fields.

Example transaction fields:

rand write_address;
rand write_enable;
rand data_in;

A generator creates and randomizes this transaction object.

The randomized transaction is sent from the generator to the driver using a mailbox.

Generator Side

The generator performs:

mbox.put(tr);

Here, `tr` is the randomized transaction object.

Driver Side

The driver receives the transaction using:

mbox.get(tr);

Then the driver copies transaction fields to interface signals.

Example:

intf.write_address = tr.write_address;
intf.write_enable = tr.write_enable;
intf.data_in   = tr.data_in;

So, in SystemVerilog:

- Generator randomizes the transaction.
- Mailbox transfers the transaction.
- Driver gets the transaction.
- Driver drives the DUT interface.

5. Driver in UVM

In UVM, the flow is different.

Instead of generator-to-driver mailbox communication, UVM uses:

- Sequence
- Sequencer
- Driver

Basic flow:

Sequence → Sequencer → Driver → Interface → DUT

The sequence creates sequence items.

The sequencer sends those sequence items to the driver.

The driver converts the sequence items into pin-level activity and drives the DUT.

6. What Is a Sequence?

A sequence is a UVM object that generates sequence items.

It contains or controls the transaction data that the driver needs to send to the DUT.

A sequence may generate one or multiple sequence items.

In a testbench, there can be many sequences.

The sequencer decides which sequence item is passed to the driver.

7. What Is a Sequence Item?

A sequence item is similar to a transaction object.

It contains the fields required by the driver.

Example fields:

- Address
- Data
- Control signals
- Read/write information

The driver receives the sequence item and converts it into interface signal activity.

Sequence items are discussed in more detail separately.

8. Role of the Sequencer

The sequencer acts as a mediator between the sequence and the driver.

Its main job is to pass sequence items from the sequence to the driver.

The sequencer:

- Receives sequence items from sequences
- Stores or manages sequence items internally
- Provides sequence items to the driver
- Handles driver-sequence communication

The driver does not directly talk to the sequence. It communicates with the sequencer.

9. Driver Class in UVM

A user-defined driver must extend `uvm_driver`.

Example:

class driver extends uvm_driver #(write_transaction);

Here:

- `driver` is the user-defined driver class.
- `uvm_driver` is the built-in UVM driver base class.
- `write_transaction` is the sequence item type.

10. UVM Driver as a Parameterized Class

`uvm_driver` is a parameterized class.

It can take two type parameters:

uvm_driver #(REQ, RSP)

Where:

- `REQ` means request sequence item
- `RSP` means response sequence item

`RSP` is optional.

Usually, `REQ` and `RSP` are of the same class type unless explicitly defined differently.

Example:

class driver extends uvm_driver #(write_transaction);

In this case, the request item type is `write_transaction`.

11. Virtual Interface in UVM Driver

The driver needs an interface handle to drive DUT pins.

In SystemVerilog, this interface handle may be passed through the constructor using `new`.

In UVM, the interface handle is usually passed using **UVM Config DB**.

Typical idea:

- Top module sets the virtual interface into Config DB.
- Driver retrieves the virtual interface from Config DB.
- Driver uses that virtual interface to drive DUT signals.

The main focus here is not Config DB, but driver-sequencer communication.

12. Driver-Sequencer Connection

The driver and sequencer communicate using predefined UVM TLM ports.

The sequencer provides sequence items.

The driver pulls sequence items from the sequencer.

This connection is usually made inside the **agent**.

13. Predefined TLM Handles

Unlike manually created TLM ports, the UVM driver and sequencer already contain predefined handles.

In Driver

The driver has:

seq_item_port

This is used by the driver to request sequence items from the sequencer.

It is of type:

uvm_seq_item_pull_port

In Sequencer

The sequencer has:

seq_item_export

This provides sequence items to the driver.

It is of type:

uvm_seq_item_pull_export

These handles are predefined. The user does not need to create them using `new`.

14. Driver-Sequencer TLM Connection

The driver and sequencer are connected in the agent.

Typical connection:

driver_h.seq_item_port.connect(sequencer_h.seq_item_export);

This allows the driver to pull sequence items from the sequencer.

15. Sequencer Request FIFO Concept

The sequencer can be thought of as having a request FIFO.

Sequence items generated by sequences are placed into the sequencer.

The driver pulls these items from the sequencer request FIFO.

The driver can retrieve sequence items one by one and drive them to the DUT.

16. Driver Methods for Getting Sequence Items

There are two common approaches for driver-sequencer communication:

1. `get_next_item()` / `try_next_item()` with `item_done()`
2. `get()` / `peek()` / `put()`

17. `get_next_item()`

`get_next_item()` is a blocking task.

It waits until a request sequence item is available in the sequencer request FIFO.

Example:

seq_item_port.get_next_item(req);

Here:

- `seq_item_port` is the driver's predefined TLM port.
- `req` is the request sequence item handle.

When an item becomes available, it is copied or assigned into `req`.

The driver can then use `req` to drive the DUT.

18. Behaviour of `get_next_item()`

`get_next_item()` blocks until an item is available.

This means:

- If no sequence item is available, the driver waits.
- If a sequence item is available, the driver receives it.
- The driver must later call `item_done()` to complete the handshake.

19. `try_next_item()`

`try_next_item()` is a non-blocking method.

It checks whether a sequence item is available.

If an item is available:

- It returns the item handle.

If no item is available:

- It returns `null`.

Unlike `get_next_item()`, it does not wait.

20. `item_done()`

`item_done()` is a non-blocking function.

It must be called after:

- `get_next_item()`
- Successful `try_next_item()`

Example:

seq_item_port.item_done();

The purpose of `item_done()` is to complete the driver-sequencer handshake.

It tells the sequencer that the driver has finished processing the current sequence item.

21. Importance of `item_done()`

Calling `item_done()` is mandatory when using `get_next_item()` or successful `try_next_item()`.

If `item_done()` is not called:

- The driver-sequencer handshake is not completed.
- The next sequence item may not be issued.
- The sequence may remain blocked.

`item_done()` may optionally pass a response item back to the sequencer.

If no response is needed, it can be called without an argument.

22. `get()` Method

`get()` is a blocking method.

It waits until a request item is available in the sequencer request FIFO.

Example:

seq_item_port.get(req);

It retrieves a sequence item from the sequencer.

Unlike the `get_next_item()` approach, repeated `get()` calls can request the next item directly without requiring `item_done()`.

23. `peek()` Method

`peek()` is also a blocking method.

It waits until a request item is available.

However, it does not consume the item.

This means:

- `peek()` gives access to the current item.
- The item remains available.
- A later `peek()` or `get()` may return the same item.

24. `put()` Method

`put()` is a non-blocking method.

It is used to place a response item into the sequencer response FIFO.

Example:

seq_item_port.put(rsp);

Important points:

- `put()` is used for response handling.
- It is not part of the main `get_next_item()` / `item_done()` handshake.
- It can be called at any time if a response needs to be sent.
- In the `get/put` approach, `put()` should pass a response sequence item.

25. `get_next_item()` / `item_done()` Approach

In this approach:

1. Driver calls `get_next_item(req)`.
2. Driver receives a sequence item.
3. Driver drives the item to the DUT.
4. Driver calls `item_done()`.
5. Sequencer can send the next item.

This approach requires `item_done()` before requesting the next sequence item.

26. `get()` / `put()` Approach

In this approach:

1. Driver calls `get(req)`.
2. Driver receives a sequence item.
3. Driver can call `get(req)` again to get the next item.
4. If a response is required, driver calls `put(rsp)`.

Here, `get()` can retrieve the next request item without waiting for `item_done()`.

27. Difference Between the Two Approaches

Feature	`get_next_item()` / `item_done()`	`get()` / `put()`
Request method	`get_next_item()`	`get()`
Handshake completion	`item_done()` required	`put()` used only for response
Can get next item immediately?	No, must call `item_done()` first	Yes, repeated `get()` can retrieve next item
Response argument	Optional in `item_done()`	Required when using `put()` for response
Typical use	Common driver-sequencer handshake	Request-response style flow

28. Driver `run_phase` Concept

A typical driver `run_phase` uses the sequencer port to receive items.

Example concept:

```
task run_phase(uvm_phase phase);
  forever begin
    seq_item_port.get_next_item(req);

    // Drive req fields to DUT through interface

    seq_item_port.item_done();
  end
endtask
```

This loop continuously receives sequence items and drives them to the DUT.

29. Summary of Data Flow

The complete UVM flow is:

Sequence
↓
Sequencer
↓
Driver
↓
Interface
↓
DUT

The sequence generates sequence items.

The sequencer manages and forwards them.

The driver retrieves the items.

The driver drives the DUT through the interface.

30. Where Connection Is Made

The driver and sequencer are usually connected inside the agent.

Example:

drv_h.seq_item_port.connect(seqr_h.seq_item_export);

The agent contains both:

- Sequencer
- Driver

So, the agent is the correct place to connect them.

31. Key Points to Remember

- A UVM driver converts transaction-level data into pin-level DUT activity.
- A UVM driver extends `uvm_driver`.
- `uvm_driver` is parameterized by request and optional response sequence item types.
- A sequence item is similar to a transaction object.
- A sequence generates sequence items.
- A sequencer passes sequence items from sequences to the driver.
- The sequencer acts as a mediator between sequence and driver.
- Driver and DUT communicate through a virtual interface.
- The virtual interface is commonly passed to the driver through UVM Config DB.
- Driver and sequencer communicate using predefined TLM handles.
- Driver has `seq_item_port`.
- Sequencer has `seq_item_export`.
- These are connected inside the agent.

- `get_next_item()` is blocking.
- `try_next_item()` is non-blocking.
- `item_done()` completes the driver-sequencer handshake.
- `get()` is blocking and can retrieve request items directly.
- `peek()` observes an item without consuming it.
- `put()` is used to send response items back to the sequencer.

32. Final Summary

The UVM driver receives sequence items from the sequencer and drives them to the DUT using a virtual interface.

The sequencer acts as the bridge between sequences and the driver.

The driver and sequencer are connected through built-in TLM handles:

driver.seq_item_port.connect(sequencer.seq_item_export);

The driver can retrieve sequence items using methods such as `get_next_item()`, `try_next_item()`, `get()`, and `peek()`.

When using `get_next_item()` or `try_next_item()`, the driver must call `item_done()` to complete the handshake and allow the next sequence item to be issued.

UVM Sequence Item and UVM Sequence

UVM Sequence Item and UVM Sequence

1. Topic Overview

This section explains two important UVM concepts:

- **UVM sequence item**
- **UVM sequence**

A sequence item represents the transaction data that must be sent to the DUT.

A sequence creates, randomizes, and sends sequence items to the sequencer.

2. What Is a UVM Sequence Item?

A **UVM sequence item** is one of the fundamental building blocks in UVM.

It represents:

- A single transaction
- A group of related data fields
- A packet of information passed between testbench components
- Data that the driver eventually drives to the DUT

A sequence item is similar to a transaction object in SystemVerilog.

3. Sequence Item Class Hierarchy

The sequence item is based on the UVM class hierarchy.

Basic hierarchy:

```
uvm_void
 └── uvm_object
     └── uvm_transaction
         └── uvm_sequence_item
             └── user-defined sequence item
```

A user-defined transaction or sequence item is usually derived from:

uvm_sequence_item

4. Sequence-Related Class Hierarchy

The sequence-related hierarchy continues from the UVM object branch.

Conceptually:

```
uvm_sequence_item
 └── uvm_sequence_base
     └── uvm_sequence
         └── user-defined sequence
```

A user-defined sequence is derived from:

uvm_sequence

5. Where Sequence Items Are Used

Sequence items are generated inside a sequence.

The sequence sends the sequence item to the sequencer.

The sequencer forwards the sequence item to the driver.

The driver then converts the sequence item fields into pin-level activity and drives the DUT.

Basic flow:

Sequence → Sequencer → Driver → Interface → DUT

6. Role of a Sequence Item

A sequence item acts as a data container.

It can carry fields such as:

- Address
- Data
- Read/write control
- Burst type
- Transfer type
- Protocol-specific signals

For example, in a bus protocol transaction, the sequence item may contain all fields needed for one bus transfer.

7. AHB Example

Assume the DUT is an AHB slave.

The DUT may require signals such as:

- `HWRITE`
- `HTRANS`
- `HBURST`

The driver must drive these signals to the DUT through an interface.

The sequencer does not send these values one by one.

Instead, the values are grouped inside a sequence item object.

The sequence item may contain:

```
rand bit       hwrite;
rand bit [1:0] htrans;
rand bit [2:0] hburst;
```

This object is then passed from the sequence to the sequencer, then from the sequencer to the driver.

8. Sequence Item as an Object

The sequence item is an object, not a component.

It is not part of the UVM component hierarchy.

Component hierarchy includes:

- Test
- Environment
- Agent
- Sequencer
- Driver
- Monitor
- Scoreboard

Sequence items and sequences are objects used to generate and carry transaction data.

9. Difference Between Sequence, Sequencer, and Sequence Item

Concept	Type	Main Role
Sequence item	Object	Holds transaction fields
Sequence	Object	Creates, randomizes, and sends sequence items
Sequencer	Component	Passes sequence items to the driver
Driver	Component	Converts sequence items into pin-level DUT activity

10. Data Flow Explanation

The sequence item contains all randomized transaction fields.

The sequence:

1. Creates a sequence item object.
2. Randomizes the sequence item fields.
3. Sends the sequence item to the sequencer.

The sequencer:

1. Receives the sequence item from the sequence.
2. Provides it to the driver.

The driver:

1. Gets the sequence item from the sequencer.
2. Extracts the fields.
3. Drives the DUT pins through the interface.

11. Declaring a Sequence Item

A sequence item class is declared by extending `uvm_sequence_item`.

Example:

```
class ahb_sequence_item extends uvm_sequence_item;

  rand bit       hwrite;
  rand bit [1:0] htrans;
  rand bit [2:0] hburst;

  `uvm_object_utils(ahb_sequence_item)

  function new(string name = "ahb_sequence_item");
    super.new(name);
  endfunction

endclass
```

12. Important Points in Sequence Item Declaration

A sequence item should contain:

- Data fields required by the driver
- `rand` keyword for fields that need randomization
- Factory registration using `uvm_object_utils`
- Constructor with only the `name` argument

Since it is an object, it does not need a parent argument.

13. What Is a UVM Sequence?

A **UVM sequence** is an object that generates sequence items.

It defines the stimulus operation.

The sequence creates and controls transaction generation.

The sequence sends sequence items to the driver through the sequencer.

14. UVM Sequence as a Parameterized Class

`uvm_sequence` is a parameterized class.

The parameter is the sequence item type.

Example:

class ahb_sequence extends uvm_sequence #(ahb_sequence_item);

Here:

- `ahb_sequence` is the user-defined sequence.
- `uvm_sequence` is the base class.
- `ahb_sequence_item` is the sequence item type generated by the sequence.

15. Declaring a UVM Sequence

Basic sequence declaration:

```
class ahb_sequence extends uvm_sequence #(ahb_sequence_item);

  `uvm_object_utils(ahb_sequence)

  function new(string name = "ahb_sequence");
    super.new(name);
  endfunction

  task body();
    // sequence operations are written here
  endtask

endclass
```

16. Role of the `body()` Task

The main operation of a sequence is written inside the `body()` task.

The `body()` task defines what the sequence should do.

Typical operations inside `body()` include:

1. Creating the sequence item.
2. Randomizing the sequence item.
3. Sending the sequence item to the sequencer.
4. Optionally receiving a response from the sequencer or driver.

17. `pre_body()` and `post_body()`

Along with `body()`, UVM also supports:

- `pre_body()`
- `post_body()`

These are optional callback-style tasks.

`pre_body()`

Used for operations that must happen before the main `body()` task.

`post_body()`

Used for operations that must happen after the main `body()` task.

These are similar in concept to `pre_randomize()` and `post_randomize()` in SystemVerilog randomization.

18. Steps Performed Inside a Sequence

Inside a sequence, the common steps are:

1. Create the sequence item instance.
2. Randomize the sequence item fields.
3. Send the randomized sequence item to the driver through the sequencer.

These operations are usually written inside the `body()` task.

19. Creating a Sequence Item

Before randomizing a sequence item, an object must be created.

Example:

ahb_sequence_item req;

req = ahb_sequence_item::type_id::create("req");

Without creating the object, the sequence cannot randomize or send it.

20. Randomizing the Sequence Item

After creating the sequence item, randomize it:

assert(req.randomize());

This randomizes fields declared with the `rand` keyword.

Example fields:

```
rand bit       hwrite;
rand bit [1:0] htrans;
rand bit [2:0] hburst;
```

21. Sending the Sequence Item

After randomization, the sequence item must be sent to the sequencer.

The sequencer then provides it to the driver.

This is the main purpose of the sequence.

22. Methods for Sending Sequence Items

There are different ways to create, randomize, and send sequence items.

Two common approaches are:

1. Using UVM sequence macros
2. Using built-in sequence methods

23. UVM Sequence Macros

UVM provides macros such as:

```
`uvm_do
`uvm_create
`uvm_send
```

These macros can help create, randomize, and send sequence items.

They reduce manual coding.

24. Built-In Sequence Methods

Instead of macros, built-in methods from the base sequence class can be used.

Common methods include:

```
wait_for_grant()
send_request()
wait_for_item_done()
```

Another commonly used method pair is:

```
start_item()
finish_item()
```

These methods provide more control than macros.

25. Purpose of These Methods

All these methods are used to perform the same basic sequence operations:

1. Create the item.
2. Randomize the item.
3. Send the item to the sequencer.
4. Complete the sequence item transfer.

26. Recommended Conceptual Flow

A clear sequence flow is:

```
task body();

  ahb_sequence_item req;

  req = ahb_sequence_item::type_id::create("req");

  start_item(req);

  assert(req.randomize());

  finish_item(req);

endtask
```

Meaning:

- `create()` creates the sequence item.
- `start_item()` starts the sequence item handshake with the sequencer.
- `randomize()` generates valid randomized values.
- `finish_item()` sends the item to the sequencer.

27. Complete Data Flow Summary

The overall UVM stimulus flow is:

Sequence item:
  Holds randomized fields such as HWRITE, HTRANS, HBURST

Sequence:
  Creates and randomizes the sequence item

Sequencer:
  Receives the sequence item from the sequence

Driver:

Gets the sequence item from the sequencer

Interface:
Carries pin-level signals from driver to DUT

DUT:
Receives the driven signal activity

28. Key Points to Remember

- A sequence item is a transaction object.
- A sequence item holds protocol-specific data fields.
- Sequence item fields are usually declared with `rand`.
- A sequence item extends `uvm_sequence_item`.
- A sequence item is registered using `uvm_object_utils`.
- A sequence creates and randomizes sequence items.
- A sequence extends `uvm_sequence #(sequence_item_type)`.
- A sequence is not part of the UVM component hierarchy.
- A sequencer is a component and acts as a bridge between sequence and driver.
- The driver receives sequence items from the sequencer.
- The driver converts sequence item fields into pin-level DUT signals.
- The `body()` task contains the main sequence operation.
- `pre_body()` and `post_body()` are optional callbacks.
- Sequence items can be sent using UVM macros or built-in sequence methods.
- Common sequence methods include `start_item()` and `finish_item()`.

29. Final Summary

A UVM sequence item is a transaction object that stores the data needed by the driver.

A UVM sequence creates, randomizes, and sends sequence items to the sequencer.

The sequencer passes those items to the driver, and the driver drives the DUT through an interface.

In simple terms:

Sequence item = data packet
Sequence = packet generator
Sequencer = bridge
Driver = pin-level driver
DUT = design being verified

UVM Sequence Part 2: Sequence Macros and Methods

1. Topic Overview

This section explains how a UVM sequence creates, randomizes, and sends sequence items to the driver through the sequencer.

The main approaches are:

- UVM sequence macros
- Built-in sequence methods
- `start_item()` and `finish_item()`

The goal of all these approaches is the same:

1. Create a sequence item.
2. Randomize the sequence item.
3. Send the sequence item to the driver through the sequencer.

2. UVM Sequence Macros

UVM provides sequence macros to simplify sequence item handling.

These macros can perform common operations such as:

- Creating a sequence item
- Randomizing a sequence item
- Sending a sequence item to the driver
- Applying inline constraints
- Applying priority

3. `uvm_do Macro

The basic macro is:

`uvm_do(sequence_item)

This macro performs three operations in one line:

1. Creates the sequence item.
2. Randomizes the sequence item.
3. Sends the sequence item to the driver through the sequencer.

It is usually called inside the sequence `body()` task.

4. `uvm_do_with Macro

Syntax:

`uvm_do_with(sequence_item, constraint)

This is similar to `uvm_do, but it applies constraints during randomization.

It performs:

1. Sequence item creation
2. Randomization with the given constraint
3. Sending the item to the driver

This works like inline constraints in SystemVerilog randomization.

5. `uvm_do_pri Macro

Syntax:

`uvm_do_pri(sequence_item, priority)

This is similar to `uvm_do, but it also considers the given sequence item priority.

Priority handling is useful when multiple sequence items or sequences compete for sequencer access.

6. `uvm_do_pri_with Macro

Syntax:

`uvm_do_pri_with(sequence_item, priority, constraint)

This macro combines both features:

- Priority
- Inline constraints

It creates, randomizes with constraints, assigns priority, and sends the sequence item.

7. `uvm_create Macro

Syntax:

`uvm_create(sequence_item)

This macro only creates the sequence item.

It does not:

- Randomize the item
- Send the item to the driver

If this macro is used, randomization and sending must be done separately.

8. `uvm_send Macro

Syntax:

`uvm_send(sequence_item)

This macro only sends the sequence item.

It does not:

- Create the item
- Randomize the item

Before using `uvm_send, the sequence item must already be created and randomized by the user.

9. `uvm_rand_send Macro

Syntax:

`uvm_rand_send(sequence_item)

This macro randomizes and sends the sequence item.

It does not create the item.

Before using this macro, the sequence item must already be created.

10. `uvm_rand_send_with Macro

Syntax:

`uvm_rand_send_with(sequence_item, constraint)

This macro:

1. Randomizes the already-created sequence item using the given constraint.
2. Sends it to the driver through the sequencer.

It does not create the sequence item.

11. Priority-Based Rand Send Macros

UVM also provides priority-based send macros.

These are used when sequence item priority must be considered.

Examples:

`uvm_rand_send_pri(sequence_item, priority)

`uvm_rand_send_pri_with(sequence_item, priority, constraint)

These macros send randomized items with priority support.

12. Sequence Macro Summary

Macro	Creates Item	Randomizes Item	Sends Item	Supports Constraint	Supports Priority
`uvm_do	Yes	Yes	Yes	No	No
`uvm_do_with	Yes	Yes	Yes	Yes	No
`uvm_do_pri	Yes	Yes	Yes	No	Yes
`uvm_do_pri_with	Yes	Yes	Yes	Yes	Yes
`uvm_create	Yes	No	No	No	No
`uvm_send	No	No	Yes	No	No
`uvm_rand_send	No	Yes	Yes	No	No
`uvm_rand_send_with	No	Yes	Yes	Yes	No

13. Important Note on Sequence Macros

UVM sequence macros do not invoke:

- `pre_body()`
- `post_body()`

Also, sequence macros are generally not recommended for large professional environments because they expand into more code and may slow down simulation.

They are useful for learning and quick examples, but built-in methods are usually preferred for better control.

14. Built-In Sequence Methods

Instead of macros, UVM provides built-in methods in the `uvm_sequence_base` class.

These methods provide more control over sequence item creation, randomization, and transfer.

There are two main method-based approaches:

1. Approach A: `create_item()`, `wait_for_grant()`, `send_request()`, `wait_for_item_done()`, `get_response()`

2. Approach B: `start_item()` and `finish_item()`

15. Approach A: Detailed Method Flow

Approach A uses multiple explicit methods.

The main steps are:

1. Create the sequence item.
2. Wait for sequencer grant.
3. Randomize the item.
4. Send the request.
5. Wait for item completion.
6. Optionally get response from the driver.

16. `create_item()`

`create_item()` is a function.

It creates and initializes the sequence item using the factory.

Purpose:

- Creates the sequence item instance
- Initializes the request item or sequence

Example concept:

req = create_item(...);

It is used before randomization and sending.

17. `wait_for_grant()`

`wait_for_grant()` is a task.

It sends a request to the current sequencer and waits until the sequencer grants access.

Since it waits, it is a blocking method.

Purpose:

- Requests permission from the sequencer
- Blocks until the sequencer grants the sequence

Conceptually:

Sequence asks sequencer for permission.
Sequencer grants access.
Sequence can proceed.

18. `send_request()`

`send_request()` sends the sequence item to the driver through the sequencer.

It is called after the sequencer grants access.

Purpose:

- Sends the randomized sequence item to the sequencer
- The sequencer forwards the item to the driver

If the `rerandomize` argument is set to `1`, the item is randomized before being sent.

By default, rerandomization is usually disabled.

19. `wait_for_item_done()`

`wait_for_item_done()` is a task.

It blocks until the driver calls:

- `item_done()`
- or `put()`

This method is used when the sequence needs to wait until the driver completes processing the item.

It is optional, depending on the sequence-driver communication style.

20. `get_response()`

`get_response(rsp)` is used to receive a response from the driver.

It is used when the driver sends a response item back to the sequence through the sequencer.

Example concept:

get_response(rsp);

Here, `rsp` is the response sequence item.

21. Approach A Flow

A typical Approach A sequence flow is:

req = create_item(...);

wait_for_grant();

assert(req.randomize());

send_request(req);

wait_for_item_done();

get_response(rsp);

Meaning:

- Create the item.
- Wait for sequencer permission.
- Randomize the item.
- Send it to the driver.
- Wait for driver completion.
- Optionally collect response.

22. Approach B: `start_item()` and `finish_item()`

Approach B uses two main tasks:

start_item(req);
finish_item(req);

These tasks are defined in the `uvm_sequence_base` class.

They are commonly used because they are simpler and cleaner than Approach A.

23. `start_item()`

`start_item(req)` starts the sequence item operation.

It blocks until the sequencer grants access to the sequence.

Internally, it performs grant-related handling similar to `wait_for_grant()`.

Purpose:

- Waits for sequencer grant
- Begins the transaction handshake

24. `finish_item()`

`finish_item(req)` completes the sequence item operation.

It sends the request item to the sequencer so that it can be forwarded to the driver.

Internally, it calls methods such as:

- `send_request()`
- `wait_for_item_done()`

Purpose:

- Sends the randomized item
- Waits for the driver-side transfer protocol to complete

25. Important Rule for `start_item()` and `finish_item()`

There should be no simulation delay between:

start_item(req);

and

finish_item(req);

The usual flow is:

```
start_item(req);
assert(req.randomize());
finish_item(req);
```

Randomization is allowed between them, but delay should be avoided.

26. Approach B Flow

A typical sequence body using Approach B is:

```
task body();

  req = sequence_item::type_id::create("req");

  start_item(req);

  assert(req.randomize());

  finish_item(req);

endtask
```

Meaning:

1. Create the sequence item.
2. Wait for sequencer grant using `start_item()`.
3. Randomize the sequence item.
4. Send it using `finish_item()`.

27. Approach A vs Approach B

Feature	Approach A	Approach B
Methods used	`wait_for_grant()`, `send_request()`, `wait_for_item_done()`, `get_response()`	`start_item()`, `finish_item()`
Control level	More explicit control	Simpler wrapper-based flow
Grant handling	Done using `wait_for_grant()`	Done inside `start_item()`
Sending item	Done using `send_request()`	Done inside `finish_item()`
Completion wait	Done using `wait_for_item_done()`	Handled inside `finish_item()`
Ease of use	More detailed	Easier and commonly used

28. Main Purpose of All Methods

Whether macros or methods are used, the goal remains the same:

Create sequence item → Randomize sequence item → Send to driver through sequencer

Macros do this in fewer lines.

Built-in methods give better control and are generally preferred.

29. Sequence and Sequencer Relationship

A sequence is a standalone object.

It is not part of the UVM component hierarchy.

A sequencer is a UVM component.

It is part of the UVM testbench hierarchy.

This raises an important question:

How does a standalone sequence know which sequencer to send items to?

This is handled when the sequence is started on a specific sequencer.

That topic is discussed separately.

30. Key Points to Remember

- UVM sequence macros simplify sequence item operations.

- `` `uvm_do `` creates, randomizes, and sends an item.
- `` `uvm_do_with `` also applies inline constraints.
- `` `uvm_create `` only creates the item.
- `` `uvm_send `` only sends an already-created item.
- `` `uvm_rand_send `` randomizes and sends an already-created item.
- Sequence macros do not call `pre_body()` or `post_body()`.
- Sequence macros may slow simulation and are generally not preferred for large projects.
- Built-in sequence methods provide better control.
- `create_item()` creates the item using the factory.
- `wait_for_grant()` waits for sequencer permission.
- `send_request()` sends the item through the sequencer.
- `wait_for_item_done()` waits until the driver completes the item.
- `get_response()` retrieves a driver response.
- `start_item()` and `finish_item()` are wrapper methods over the detailed method flow.
- `start_item()` waits for sequencer grant.
- `finish_item()` sends the request and handles completion.
- No delay should be placed between `start_item()` and `finish_item()`.
- A sequence is an object, while a sequencer is a component.

31. Final Summary

UVM sequences use sequence items to generate stimulus for the driver.

Inside the sequence `body()` task, the sequence item must be created, randomized, and sent to the driver through the sequencer.

This can be done using UVM macros such as `` `uvm_do `` or by using built-in sequence methods.

For better control and professional testbench development, the `start_item()` and `finish_item()` approach is commonly preferred.

UVM Sequence start() Method

UVM Sequence `start()` Method

1. Topic Overview

This section explains how a **UVM sequence communicates with a sequencer**.

Main topics covered:

- Why the `start()` method is needed
- How a sequence finds the sequencer
- Role of `m_sequencer`
- How `start()` connects sequence and sequencer
- How the sequence `body()` task begins execution

2. Sequence and Sequencer Relationship

A **sequence** is a standalone UVM object.

It is not part of the UVM testbench component hierarchy.

A **sequencer** is a UVM component.

It is part of the testbench hierarchy.

Example hierarchy:

```
test
 └── environment
     └── agent
         ├── sequencer
         └── driver
```

Since the sequence is not part of this hierarchy, it does not automatically know where the sequencer is located.

3. Main Question

The important question is:

How does a standalone sequence know which sequencer to send sequence items to?

This is solved using the UVM sequence `start()` method.

4. What Does Starting a Sequence Mean?

Starting a sequence means executing a sequence on a specified sequencer.

In simple terms:

Starting a sequence = running a sequence on a sequencer

When a sequence is started on a sequencer:

1. The sequence connects to that sequencer.
2. The sequence begins execution.
3. The sequence generates sequence items.
4. The sequencer forwards those items to the driver.

5. `start()` Method

The `start()` method is used to start a sequence on a sequencer.

Typical syntax:

```
sequence_h.start(sequencer_h);
```

Here:

- `sequence_h` is the sequence handle.
- `sequencer_h` is the sequencer handle.
- The argument tells the sequence which sequencer to use.

6. Where `start()` Is Called

The `start()` method is usually called inside the **test class**.

Typical flow inside the test:

1. Declare a sequence handle.
2. Create the sequence object.
3. Call `start()` using the sequence handle.
4. Pass the sequencer handle or path as the argument.

Example concept:

```
seq_h = sequence_type::type_id::create("seq_h");
seq_h.start(env_h.agent_h.sequencer_h);
```

This starts the sequence on the sequencer located inside the agent.

7. Why the Sequencer Path Is Passed

The sequence must know where to send its generated sequence items.

So, when calling `start()`, the sequencer handle is passed as an argument.

Example:

seq_h.start(env_h.agent_h.sequencer_h);

This tells the sequence:

Send your sequence items to this sequencer.

8. Role of `m_sequencer`

UVM internally uses a handle called:

m_sequencer

`m_sequencer` is an internal handle used by the sequence to store the sequencer on which it is running.

When `start()` is called, the sequencer handle passed to `start()` is assigned to the sequence's internal `m_sequencer`.

9. Where `m_sequencer` Comes From

`m_sequencer` is declared inside the UVM sequence-related base classes.

It is part of the inherited sequence infrastructure.

Conceptual hierarchy:

```
uvm_object
 └── uvm_transaction
     └── uvm_sequence_item
         └── uvm_sequence_base
             └── uvm_sequence
                 └── user-defined sequence
```

Since a user-defined sequence extends `uvm_sequence`, it inherits access to the internal sequencer handle.

10. Meaning of `m_sequencer`

`m_sequencer` is a handle to the sequencer.

It allows the sequence to communicate with the sequencer after the sequence is started.

Before `start()` is called, the sequence does not know the actual sequencer instance.

After `start(sequencer_h)` is called:

m_sequencer points to sequencer_h

Now the sequence can send sequence items to that sequencer.

11. What Happens Internally When `start()` Is Called

When this is called:

seq_h.start(env_h.agent_h.sequencer_h);

UVM performs two important actions:

1. Assigns the sequencer handle to the sequence's internal `m_sequencer`.
2. Starts executing the sequence's `body()` task.

So, the `start()` method both connects the sequence to the sequencer and begins sequence execution.

12. `body()` Task Execution

The main stimulus logic of a sequence is written inside the `body()` task.

When `start()` is called, the `body()` task starts executing.

Inside `body()`, the sequence usually:

1. Creates a sequence item.
2. Randomizes the sequence item.
3. Sends the sequence item to the sequencer.
4. The sequencer forwards it to the driver.

13. Complete Flow

The complete flow is:

Test creates sequence
↓
Test calls sequence.start(sequencer)
↓
start() assigns sequencer handle to m_sequencer
↓
Sequence body() starts
↓
Sequence creates and randomizes sequence items
↓
Sequence sends items to sequencer
↓
Sequencer forwards items to driver
↓
Driver drives DUT

14. Example Hierarchy

Assume the sequencer is inside an agent, and the agent is inside an environment.

Example:

```
test
 └── env_h
     └── agent_h
         └── sequencer_h
```

The sequence can be started as:

seq_h.start(env_h.agent_h.sequencer_h);

This connects the sequence to `sequencer_h`.

15. `start()` Method Arguments

The `start()` method can take up to four arguments.

The most important argument is the first one:

start(sequencer)

The first argument is the sequencer handle.

The remaining arguments have default values and are usually not modified in beginner examples.

Conceptual form:

task start(uvm_sequencer_base sequencer, ...);

16. Important Internal Concept

A user-defined sequence does not directly belong to the testbench hierarchy.

However, once it is started on a sequencer, it gets the sequencer handle through `m_sequencer`.

This allows the sequence to send sequence items to the correct sequencer.

17. Why This Is Needed

Without `start()`:

- The sequence exists as an object.

- It is not connected to any sequencer.
- It cannot send sequence items to the driver.

With `start()`:

- The sequence is linked to a specific sequencer.
- The `body()` task executes.
- Sequence items are sent through that sequencer.

18. Key Points to Remember

- A sequence is a standalone object.
- A sequencer is a UVM component.
- The sequence is not part of the component hierarchy.
- The sequencer is part of the hierarchy.
- The sequence must be started on a sequencer.
- The `start()` method is used to start a sequence.
- `start()` is usually called from the test class.
- The sequencer handle is passed as the first argument to `start()`.
- The sequencer handle is stored internally in `m_sequencer`.
- `m_sequencer` allows the sequence to communicate with the sequencer.
- Calling `start()` also triggers the sequence `body()` task.
- The `body()` task generates and sends sequence items.
- The sequencer forwards sequence items to the driver.
- The driver drives the DUT.

19. Final Summary

The `start()` method connects a standalone UVM sequence to a specific sequencer.

When the test calls:

seq_h.start(env_h.agent_h.sequencer_h);

the sequencer handle is assigned internally to `m_sequencer`.

Then the sequence `body()` task starts running, generates sequence items, and sends them to the sequencer.

The sequencer then passes those items to the driver, which drives the DUT.

UVM Sequence, Sequencer, and Driver Communication

UVM Sequence, Sequencer, and Driver Communication

1. Topic Overview

This section explains how a **UVM sequence communicates with a sequencer and driver** using a coding example.

The example shows:

- Sequence item creation
- Sequencer declaration
- Driver declaration
- Sequence body execution
- Driver-sequencer connection
- Starting a sequence from the test
- Use of `wait_for_grant()`, `send_request()`, `wait_for_item_done()`, `get_next_item()`, and `item_done()`

2. Required UVM Setup

The UVM package and macro file must be included.

```
import uvm_pkg::*;
`include "uvm_macros.svh"
```

These are required for using UVM classes, macros, and phase methods.

3. Sequence Item Class

The sequence item represents the transaction data.

It extends:

uvm_sequence_item

Example:

```
class sequence_item extends uvm_sequence_item;

  `uvm_object_utils(sequence_item)

  rand int data;

  function new(string name = "sequence_item");
    super.new(name);
```

```
  endfunction

endclass
```

Key Points

- `sequence_item` is an object, not a component.
- It contains randomized transaction fields.
- In this example, it contains one randomized field:

```
rand int data;
```

- It is registered using:

```
`uvm_object_utils(sequence_item)
```

4. Sequencer Class

The sequencer passes sequence items from the sequence to the driver.

It extends:

```
uvm_sequencer #(sequence_item)
```

Example:

```
class sequencer extends uvm_sequencer #(sequence_item);

  `uvm_component_utils(sequencer)

  function new(string name = "sequencer", uvm_component parent = null);
    super.new(name, parent);
  endfunction

endclass
```

Key Points

- The sequencer is a UVM component.
- It is parameterized with the sequence item type.
- It is registered using:

```
`uvm_component_utils(sequencer)
```

5. Driver Class

The driver receives sequence items from the sequencer.

It extends:

uvm_driver #(sequence_item)

Example:

```
class driver extends uvm_driver #(sequence_item);

  `uvm_component_utils(driver)

  function new(string name = "driver", uvm_component parent = null);
    super.new(name, parent);
  endfunction

endclass
```

Key Points

- The driver is parameterized with the same sequence item type.
- It uses the predefined UVM TLM port:

seq_item_port

- This port is already available inside `uvm_driver`.
- The user does not need to create it manually.

6. Driver `run_phase`

The driver gets sequence items from the sequencer inside `run_phase`.

Example structure:

```
task run_phase(uvm_phase phase);

  forever begin
    seq_item_port.get_next_item(req);

    #50;

    seq_item_port.item_done();

    `uvm_info(
      get_type_name(),
      "After driver called item_done",
      UVM_LOW
    )
  end

endtask
```

Key Points

- `get_next_item(req)` gets a sequence item from the sequencer.
- It is a blocking method.
- The driver waits until a sequence item is available.
- After processing the item, the driver calls:

seq_item_port.item_done();

- `item_done()` completes the driver-sequencer handshake.
- When `item_done()` is called, the sequence-side `wait_for_item_done()` becomes unblocked.

7. Sequence Class

The sequence creates, randomizes, and sends sequence items.

It extends:

uvm_sequence #(sequence_item)

Example:

```
class sequence_ex extends uvm_sequence #(sequence_item);

  `uvm_object_utils(sequence_ex)

  sequence_item req;

  function new(string name = "sequence_ex");
    super.new(name);
  endfunction

endclass
```

Key Points

- The sequence is an object, not a component.
- It is parameterized with the sequence item type.
- It contains a sequence item handle:

sequence_item req;

8. Sequence `body()` Task

The main sequence operation is written inside the `body()` task.

Example flow:

```
task body();

  req = sequence_item::type_id::create("req");

  `uvm_info(
    get_type_name(),
    "Base sequence inside body method",
    UVM_LOW
  )

  wait_for_grant();

  assert(req.randomize());

  send_request(req);

  `uvm_info(
    get_type_name(),
    "Before wait_for_item_done",
    UVM_LOW
  )

  wait_for_item_done();

endtask
```

9. Operations Inside `body()`

The `body()` task performs the following steps:

1. Creates the sequence item object.
2. Waits for grant from the sequencer.
3. Randomizes the sequence item.
4. Sends the sequence item request to the sequencer.
5. Waits until the driver completes the item.

10. `wait_for_grant()`

`wait_for_grant()` requests permission from the sequencer.

It blocks until the sequencer grants access to the sequence.

Purpose:

Sequence asks sequencer for permission before sending an item.

11. `randomize()`

After the grant is received, the sequence item is randomized.

assert(req.randomize());

This randomizes fields declared with the `rand` keyword, such as:

rand int data;

12. `send_request()`

The sequence sends the randomized item using:

send_request(req);

This sends the sequence item to the sequencer.

The sequencer then makes it available to the driver.

13. `wait_for_item_done()`

After sending the request, the sequence waits using:

wait_for_item_done();

This method remains blocked until the driver calls:

seq_item_port.item_done();

So, `wait_for_item_done()` synchronizes the sequence with driver completion.

14. Agent Class

The agent contains both:

- Sequencer
- Driver

Example:

```
class agent extends uvm_agent;

  `uvm_component_utils(agent)

  sequencer sqr_h;
  driver    drv_h;

endclass
```

15. Agent `build_phase`

The sequencer and driver are created inside the agent `build_phase`.

```
function void build_phase(uvm_phase phase);
  super.build_phase(phase);

  sqr_h = sequencer::type_id::create("sqr_h", this);
  drv_h = driver::type_id::create("drv_h", this);
endfunction
```

Key Points

- Components are created using factory `create()`.
- The agent builds both the sequencer and driver.

16. Agent `connect_phase`

The driver and sequencer are connected inside the agent `connect_phase`.

```
function void connect_phase(uvm_phase phase);
  super.connect_phase(phase);

  drv_h.seq_item_port.connect(sqr_h.seq_item_export);
endfunction
```

Key Points

- Driver has:

seq_item_port

- Sequencer has:

seq_item_export

- This connection allows the driver to pull sequence items from the sequencer.

17. Environment Class

The environment contains the agent.

```
class environment extends uvm_env;

  `uvm_component_utils(environment)
```

```
  agent agent_h;

endclass
```

Inside the environment `build_phase`, the agent is created:

```
function void build_phase(uvm_phase phase);
  super.build_phase(phase);

  agent_h = agent::type_id::create("agent_h", this);
endfunction
```

18. Test Class

The test contains:

- Environment handle
- Sequence handle

```
environment env_h;
sequence_ex seq_h;
```

Inside the test `build_phase`, both are created:

```
function void build_phase(uvm_phase phase);
  super.build_phase(phase);

  env_h = environment::type_id::create("env_h", this);
  seq_h = sequence_ex::type_id::create("seq_h");
endfunction
```

19. Starting the Sequence

The sequence is started inside the test `run_phase`.

```
task run_phase(uvm_phase phase);
  super.run_phase(phase);

  phase.raise_objection(this);

  seq_h.start(env_h.agent_h.sqr_h);

  phase.drop_objection(this);
endtask
```

Key Points

- `start()` connects the sequence to the sequencer.
- The sequencer path is passed as an argument:

env_h.agent_h.sqr_h

- Calling `start()` triggers the sequence `body()` task.
- Objections are used to keep the `run_phase` active until the sequence completes.

20. Top Module

The top module starts the UVM test.

```
module tb;

  initial begin
    run_test("test");
  end

endmodule
```

21. Complete Communication Flow

The full flow is:

```
Test starts sequence
        ↓
Sequence body() executes
        ↓
Sequence creates and randomizes sequence_item
        ↓
Sequence sends item to sequencer
        ↓
Driver gets item using get_next_item()
        ↓
Driver processes item
        ↓
Driver calls item_done()
        ↓
Sequence wait_for_item_done() unblocks
```

22. Expected Output Flow

The output shows the order of execution:

```
Base sequence inside body method
Before wait_for_item_done
After driver called item_done
```

This confirms:

- The sequence body started.
- The sequence sent the request.
- The sequence waited for driver completion.
- The driver called `item_done()`.
- Sequence-driver communication was successful.

23. Important Method Relationship

Sequence Side	Driver Side	Meaning
`wait_for_grant()`	Sequencer grants access	Sequence waits for permission
`send_request(req)`	`get_next_item(req)`	Driver receives item
`wait_for_item_done()`	`item_done()`	Driver signals completion

24. Key Points to Remember

- `sequence_item` extends `uvm_sequence_item`.
- `sequencer` extends `uvm_sequencer #(sequence_item)`.
- `driver` extends `uvm_driver #(sequence_item)`.
- `sequence_ex` extends `uvm_sequence #(sequence_item)`.
- The sequence item contains randomized data.
- The sequence creates and randomizes the sequence item.
- The sequence sends the item using `send_request()`.
- The driver receives the item using `get_next_item()`.
- The driver completes the handshake using `item_done()`.
- The sequence waits for completion using `wait_for_item_done()`.
- The driver and sequencer are connected in the agent.
- The sequence is started from the test using `seq_h.start(sequencer_handle)`.
- `run_test("test")` starts the UVM testbench.

25. Final Summary

This example demonstrates the complete connection between a UVM sequence, sequencer, and driver.

The sequence creates and randomizes a sequence item, then sends it to the sequencer.

The driver receives the item from the sequencer using `get_next_item()` and completes the handshake using `item_done()`.

The sequence waits using `wait_for_item_done()`, so it continues only after the driver has finished processing the item.

UVM Virtual Sequence and Virtual Sequencer

UVM Virtual Sequence and Virtual Sequencer

1. Topic Overview

This section explains:

- Virtual sequence
- Virtual sequencer
- Why they are called “virtual”
- When they are needed
- How they coordinate multiple sequences and sequencers
- Difference between using and not using virtual sequence/sequencer
- Role of `m_sequencer` and `p_sequencer`

2. Normal Sequence-Sequencer-Driver Flow

In a normal UVM setup:

Sequence → Sequencer → Driver

The sequence sends sequence items to the sequencer.

The sequencer forwards the sequence items to the driver.

The sequencer and driver are connected using TLM ports.

Example:

driver.seq_item_port.connect(sequencer.seq_item_export);

Here:

- The sequence generates sequence items.
- The sequencer controls item transfer.
- The driver receives items and drives the DUT.

3. How a Normal Sequence Finds a Sequencer

A sequence is not part of the UVM component hierarchy.

A sequencer is part of the UVM hierarchy.

The sequence knows the sequencer through the internal handle:

m_sequencer

When the sequence is started using:

seq_h.start(env_h.agent_h.seqr_h);

the sequencer handle is assigned to `m_sequencer`.

Then the sequence can send sequence items to that sequencer.

4. What Is a Virtual Sequence?

A **virtual sequence** is a sequence used to coordinate other sequences.

It usually does not directly create or send sequence items to a driver.

Instead, it starts and controls other lower-level sequences.

A virtual sequence is useful when multiple protocol sequences must be controlled from one place.

Example:

Virtual Sequence
 ├── Sequence A
 └── Sequence B

The virtual sequence may start:

- Sequence A on Sequencer A
- Sequence B on Sequencer B

5. What Is a Virtual Sequencer?

A **virtual sequencer** is a sequencer used to hold handles to multiple real sequencers.

It is not connected directly to any driver.

It does not process sequence items.

It only stores references to real sequencers.

Example:

Virtual Sequencer
 ├── Sequencer A handle
 └── Sequencer B handle

The real sequencers are connected to their respective drivers.

The virtual sequencer only helps the virtual sequence access those sequencers.

6. Important Point

There is no special UVM base class called:

uvm_virtual_sequence
uvm_virtual_sequencer

A virtual sequence is still derived from:

uvm_sequence

A virtual sequencer is still derived from:

uvm_sequencer

They are called “virtual” because of their usage pattern, not because UVM provides separate virtual base classes.

7. Why They Are Called Virtual

They are called virtual because:

- A virtual sequence coordinates other sequences.
- A virtual sequencer holds handles to real sequencers.
- A virtual sequencer is not connected to any driver.
- A virtual sequencer does not generate, receive, or process sequence items directly.

So, “virtual” means it is used for coordination, not direct driving.

8. When Virtual Sequence and Virtual Sequencer Are Needed

Use virtual sequences and virtual sequencers when stimulus must be coordinated across multiple agents or interfaces.

Example multi-interface system:

SoC
 ├── UART IP
 └── Ethernet IP

The testbench may contain:

Agent 1 → Sequencer 1 → Driver 1 → UART
Agent 2 → Sequencer 2 → Driver 2 → Ethernet

If UART and Ethernet stimulus must be coordinated together, a virtual sequence and virtual sequencer are useful.

9. When They Are Not Needed

If the testbench has only one driving agent, a virtual sequencer is usually not needed.

Example:

Test
└── Environment
    └── Agent
        ├── Sequencer
        └── Driver

Here, the sequence can be started directly on the single sequencer.

10. Multiple Agents Without Coordination

If there are multiple agents but their stimulus does not need coordination, a virtual sequencer may still not be required.

Example:

Agent 1 → Sequencer 1 → Driver 1
Agent 2 → Sequencer 2 → Driver 2

Each sequence can be started directly on its own sequencer from the test.

11. Multiple Agents With Coordination

If multiple agents must work together in a controlled order or parallel pattern, virtual sequence and virtual sequencer are useful.

Example:

Virtual Sequence
├── Starts UART sequence on UART sequencer
└── Starts Ethernet sequence on Ethernet sequencer

This allows one central sequence to control the complete multi-interface scenario.

12. Example Without Virtual Sequence and Virtual Sequencer

Assume two agents:

Agent 1 → Sequencer 1 → Driver 1
Agent 2 → Sequencer 2 → Driver 2

There are two sequences:

Sequence A → Sequencer 1
Sequence B → Sequencer 2

Without virtual sequence/sequencer, the test directly starts each sequence.

Example:

```
seq_a.start(env_h.agent1_h.seqr1_h);
seq_b.start(env_h.agent2_h.seqr2_h);
```

Here, the test controls each sequence separately.

13. Problem Without Virtual Sequence

For small tests, directly starting each sequence from the test is fine.

But for larger SoC-level verification, this becomes harder to manage.

Problems:

- Test becomes crowded.
- Multi-interface coordination becomes difficult.
- Reuse becomes harder.
- Complex timing between protocol sequences becomes messy.

A virtual sequence solves this by moving the coordination logic into one sequence.

14. Virtual Sequence Structure

A virtual sequence may contain handles to lower-level sequences.

Example:

```
class virtual_sequence extends uvm_sequence #(sequence_item);

  sequence_a seq_a_h;
  sequence_b seq_b_h;

endclass
```

It can start these sequences in a required order or in parallel.

15. Virtual Sequencer Structure

A virtual sequencer contains handles to the real sequencers.

Example:

```
class virtual_sequencer extends uvm_sequencer #(sequence_item);

  sequencer_a seqr_a_h;
  sequencer_b seqr_b_h;

endclass
```

These real sequencer handles point to the sequencers inside the actual agents.

The virtual sequencer itself is not connected to any driver.

16. Real Sequencer and Driver Connection

The real sequencers are connected to their respective drivers.

Example:

Sequencer A → Driver A
Sequencer B → Driver B

The virtual sequencer is only a handle holder.

It does not replace the real sequencers.

17. Starting a Virtual Sequence

When using a virtual sequence, the test starts only the virtual sequence.

Example:

vseq_h.start(env_h.vseqr_h);

Here:

- `vseq_h` is the virtual sequence handle.
- `env_h.vseqr_h` is the virtual sequencer handle.

This connects the virtual sequence to the virtual sequencer.

18. What Happens When Virtual Sequence Starts

When this is called:

vseq_h.start(env_h.vseqr_h);

the virtual sequencer handle is assigned internally to the virtual sequence's `m_sequencer`.

Then the virtual sequence `body()` task starts executing.

Inside the `body()` task, the virtual sequence starts lower-level sequences on real sequencers.

19. Starting Lower-Level Sequences Inside Virtual Sequence

Inside the virtual sequence body:

seq_a_h.start(p_sequencer.seqr_a_h);

seq_b_h.start(p_sequencer.seqr_b_h);

Here:

- `seq_a_h` starts on real sequencer A.
- `seq_b_h` starts on real sequencer B.
- `p_sequencer` gives access to the virtual sequencer and its internal sequencer handles.

20. Without Virtual Sequence vs With Virtual Sequence

Without Virtual Sequence

The test directly starts each sequence:

seq_a_h.start(env_h.agent1_h.seqr_a_h);
seq_b_h.start(env_h.agent2_h.seqr_b_h);

With Virtual Sequence

The test starts only the virtual sequence:

vseq_h.start(env_h.vseqr_h);

Then, inside the virtual sequence:

seq_a_h.start(p_sequencer.seqr_a_h);
seq_b_h.start(p_sequencer.seqr_b_h);

This keeps the test cleaner and moves coordination logic into the virtual sequence.

21. Role of `m_sequencer`

`m_sequencer` is a predefined internal sequencer handle.

It is inherited by sequence classes.

When a sequence is started, the sequencer passed to `start()` is assigned to `m_sequencer`.

Example:

vseq_h.start(env_h.vseqr_h);

After this call:

m_sequencer points to env_h.vseqr_h

22. Role of `p_sequencer`

`p_sequencer` is a typed sequencer handle.

Unlike `m_sequencer`, it is not automatically declared.

The user declares it using:

`uvm_declare_p_sequencer(virtual_sequencer)

This macro casts `m_sequencer` to the required virtual sequencer type.

After this, the virtual sequence can directly access handles inside the virtual sequencer.

Example:

p_sequencer.seqr_a_h
p_sequencer.seqr_b_h

23. Why `p_sequencer` Is Useful

Without `p_sequencer`, the virtual sequence may need to use full hierarchical paths or manual casting.

With `p_sequencer`, the code becomes cleaner.

Example:

seq_a_h.start(p_sequencer.seqr_a_h);
seq_b_h.start(p_sequencer.seqr_b_h);

This is easier than repeatedly writing long paths.

24. Typical Virtual Sequence Code Pattern

```
class virtual_sequence extends uvm_sequence #(sequence_item);

  `uvm_object_utils(virtual_sequence)
  `uvm_declare_p_sequencer(virtual_sequencer)

  sequence_a seq_a_h;
  sequence_b seq_b_h;

  function new(string name = "virtual_sequence");
    super.new(name);
  endfunction

  task body();

    seq_a_h = sequence_a::type_id::create("seq_a_h");
```

```
    seq_b_h = sequence_b::type_id::create("seq_b_h");

    seq_a_h.start(p_sequencer.seqr_a_h);
    seq_b_h.start(p_sequencer.seqr_b_h);

  endtask

endclass
```

25. Typical Virtual Sequencer Code Pattern

```
class virtual_sequencer extends uvm_sequencer #(sequence_item);

  `uvm_component_utils(virtual_sequencer)

  sequencer_a seqr_a_h;
  sequencer_b seqr_b_h;

  function new(string name = "virtual_sequencer", uvm_component parent = null);
    super.new(name, parent);
  endfunction

endclass
```

The actual sequencer handles are usually assigned in the environment.

26. Test-Level Flow With Virtual Sequence

Inside the test:

```
virtual_sequence vseq_h;

task run_phase(uvm_phase phase);

  phase.raise_objection(this);

  vseq_h = virtual_sequence::type_id::create("vseq_h");
  vseq_h.start(env_h.vseqr_h);

  phase.drop_objection(this);

endtask
```

The test only starts the virtual sequence.

The virtual sequence controls the lower-level sequences.

27. Conceptual Flow

Test

↓
Starts Virtual Sequence on Virtual Sequencer
↓
Virtual Sequence body() executes
↓
Virtual Sequence starts Sequence A on Sequencer A
↓
Virtual Sequence starts Sequence B on Sequencer B
↓
Sequencer A sends items to Driver A
↓
Sequencer B sends items to Driver B
↓
Drivers drive respective DUT interfaces

28. Key Points to Remember

- A virtual sequence coordinates other sequences.
- A virtual sequencer holds handles to real sequencers.
- There is no special UVM base class called `uvm_virtual_sequence`.
- There is no special UVM base class called `uvm_virtual_sequencer`.
- Virtual sequence extends `uvm_sequence`.
- Virtual sequencer extends `uvm_sequencer`.
- A virtual sequencer is not connected to any driver.
- A virtual sequencer does not process sequence items.
- Real sequencers are connected to real drivers.
- Virtual sequence is useful for multi-agent and multi-protocol coordination.
- If only one agent exists, virtual sequence/sequencer is usually not needed.
- If multiple agents do not need stimulus coordination, virtual sequence/sequencer may not be needed.
- If coordinated stimulus is required, virtual sequence/sequencer is useful.
- The test starts the virtual sequence on the virtual sequencer.
- The virtual sequence starts lower-level sequences on real sequencers.
- `m_sequencer` is the internal sequencer handle.
- `p_sequencer` is a typed sequencer handle declared using `uvm_declare_p_sequencer`.
- `p_sequencer` makes access to virtual sequencer handles easier and cleaner.

29. Final Summary

A virtual sequence and virtual sequencer are used to coordinate stimulus across multiple agents.

The virtual sequencer stores handles to real sequencers, while the virtual sequence starts and controls lower-level sequences on those real sequencers.

The virtual sequencer is not connected to any driver and does not process sequence items directly.

In the test, only the virtual sequence is started:

vseq_h.start(env_h.vseqr_h);

Inside the virtual sequence, individual sequences are started on their respective real sequencers:

```
seq_a_h.start(p_sequencer.seqr_a_h);
seq_b_h.start(p_sequencer.seqr_b_h);
```

This gives a clean and scalable way to coordinate multi-interface UVM stimulus.

UVM Callbacks in SystemVerilog

UVM Callbacks in SystemVerilog

1. Topic Overview

This section explains **callbacks in UVM**, including:

- Callback concept in SystemVerilog
- Hook methods
- Why callbacks are used
- Callback registration
- Callback class creation
- Adding and deleting callbacks
- Callback-based driver modification example

2. What Is a Callback?

SystemVerilog does not have function pointers like C.

Instead, callback behaviour is implemented using object-oriented programming.

A callback is usually a virtual method, also called a **hook**, declared in a base class.

A derived class can override this hook method and add custom behaviour.

The main component calls the hook at a specific point during execution.

3. Simple SystemVerilog Callback Example

A common example of callbacks in SystemVerilog is:

pre_randomize()
post_randomize()

When `randomize()` is called, SystemVerilog internally calls:

1. `pre_randomize()`
2. Randomization logic
3. `post_randomize()`

The user does not call `pre_randomize()` or `post_randomize()` directly.

If the user defines these methods, their code executes.

If the user does not define them, nothing happens.

4. Callback Hook Concept

A **hook** is a virtual method declared in a class.

By default, the hook may have an empty body.

If the user wants extra functionality, the hook is overridden in a derived callback class.

Example callback hooks:

pre_drive()
post_drive()
modify_packet()

These methods can be called from a driver, monitor, or other UVM component.

5. Why Callbacks Are Needed

Callbacks are used to modify or extend component behaviour without changing the original source code.

They help with:

- Component reuse
- Adding custom test behaviour
- Making drivers and monitors flexible
- Fault injection
- Delay insertion
- Logging
- Packet modification
- Allowing multiple users to hook into the same event

6. Callback Use in Reusable Components

UVM components such as drivers, monitors, and scoreboards are usually written to be generic and reusable.

If the driver source code is modified every time new behaviour is needed, reusability is lost.

Callbacks solve this problem.

They allow behaviour to be added from the test without editing the original component code.

7. Driver Callback Example Concept

Assume a driver is already written.

Later, the user may want to make the same driver behave differently, such as:

- Normal driver
- Logging driver
- Slow driver
- Fault-injection driver
- Corrupt-packet driver

Instead of writing separate drivers for each case, callbacks allow extra behaviour to be injected into the same driver.

8. Fault and Delay Injection

Callbacks are useful for corner-case testing.

Fault Injection

A callback can corrupt a transaction or signal before it is driven to the DUT.

This helps test how the DUT responds to faulty input.

Delay Injection

A callback can add delay before driving a packet.

This helps model slow response behaviour.

9. UVM Callback Mechanism

UVM callbacks are based on the hook-method pattern.

The basic flow is:

1. Declare a callback class.
2. Declare hook methods inside the callback class.
3. Derive a new callback class and override the hook method.
4. Register the component and callback type.
5. Call the hook inside the component.
6. Add the callback object from the test.

10. Callback Registration Macro

To tell UVM that a component supports a callback type, use:

`uvm_register_cb(T, CB)

Arguments

Argument	Meaning
`T`	Component type where the callback hook is used
`CB`	User-defined callback class type

Example:

`uvm_register_cb(driver, driver_cb)

This means the `driver` component supports callbacks of type `driver_cb`.

11. Why Registration Is Required

If the callback pair is not registered, UVM will not know that the component supports that callback type.

As a result, callback-related operations may produce warnings or may not work as expected.

12. Callback Base Class

A user-defined callback class must extend:

uvm_callback

Example:

class driver_cb extends uvm_callback;

Since callback classes are UVM objects, they are registered using:

`uvm_object_utils(driver_cb)

13. Declaring a Callback Hook

Inside the callback class, declare a virtual hook method.

Example:

```
virtual task modify_packet();
endtask
```

This method has no body by default.

It acts as a placeholder that can be overridden later.

14. Derived Callback Class

A derived callback class extends the base callback class.

Example:

class derived_cb extends driver_cb;

This class overrides the hook method and adds the required functionality.

Example:

```
task modify_packet();
  `uvm_info(get_type_name(), "Injecting extra packet", UVM_LOW)
endtask
```

15. Callback Class Structure

Basic callback class:

```
class driver_cb extends uvm_callback;

  `uvm_object_utils(driver_cb)

  function new(string name = "driver_cb");
    super.new(name);
  endfunction

  virtual task modify_packet();
  endtask

endclass
```

Derived callback class:

```
class derived_cb extends driver_cb;

  `uvm_object_utils(derived_cb)

  function new(string name = "derived_cb");
    super.new(name);
  endfunction

  task modify_packet();
    `uvm_info(
      get_type_name(),
      "Inside modify_packet method: injecting extra packet",
      UVM_LOW
    )
  endtask

endclass
```

16. Enum Example for Packet Type

The example uses an enum packet type.

```
typedef enum {
  DATA_1,
  DATA_2,
  EXTRA_DATA_1,
  EXTRA_DATA_2
} packet_type;
```

A variable is declared:

packet_type packet;

The derived callback can randomize this packet type and apply constraints.

Example concept:

```
std::randomize(packet) with {
  packet inside {EXTRA_DATA_1, EXTRA_DATA_2};
};
```

This shows how a callback can inject or modify packet behaviour.

17. Driver Class

The driver is the component where the callback hook is used.

It extends:

uvm_component

Example:

class driver extends uvm_component;

It is registered using:

`uvm_component_utils(driver)

The callback support is registered using:

`uvm_register_cb(driver, driver_cb)

18. Calling the Callback Hook in Driver

Inside the driver, the callback hook is called using:

`uvm_do_callbacks(driver, driver_cb, modify_packet())

This tells UVM:

- The component type is `driver`.
- The callback type is `driver_cb`.
- The hook method is `modify_packet()`.

If a callback object is added from the test, the callback method runs.

If no callback is added, nothing extra happens.

19. Driver `run_phase`

The driver can call its normal driving logic and then invoke the callback hook.

Example:

```
task run_phase(uvm_phase phase);

  drive();

  `uvm_do_callbacks(driver, driver_cb, modify_packet())

endtask
```

This allows extra callback behaviour to be inserted after normal driver execution.

20. Driver `drive()` Task

The driver has a normal `drive()` task.

Example:

```
task drive();

  `uvm_info(
    get_full_name(),
    "Inside driver class method",
    UVM_LOW
  )

  std::randomize(packet) with {
    packet == DATA_1;
  };

endtask
```

This represents the default driver behaviour.

21. Environment Class

The environment contains the driver.

```
class environment extends uvm_env;

  `uvm_component_utils(environment)

  driver drv_h;
```

endclass

Inside `build_phase`, the driver is created:

drv_h = driver::type_id::create("drv_h", this);

22. Base Test

The base test creates the environment.

class base_test extends uvm_test;

  `uvm_component_utils(base_test)

  environment env_h;

endclass

Inside `build_phase`:

env_h = environment::type_id::create("env_h", this);

If only the base test is run, only the normal driver method executes.

Expected output:

Inside driver class method

23. Test with Callback

A second test is created by extending the base test.

class test_2 extends base_test;

This test creates a derived callback object.

derived_cb dcb;

Inside `build_phase`:

dcb = derived_cb::type_id::create("dcb");

Then the callback is added to the driver.

24. Adding a Callback

A callback is added using:

uvm_callbacks#(driver, driver_cb)::add(env_h.drv_h, dcb);

Meaning

- `driver` is the component type.
- `driver_cb` is the callback type.
- `env_h.drv_h` is the driver instance where the callback is added.
- `dcb` is the derived callback object that provides the new behaviour.

After this, when the driver calls the callback hook, `derived_cb.modify_packet()` executes.

25. Output with Callback

When `test_2` is run, the output shows:

Inside driver class method
Inside modify_packet method: injecting extra packet

This confirms that the callback added extra functionality without modifying the driver source code.

26. Deleting a Callback

A callback can also be removed using the delete method.

Example concept:

uvm_callbacks#(driver, driver_cb)::delete(env_h.drv_h, dcb);

After deletion, the callback method will no longer execute for that driver instance.

27. Important Callback Macros and Methods

Macro / Method	Purpose
`uvm_register_cb(T, CB)`	Registers a component type with a callback type
`uvm_do_callbacks(T, CB, METHOD)`	Calls the callback hook method
`uvm_callbacks#(T, CB)::add()`	Adds a callback object to a component

`uvm_callbacks#(T, CB)::delete()`	Removes a callback object from a component

28. Complete Callback Flow

The complete callback flow is:

Create callback base class
↓
Declare virtual hook method
↓
Create derived callback class
↓
Override hook method
↓
Register component-callback pair
↓
Call hook inside component
↓
Create callback object in test
↓
Add callback object to component
↓
Callback code executes during component run

29. Key Points to Remember

- Callbacks are virtual hook methods used to extend behaviour.
- SystemVerilog callbacks are implemented using OOP concepts.
- `pre_randomize()` and `post_randomize()` are common callback examples.
- UVM callbacks allow driver, monitor, or scoreboard behaviour to be changed without modifying source code.
- Callback classes extend `uvm_callback`.
- Callback classes are UVM objects and use `uvm_object_utils`.
- The component using callbacks must register the callback type using `uvm_register_cb`.
- The component calls the hook using `uvm_do_callbacks`.
- The test adds callback behaviour using `uvm_callbacks#(...)::add()`.
- A callback can be removed using `delete()`.
- Callbacks are useful for logging, delay insertion, packet modification, fault injection, and error injection.
- If no callback object is added, the hook call does nothing.
- If a callback object is added, the overridden callback method executes.

30. Final Summary

UVM callbacks provide a clean way to modify component behaviour without editing the original component code.

A driver can include an empty callback hook such as `modify_packet()`.

A derived callback class can override this hook and add extra behaviour, such as injecting packets, adding delay, or corrupting transactions.

The test adds the callback object to the driver using:

uvm_callbacks#(driver, driver_cb)::add(env_h.drv_h, dcb);

When the driver reaches the callback hook, the added callback method executes.

This makes UVM components more reusable, flexible, and test-specific without duplicating or modifying the original driver code.

UVM Driver and Monitor for D Flip-Flop Testbench

UVM Driver and Monitor for D Flip-Flop Testbench

1. Topic Overview

This section continues the UVM testbench development for a **D Flip-Flop** design.

The previous part covered:

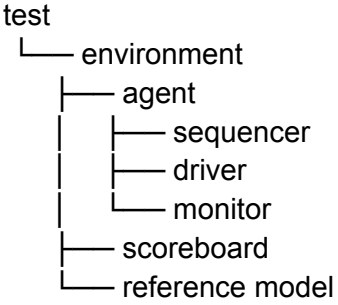
- Sequence item
- Sequence
- Sequencer
- Basic testbench architecture

This part covers:

- UVM driver
- UVM monitor
- Use of virtual interface
- Use of Config DB
- Driver-to-DUT signal driving
- Monitor observation and analysis port transfer

2. UVM Testbench Context

The planned UVM testbench contains:



The driver and monitor communicate with the DUT through a virtual interface.

The monitor sends observed transactions to the reference model and scoreboard using an analysis port.

3. UVM Driver

The driver is responsible for receiving transactions from the sequencer and driving the DUT input through the virtual interface.

For the D Flip-Flop, the driver drives the input signal:

d

4. Driver Class Declaration

The driver class extends `uvm_driver`.

It is parameterized with the transaction type.

class dff_driver extends uvm_driver #(dff_txn);

The driver is a UVM component, so it is registered using:

`uvm_component_utils(dff_driver)

5. Virtual Interface in Driver

The driver needs access to the DUT interface.

So, a virtual interface handle is declared inside the driver:

virtual dff_inf vif;

Purpose

The virtual interface allows the driver to assign transaction values to DUT signals.

Example:

vif.d = txn.d;

6. Driver Constructor

The driver constructor follows the standard UVM component format.

```
function new(string name = "dff_driver", uvm_component parent = null);
  super.new(name, parent);
endfunction
```

Since the driver is a component, it has both:

- `name`
- `parent`

7. Driver `build_phase`

The driver receives the virtual interface from the UVM Config DB inside the `build_phase`.

```
function void build_phase(uvm_phase phase);
  super.build_phase(phase);

  if (!uvm_config_db#(virtual dff_inf)::get(this, "", "vif", vif)) begin
    `uvm_fatal(get_type_name(), "Virtual interface is not received")
  end
endfunction
```

8. Purpose of Config DB in Driver

The Config DB is used to pass the virtual interface from the top-level module to the driver.

The driver calls:

uvm_config_db#(virtual dff_inf)::get(...)

If the interface is not received, the driver reports a fatal error.

This prevents the simulation from continuing without a valid DUT connection.

9. Driver `run_phase`

The main driver activity happens inside the `run_phase`.

The driver repeatedly:

1. Gets a transaction from the sequencer.
2. Drives the transaction value onto the interface.
3. Calls `item_done()` to complete the sequencer-driver handshake.
4. Prints the driven data.

10. Driver Transaction Flow

Inside the `run_phase`, a transaction handle is declared:

dff_txn txn;

The driver receives a transaction using:

seq_item_port.get_next_item(txn);

This is a built-in TLM port available in `uvm_driver`.

The user does not need to declare it manually.

11. Driving Data to DUT

After receiving the transaction, the driver copies the transaction value to the interface:

vif.d = txn.d;

This drives the D Flip-Flop input.

12. Completing the Driver-Sequencer Handshake

After driving the transaction, the driver calls:

seq_item_port.item_done();

This tells the sequencer that the current transaction has been processed.

Without `item_done()`, the next transaction may not be issued correctly.

13. Driver Debug Message

The driver prints the driven value using `uvm_info`.

Example:

```
`uvm_info(
  get_type_name(),
  $sformatf("Driver driven data: d = %0d", txn.d),
  UVM_LOW
)
```

This helps verify that the driver is receiving and driving transactions correctly.

14. Complete Driver Operation Summary

The driver performs the following operations:

Sequencer transaction
↓
Driver receives transaction using get_next_item()
↓
Driver drives txn.d to vif.d
↓
Driver calls item_done()
↓
Driver prints driven value

15. UVM Monitor

The monitor observes DUT interface signals and converts them back into transaction-level data.

For the D Flip-Flop, the monitor observes:

- Input `d`
- Output `q`

The observed values are stored in a transaction object.

16. Monitor Class Declaration

The monitor extends `uvm_monitor`.

class dff_monitor extends uvm_monitor;

It is registered using:

`uvm_component_utils(dff_monitor)

17. Virtual Interface in Monitor

The monitor also needs access to the DUT interface.

So, it declares a virtual interface handle:

virtual dff_inf vif;

The monitor uses this handle to sample DUT input and output signals.

18. Monitor Constructor

The monitor constructor follows the standard UVM component format.

```
function new(string name = "dff_monitor", uvm_component parent = null);
  super.new(name, parent);
endfunction
```

19. Monitor `build_phase`

The monitor retrieves the virtual interface from Config DB.

```
function void build_phase(uvm_phase phase);
  super.build_phase(phase);

  if (!uvm_config_db#(virtual dff_inf)::get(this, "", "vif", vif)) begin
    `uvm_fatal(get_type_name(), "Virtual interface is not received")
  end
```

endfunction

If the interface is not received, a fatal message is printed.

20. Monitor `run_phase`

The monitor continuously observes DUT signals during simulation.

It samples values at the positive edge of the clock.

@(posedge vif.clk);

At every clock edge, the monitor creates a transaction and stores the observed signal values.

21. Creating Transaction in Monitor

Inside the monitor `run_phase`, the transaction object is created using the factory:

txn = dff_txn::type_id::create("txn", this);

This transaction stores the sampled values.

22. Sampling Interface Signals

The monitor copies interface values into the transaction object.

```
txn.d        = vif.d;
txn.q_expected = vif.q;
```

Here:

- `txn.d` stores the observed input.
- `txn.q_expected` stores the observed DUT output.

In the transcript, the DUT output `q` is stored into the transaction field named `q_expected`.

23. Monitor Analysis Port

The monitor sends the observed transaction using an analysis port.

monitor_analysis_port.write(txn);

This transaction can be received by:

- Reference model
- Scoreboard

The analysis port allows the monitor to broadcast observed data to other verification components.

24. Monitor Debug Message

The monitor prints the observed input and DUT output.

Example:

```
`uvm_info(
  get_type_name(),
  $sformatf(
    "Monitor observed data: d = %0d, q = %0d",
    txn.d,
    txn.q_expected
  ),
  UVM_LOW
)
```

This helps confirm that the monitor is sampling the DUT signals correctly.

25. Complete Monitor Operation Summary

The monitor performs the following operations:

Wait for positive clock edge
↓
Create transaction object
↓
Sample vif.d and vif.q
↓
Store sampled values in transaction
↓
Send transaction through analysis port
↓
Print observed values

26. Driver vs Monitor

Component	Main Role	Direction
Driver	Drives transaction data to DUT	Testbench → DUT
Monitor	Observes DUT interface signals	DUT/interface → Testbench
Driver uses	`seq_item_port`	Gets items from sequencer
Monitor uses	Analysis port	Sends observed transactions

Driver interface use	Drives `vif.d`	Input driving
Monitor interface use	Samples `vif.d` and `vif.q`	Signal observation

27. Important Methods Used

Method	Used In	Purpose
`uvm_config_db::get()`	Driver, Monitor	Retrieves virtual interface
`get_next_item()`	Driver	Gets transaction from sequencer
`item_done()`	Driver	Completes driver-sequencer handshake
`write()`	Monitor	Sends transaction through analysis port
`@(posedge vif.clk)`	Monitor	Samples interface on clock edge

28. Key Points to Remember

- The driver extends `uvm_driver #(dff_txn)`.
- The monitor extends `uvm_monitor`.
- Both driver and monitor require a virtual interface.
- The virtual interface is retrieved using UVM Config DB.
- If the virtual interface is not received, a fatal error is reported.
- The driver receives transactions from the sequencer using `seq_item_port.get_next_item()`.
- The driver drives `txn.d` onto `vif.d`.
- The driver calls `seq_item_port.item_done()` after driving.
- The monitor samples DUT signals at the positive edge of `vif.clk`.
- The monitor creates a transaction and stores observed values.
- The monitor sends the transaction using an analysis port.
- The observed transaction is intended for the reference model and scoreboard.
- `uvm_info` messages are used for debugging driver and monitor activity.

29. Final Summary

The UVM driver and monitor form the active DUT interface layer of the D Flip-Flop testbench.

The driver receives randomized transactions from the sequencer and drives the DUT input through the virtual interface.

The monitor observes the DUT interface at each positive clock edge, stores the sampled values in a transaction, and sends the transaction to other components using an analysis port.

Together, the driver and monitor allow the UVM testbench to both stimulate and observe the D Flip-Flop DUT.

UVM Agent for D Flip-Flop Testbench

UVM Agent for D Flip-Flop Testbench

1. Topic Overview

This section explains how to build the **UVM agent** for the D Flip-Flop testbench.

The agent contains and connects:

- Driver
- Monitor
- Sequencer
- Virtual interface handle

The agent is responsible for creating these components and connecting the driver with the sequencer.

2. Role of UVM Agent

A UVM agent groups together protocol-related verification components.

For this D Flip-Flop testbench, the agent contains:

dff_agent
 ├── dff_driver
 ├── dff_monitor
 └── dff_sequencer

The agent performs two main tasks:

1. Creates the driver, monitor, and sequencer.
2. Connects the driver and sequencer using TLM ports.

3. Agent Class Declaration

The agent class extends:

uvm_agent

Example:

class dff_agent extends uvm_agent;

Since the agent is a UVM component, it is registered using:

`uvm_component_utils(dff_agent)

4. Agent Constructor

The constructor follows the standard UVM component format.

```
function new(string name = "dff_agent", uvm_component parent = null);
  super.new(name, parent);
endfunction
```

Since the agent is a component, it has both:

- `name`
- `parent`

5. Handles Declared Inside Agent

The agent declares handles for:

```
dff_driver    drv_h;
dff_monitor   mon_h;
dff_sequencer seqr_h;
```

It also declares a virtual interface handle:

```
virtual dff_inf vif;
```

Purpose

- `drv_h` points to the driver.
- `mon_h` points to the monitor.
- `seqr_h` points to the sequencer.
- `vif` stores the virtual interface handle used by the driver and monitor.

6. Agent `build_phase`

The agent uses the `build_phase` to:

1. Get the virtual interface from Config DB.
2. Create the driver.
3. Create the monitor.
4. Create the sequencer.

7. Getting Virtual Interface in Agent

The virtual interface is retrieved using:

```
uvm_config_db#(virtual dff_inf)::get(this, "", "vif", vif)
```

If the agent does not receive the virtual interface, it prints a fatal error.

Example:

```
if (!uvm_config_db#(virtual dff_inf)::get(this, "", "vif", vif)) begin
  `uvm_fatal(get_type_name(), "Virtual interface is not received")
end
```

8. Creating Driver, Monitor, and Sequencer

Inside the `build_phase`, the agent creates all three components using factory creation.

```
drv_h = dff_driver::type_id::create("drv_h", this);
mon_h = dff_monitor::type_id::create("mon_h", this);
seqr_h = dff_sequencer::type_id::create("seqr_h", this);
```

Key Point

Since these are UVM components, they are created using:

type_id::create("instance_name", this)

The `this` argument sets the agent as the parent component.

9. Agent `connect_phase`

The `connect_phase` is used to connect the driver and sequencer.

The driver has a built-in TLM port:

seq_item_port

The sequencer has a built-in TLM export:

seq_item_export

They are connected as:

drv_h.seq_item_port.connect(seqr_h.seq_item_export);

This connection allows the driver to receive transactions from the sequencer.

10. Passing Virtual Interface to Driver and Monitor

After connecting the driver and sequencer, the agent passes the virtual interface handle to the driver and monitor.

```
drv_h.vif = vif;
mon_h.vif = vif;
```

Meaning

- The driver uses `vif` to drive DUT input signals.
- The monitor uses `vif` to observe DUT input and output signals.

11. Complete Agent Flow

The agent performs the following flow:

```
Get virtual interface from Config DB
        ↓
Create driver
        ↓
Create monitor
        ↓
Create sequencer
        ↓
Connect driver seq_item_port to sequencer seq_item_export
        ↓
Pass virtual interface to driver and monitor
```

12. Complete Agent Code Structure

```
class dff_agent extends uvm_agent;

  `uvm_component_utils(dff_agent)

  dff_driver    drv_h;
  dff_monitor   mon_h;
  dff_sequencer seqr_h;

  virtual dff_inf vif;

  function new(string name = "dff_agent", uvm_component parent = null);
    super.new(name, parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);

    if (!uvm_config_db#(virtual dff_inf)::get(this, "", "vif", vif)) begin
      `uvm_fatal(get_type_name(), "Virtual interface is not received")
    end

    drv_h  = dff_driver::type_id::create("drv_h", this);
    mon_h  = dff_monitor::type_id::create("mon_h", this);
    seqr_h = dff_sequencer::type_id::create("seqr_h", this);
```

```
  endfunction

  function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);

    drv_h.seq_item_port.connect(seqr_h.seq_item_export);

    drv_h.vif = vif;
    mon_h.vif = vif;
  endfunction

endclass
```

13. Important Notes

- The agent extends `uvm_agent`.
- The agent is registered using `uvm_component_utils`.
- The agent contains driver, monitor, and sequencer handles.
- The agent also contains a virtual interface handle.
- The virtual interface is retrieved using Config DB.
- Driver, monitor, and sequencer are created in the `build_phase`.
- Driver and sequencer are connected in the `connect_phase`.
- The driver receives transactions from the sequencer through TLM connection.
- The virtual interface is passed to both driver and monitor.
- The driver uses the interface to drive DUT signals.
- The monitor uses the interface to observe DUT signals.

14. Final Summary

The UVM agent is the block that groups the D Flip-Flop driver, monitor, and sequencer.

In the `build_phase`, it creates these components and retrieves the virtual interface.

In the `connect_phase`, it connects the driver to the sequencer using:

drv_h.seq_item_port.connect(seqr_h.seq_item_export);

It also passes the virtual interface to the driver and monitor.

With this, the D Flip-Flop UVM agent is ready. The next block to develop is the scoreboard.

UVM Scoreboard for D Flip-Flop Testbench

UVM Scoreboard for D Flip-Flop Testbench

1. Topic Overview

This section explains how to develop the **UVM scoreboard** for a D Flip-Flop testbench.

The previous parts covered:

- Driver
- Monitor
- Agent

This part covers:

- Scoreboard class creation
- Analysis `write()` method
- Pass/fail comparison logic
- Transaction counting
- Report phase summary

2. Role of Scoreboard

The scoreboard checks whether the DUT output is correct.

For a D Flip-Flop:

Expected behaviour:
q = d

So, the scoreboard compares:

- Input value `d`
- Observed or expected output value `q_expected`

If both match, the transaction passes.

If they do not match, the transaction fails.

3. Scoreboard Class Declaration

The scoreboard class extends:

uvm_scoreboard

Example:

class dff_scoreboard extends uvm_scoreboard;

Since the scoreboard is a UVM component, it is registered using:

`uvm_component_utils(dff_scoreboard)

4. Scoreboard Constructor

The constructor follows the standard UVM component format.

```
function new(string name = "dff_scoreboard", uvm_component parent = null);
  super.new(name, parent);
endfunction
```

Since the scoreboard is a component, it has:

- `name`
- `parent`

5. Variables Declared in Scoreboard

The scoreboard declares counters to track verification results.

```
int total_transactions;
int pass_count;
int fail_count;
```

Purpose

Variable	Meaning
`total_transactions`	Counts total checked transactions
`pass_count`	Counts passed transactions
`fail_count`	Counts failed transactions

6. Scoreboard `write()` Method

The `write()` method receives transactions from the monitor through an analysis port.

The monitor calls:

```
analysis_port.write(txn);
```

The scoreboard implements:

```
function void write(dff_txn txn);
```

Purpose

The `write()` method is used to:

1. Receive the transaction.
2. Increment the transaction count.
3. Compare input and output values.
4. Update pass/fail counters.
5. Print pass or fail messages.

7. Transaction Count

Whenever the scoreboard receives a transaction, it increments the total count.

total_transactions++;

This tracks how many transactions were checked.

8. Comparison Logic

For a D Flip-Flop, the output should match the input.

So, the scoreboard compares:

txn.q_expected == txn.d

If Match

If `q_expected` matches `d`:

pass_count++;

A pass message is printed.

If Mismatch

If `q_expected` does not match `d`:

fail_count++;

An error message is printed.

9. Pass Message

When the transaction passes, the scoreboard prints a UVM info message.

Example:

```
`uvm_info(
  get_type_name(),
  $sformatf(
    "PASS: d = %0b, q = %0b. Match.",
    txn.d,
    txn.q_expected
  ),
  UVM_LOW
)
```

This confirms that the DUT output matches the expected D Flip-Flop behaviour.

10. Fail Message

When the transaction fails, the scoreboard prints a UVM error message.

Example:

```
`uvm_error(
  get_type_name(),
  $sformatf(
    "FAIL: d = %0b, q = %0b. Mismatch.",
    txn.d,
    txn.q_expected
  )
)
```

This indicates that the DUT output does not match the expected value.

11. Scoreboard `report_phase`

The `report_phase` is used to print the final scoreboard summary.

It reports:

- Total transactions
- Pass count
- Fail count

Example:

```
function void report_phase(uvm_phase phase);
  super.report_phase(phase);

  `uvm_info(
    get_type_name(),
    $sformatf(
      "Scoreboard Summary: Total = %0d, Passed = %0d, Failed = %0d",
      total_transactions,
      pass_count,
```

```
      fail_count
    ),
    UVM_LOW
  )
endfunction
```

12. Complete Scoreboard Structure

```
class dff_scoreboard extends uvm_scoreboard;

  `uvm_component_utils(dff_scoreboard)

  int total_transactions;
  int pass_count;
  int fail_count;

  function new(string name = "dff_scoreboard", uvm_component parent = null);
    super.new(name, parent);
  endfunction

  function void write(dff_txn txn);

    total_transactions++;

    if (txn.q_expected == txn.d) begin
      pass_count++;

      `uvm_info(
        get_type_name(),
        $sformatf(
          "PASS: d = %0b, q = %0b. Match.",
          txn.d,
          txn.q_expected
        ),
        UVM_LOW
      )
    end
    else begin
      fail_count++;

      `uvm_error(
        get_type_name(),
        $sformatf(
          "FAIL: d = %0b, q = %0b. Mismatch.",
          txn.d,
          txn.q_expected
        )
      )
    end

  endfunction
```

```
  function void report_phase(uvm_phase phase);
    super.report_phase(phase);

    `uvm_info(
      get_type_name(),
      $sformatf(
        "Scoreboard Summary: Total = %0d, Passed = %0d, Failed = %0d",
        total_transactions,
        pass_count,
        fail_count
      ),
      UVM_LOW
    )
  endfunction

endclass
```

13. Complete Scoreboard Flow

Monitor sends transaction using write()
↓
Scoreboard receives transaction
↓
Total transaction count increments
↓
Scoreboard compares d and q_expected
↓
If equal: pass_count increments
↓
If not equal: fail_count increments
↓
Final summary is printed in report_phase

14. Key Points to Remember

- The scoreboard extends `uvm_scoreboard`.
- It is registered using `uvm_component_utils`.
- The scoreboard receives transactions through the analysis `write()` method.
- The monitor calls `write()`, and the scoreboard implements it.
- For a D Flip-Flop, the expected output should match the input `d`.
- The scoreboard compares `txn.q_expected` with `txn.d`.
- If they match, the transaction passes.
- If they do not match, the transaction fails.
- `total_transactions`, `pass_count`, and `fail_count` are used to track results.
- The `report_phase` prints the final verification summary.

15. Final Summary

The D Flip-Flop scoreboard checks whether the DUT output matches the expected input value.

Each transaction received from the monitor is checked using:

txn.q_expected == txn.d

If the values match, the scoreboard prints a pass message.

If they do not match, it prints an error message.

At the end of simulation, the `report_phase` prints the total number of transactions, passed transactions, and failed transactions.

UVM Environment, Test, Package, &Top for D FF TB

UVM Environment, Test, Package, and Top Module for D Flip-Flop Testbench

1. Topic Overview

This section completes the UVM testbench development for a **D Flip-Flop** design.

The previous parts covered:

- Sequence item
- Sequence
- Sequencer
- Driver
- Monitor
- Agent
- Scoreboard

This part covers:

- Environment
- Test class
- Package file
- Top module
- DUT instantiation
- Interface connection
- Config DB setup
- Running the complete UVM testbench

2. UVM Environment

The environment is the top-level container for major verification components.

For the D Flip-Flop testbench, the environment contains:

- Agent
- Scoreboard

The environment creates these components and connects the monitor to the scoreboard.

3. Environment Class Declaration

The environment class extends:

uvm_env

Example:

class dff_environment extends uvm_env;

Since the environment is a UVM component, it is registered using:

`uvm_component_utils(dff_environment)

4. Environment Constructor

The constructor follows the standard UVM component format.

```
function new(string name = "dff_environment", uvm_component parent = null);
  super.new(name, parent);
endfunction
```

Since the environment is a component, it has:

- `name`
- `parent`

5. Handles Declared in Environment

The environment declares handles for:

```
dff_agent      agent_h;
dff_scoreboard sb_h;
```

Purpose

Handle	Purpose
`agent_h`	Points to the DFF agent
`sb_h`	Points to the DFF scoreboard

6. Environment `build_phase`

The environment creates the agent and scoreboard inside the `build_phase`.

```
function void build_phase(uvm_phase phase);
  super.build_phase(phase);

  agent_h = dff_agent::type_id::create("agent_h", this);
  sb_h    = dff_scoreboard::type_id::create("sb_h", this);
endfunction
```

Key Point

Both agent and scoreboard are UVM components, so they are created using:

type_id::create("instance_name", this)

7. Environment `connect_phase`

The `connect_phase` connects the monitor to the scoreboard using the analysis port.

```
function void connect_phase(uvm_phase phase);
  super.connect_phase(phase);

  agent_h.mon_h.monitor_analysis_port.connect(sb_h.sb_imp);
endfunction
```

Meaning

- The monitor sends observed transactions through its analysis port.
- The scoreboard receives those transactions through its implementation port.
- This connection enables checking of DUT behaviour.

8. Environment Flow

The environment performs the following flow:

Create agent
↓
Create scoreboard
↓
Connect monitor analysis port to scoreboard implementation port

9. Complete Environment Structure

```
class dff_environment extends uvm_env;

  `uvm_component_utils(dff_environment)

  dff_agent      agent_h;
  dff_scoreboard sb_h;

  function new(string name = "dff_environment", uvm_component parent = null);
    super.new(name, parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
```

```
    agent_h = dff_agent::type_id::create("agent_h", this);
    sb_h    = dff_scoreboard::type_id::create("sb_h", this);
  endfunction

  function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);

    agent_h.mon_h.monitor_analysis_port.connect(sb_h.sb_imp);
  endfunction

endclass
```

10. UVM Test Class

The test class controls the overall test execution.

It is responsible for:

- Creating the environment
- Creating the sequence
- Starting the sequence on the sequencer
- Raising and dropping objections during `run_phase`

11. Test Class Declaration

The test class extends:

uvm_test

Example:

class dff_test extends uvm_test;

Since the test is a UVM component, it is registered using:

`uvm_component_utils(dff_test)

12. Handles Declared in Test

The test declares handles for:

```
dff_environment env_h;
dff_sequence    seq_h;
```

Purpose

Handle	Purpose

`env_h`	Points to the environment
`seq_h`	Points to the DFF sequence

13. Test Constructor

The constructor follows the standard UVM component format.

```
function new(string name = "dff_test", uvm_component parent = null);
  super.new(name, parent);
endfunction
```

14. Test `build_phase`

The environment is created inside the test `build_phase`.

```
function void build_phase(uvm_phase phase);
  super.build_phase(phase);

  env_h = dff_environment::type_id::create("env_h", this);
endfunction
```

15. Test `run_phase`

The `run_phase` starts the sequence on the sequencer.

```
task run_phase(uvm_phase phase);

  phase.raise_objection(this);

  seq_h = dff_sequence::type_id::create("seq_h");
  seq_h.start(env_h.agent_h.seqr_h);

  #10;

  phase.drop_objection(this);

endtask
```

Key Points

- `raise_objection()` keeps the simulation active.
- The sequence object is created using factory creation.
- The sequence is started on the agent's sequencer.
- `drop_objection()` allows the simulation to finish.

16. Starting the Sequence

The sequence is started using:

seq_h.start(env_h.agent_h.seqr_h);

Meaning

- `seq_h` is the DFF sequence.
- `env_h.agent_h.seqr_h` is the sequencer inside the agent.
- The sequence generates transactions and sends them to the driver through this sequencer.

17. Complete Test Structure

```
class dff_test extends uvm_test;

  `uvm_component_utils(dff_test)

  dff_environment env_h;
  dff_sequence    seq_h;

  function new(string name = "dff_test", uvm_component parent = null);
    super.new(name, parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);

    env_h = dff_environment::type_id::create("env_h", this);
  endfunction

  task run_phase(uvm_phase phase);

    phase.raise_objection(this);

    seq_h = dff_sequence::type_id::create("seq_h");
    seq_h.start(env_h.agent_h.seqr_h);

    #10;

    phase.drop_objection(this);

  endtask

endclass
```

18. Package File

A package file is used to collect and include all UVM testbench files.

The package includes files such as:

- Transaction
- Sequence
- Sequencer
- Driver
- Monitor
- Agent
- Scoreboard
- Environment
- Test

Example structure:

```
package dff_pkg;

  import uvm_pkg::*;
  `include "uvm_macros.svh"

  `include "dff_txn.sv"
  `include "dff_sequence.sv"
  `include "dff_sequencer.sv"
  `include "dff_driver.sv"
  `include "dff_monitor.sv"
  `include "dff_agent.sv"
  `include "dff_scoreboard.sv"
  `include "dff_environment.sv"
  `include "dff_test.sv"

endpackage
```

19. Purpose of Package

The package keeps all UVM class files organized.

Instead of including every file separately in the top module, the top module can import the package.

Example:

```
import dff_pkg::*;
```

20. DUT Design File

The D Flip-Flop design is written separately.

It contains the actual RTL design to be verified.

The DUT is instantiated in the top module and connected to the interface.

21. Interface File

The interface contains DUT signals such as:

```
logic clk;
logic d;
logic q;
```

The interface connects the DUT with the UVM testbench.

The driver uses the interface to drive `d`.

The monitor uses the interface to observe `d` and `q`.

22. Top Module

The top module connects everything together.

It is responsible for:

- Creating the interface instance
- Instantiating the DUT
- Generating the clock
- Setting the virtual interface in Config DB
- Calling `run_test()`

23. Top Module Structure

Example:

```
module tb_top;

  import uvm_pkg::*;
  import dff_pkg::*;

  dff_inf intf();

  dff dut (
    .clk(intf.clk),
    .d(intf.d),
    .q(intf.q)
  );

  initial begin
    intf.clk = 0;
    forever #5 intf.clk = ~intf.clk;
  end

  initial begin
    uvm_config_db#(virtual dff_inf)::set(
      null,
      "*",
      "vif",
```

```
    intf
  );

  run_test("dff_test");
 end

endmodule
```

24. Config DB Setup in Top Module

The top module sends the interface handle to the UVM testbench using Config DB.

```
uvm_config_db#(virtual dff_inf)::set(
 null,
 "*",
 "vif",
 intf
);
```

Meaning

Argument	Meaning
`null`	Used because top module is not a UVM component
`"*"`	Allows all matching UVM components to access the interface
`"vif"`	Key name used to retrieve the interface
`intf`	Actual interface instance

The agent, driver, and monitor retrieve this interface using the same key:

"vif"

25. Running the Test

The UVM test is started using:

run_test("dff_test");

This starts the UVM phase mechanism and runs the complete testbench.

26. Simulation Flow

The complete simulation flow is:

Top module creates interface

↓
Top module instantiates DUT
↓
Top module sets virtual interface in Config DB
↓
run_test("dff_test") starts UVM
↓
Test creates environment
↓
Environment creates agent and scoreboard
↓
Agent creates driver, monitor, and sequencer
↓
Environment connects monitor to scoreboard
↓
Test starts sequence on sequencer
↓
Driver drives randomized D values to DUT
↓
Monitor observes D and Q
↓
Scoreboard compares D and Q
↓
Pass/fail results are printed

27. Expected Simulation Output

During simulation, messages are printed from:

- Sequence
- Driver
- Monitor
- Scoreboard

Typical messages include:

- Generated transaction
- Driver driven data
- Monitor observed data
- Scoreboard pass/fail result

If the D Flip-Flop output matches the expected value, the scoreboard prints a pass message.

Example meaning:

PASS: D and Q matched.

28. Compile and Simulation Command Example

The transcript shows compilation and simulation using commands similar to:

```
vlog tb_top.sv
vsim tb_top
run -all
```

These commands compile the top module, start the simulation, and run the testbench.

29. Verification Result

The console output shows that:

- The driver drives input `d`.
- The monitor observes `d` and `q`.
- The scoreboard compares the values.
- Matching values produce pass messages.

This confirms that the UVM testbench verifies the correctness of the D Flip-Flop design.

30. Key Points to Remember

- The environment extends `uvm_env`.
- The environment creates the agent and scoreboard.
- The environment connects the monitor analysis port to the scoreboard implementation port.
- The test extends `uvm_test`.
- The test creates the environment.
- The test creates and starts the sequence.
- The sequence is started on the agent sequencer.
- The package includes all UVM class files.
- The top module instantiates the DUT and interface.
- The top module sets the virtual interface using Config DB.
- `run_test("dff_test")` starts the UVM test.
- The driver drives randomized input values.
- The monitor observes DUT signals.
- The scoreboard checks whether the DUT behaviour is correct.
- Console output shows generated, driven, monitored, and checked transactions.

31. Final Summary

This section completes the D Flip-Flop UVM testbench.

The environment creates and connects the agent and scoreboard.

The test creates the environment and starts the DFF sequence on the sequencer.

The top module connects the DUT and interface, passes the virtual interface through Config DB, generates the clock, and starts the UVM test.

The complete UVM testbench verifies the D Flip-Flop by driving randomized input values, observing the output, and checking correctness using the scoreboard.

UVM RAL Model

Introduction to UVM RAL Model

1. Topic Overview

This section introduces the **UVM RAL model**, also called the **Register Abstraction Layer model**.

RAL is used to model, access, configure, and track DUT registers inside a UVM testbench.

2. Why Registers Are Important

Most IP blocks contain internal registers.

These registers are used to:

- Configure the IP
- Control IP behaviour
- Store status information
- Enable or disable features
- Select operating modes

The IP behaviour depends on the values programmed into these registers.

3. Simple IP Example: Adder/Subtractor

Consider a simple IP that can work as either:

- Adder
- Subtractor

The IP contains a **mode register**.

Example behaviour:

Mode Register Value	IP Operation
`0`	Adder
`1`	Subtractor

So, by programming the mode register, the IP operation is configured.

4. Width Register Example

The same IP may also support different data widths.

Example:

- 1-bit operation

- 4-bit operation
- 8-bit operation
- Up to 64-bit operation

A **width register** can be used to configure the data width.

Example:

Width Register Value	Meaning
`000000`	Smaller width, such as 1-bit
`111111`	Maximum width, such as 64-bit

So, the IP can be configured using internal register values.

5. System-Level View

From a system-level perspective, the internal design details of the IP are not always important.

The important information is:

- Which registers exist
- What each register controls
- What values must be written
- What each bit field means
- What address is used to access each register

These details are usually provided in the IP specification.

6. UART IP Example

A more practical example is a UART IP.

A UART IP may contain blocks such as:

- CSR interface
- TX shifter
- TX FIFO
- TX flow control
- RX flow control
- RX shifter
- Clock generator
- RX FIFO
- Clock and reset interface
- Avalon-MM interface
- RS232 serial interface

The UART behaviour is controlled using internal registers.

7. What Is CSR?

CSR stands for **Control and Status Register**.

The CSR interface is the register access interface of an IP block.

It allows software or the testbench to:

- Configure the IP
- Control the IP
- Read status information
- Observe internal hardware behaviour through registers

8. Role of CSR Interface

The CSR interface sits between:

System bus → CSR interface → Internal IP logic

The system bus may be:

- APB
- AXI
- Avalon
- Wishbone

The CSR interface allows the bus to access internal registers.

9. CSR Interface Responsibilities

The CSR interface usually performs the following tasks:

- Decodes bus addresses
- Maps addresses to internal registers
- Handles register read operations
- Handles register write operations
- Updates control signals to internal blocks
- Collects status signals from internal blocks
- Converts bus writes into internal control updates

10. UART Register Examples

A UART IP may contain registers such as:

Register	Meaning	Access Type	Purpose
`RBR`	Receiver Buffer Register	Read-only	Holds received data
`THR`	Transmit Holding Register	Write-only	Holds data to be transmitted
`IER`	Interrupt Enable Register	Read/write	Enables UART interrupts

`IIR`	Interrupt Identification Register	Read-only	Indicates interrupt source
`DLL`	Divisor Latch LSB	Read/write	Baud-rate divisor LSB
`DLM`	Divisor Latch MSB	Read/write	Baud-rate divisor MSB

Different registers have different access permissions.

11. Register Access Types

Common register access types are:

Access Type	Meaning
Read-only	Can only be read
Write-only	Can only be written
Read-write	Can be read and written

Example:

- `RBR` is read-only.
- `THR` is write-only.
- `DLL` and `DLM` are read-write.

12. Register Address and Offset

Each register has an address or offset address.

The processor or testbench accesses a register by providing its address.

Example:

Register	Example Offset
Mode register	`0x01`
Width register	`0x10`

For a UART IP, registers such as `RBR`, `THR`, `IER`, `IIR`, `DLL`, and `DLM` also have fixed offset addresses defined in the specification.

13. Why Offset Address Is Needed

The processor or testbench must provide the correct address to access the required register.

Example:

Base address + Register offset = Actual register access address

If the wrong offset is used, the wrong register may be accessed.

14. Register Programming

Register programming means writing values into registers to configure the IP.

For UART, register programming may configure:

- Baud rate
- Word length
- Stop bits
- Parity
- Interrupts
- FIFO behaviour
- Error conditions

The exact values must be taken from the IP specification.

15. Testing Without RAL Model

Without a RAL model, the UVM testbench must directly perform bus-level reads and writes.

The test must manually know:

- Register offset addresses
- Bit positions
- Field meanings
- Reset values
- Access permissions
- Last written values

Example:

avalon_write(base_addr + offset, data);

Here, the test directly writes to a register through the bus interface.

16. Example: Configuring UART Without RAL

To configure UART as:

8-bit data, 1 stop bit, no parity

The test must know:

- LCR register offset address
- Bit fields inside LCR
- Correct value for 8-bit word length

- Correct value for stop bit
- Correct value for parity

To set baud rate, the test may need to program:

- DLAB bit
- DLL register
- DLM register

All these require manual address and bit-field handling.

17. Problems Without RAL

Without a RAL model:

- Tests become bus-aware.
- Tests become register-map-aware.
- Tests need manual bit manipulation.
- Tests must remember register addresses.
- Tests must track reset values manually.
- Tests must track last written values manually.
- Same register programming logic is repeated in many tests.
- Changing a register offset affects many tests.
- Changing the bus protocol may require rewriting tests.
- Accidental writes to read-only registers may not be caught cleanly.
- Accidental reads from write-only registers may not be caught cleanly.

This makes the testbench harder to maintain and reuse.

18. What Is a UVM RAL Model?

A UVM RAL model is a register-level representation of the DUT registers inside the UVM testbench.

It models:

- Registers
- Fields
- Access types
- Reset values
- Address offsets
- Mirror values
- Register maps

Instead of directly using bus addresses and bit masks, the test accesses registers as objects.

19. RAL Model as a Register Copy

The RAL model creates a software-level representation of the DUT register map.

If the DUT has registers, the RAL model contains matching register objects.

Example:

DUT registers RAL model registers
Mode register → Mode register object
Width register → Width register object

This allows the testbench to track register values in an organized way.

20. Mirror or Shadow Copy

The RAL model maintains a **mirror** or **shadow copy** of register values.

This means the RAL model stores what it expects each register value to be.

Benefits:

- Tracks values written to registers
- Checks expected register values
- Helps verify reset values
- Detects unexpected register changes

21. RAL Access Rules

The RAL model understands register access permissions.

Example:

- If a test tries to write to a read-only register, RAL can flag an error.
- If a test tries to read from a write-only register, RAL can flag an error.

This prevents illegal register access from silently passing.

22. Protocol Independence

RAL separates register-level tests from bus protocol details.

Without RAL, a test may call:

apb_write(address, data)
avalon_write(address, data)
axi_write(address, data)

With RAL, the test works at register level.

If the bus changes from APB to Avalon or AXI, the test code can remain mostly the same.

Only the adapter or bus connection layer needs to change.

23. Why RAL Improves Reuse

RAL improves reuse because:

- Tests are written using register objects.
- Register addresses are stored in the RAL model.
- Field positions are stored in the RAL model.
- Access rules are stored in the RAL model.
- Bus-specific details are hidden.
- Same register test can work across different bus protocols.

24. RAL Model in Testbench

The DUT contains real hardware registers.

The UVM testbench contains a RAL model that represents those registers.

Conceptually:

```
DUT
 └── Hardware registers

UVM Testbench
 └── RAL model
     └── Register objects
```

The RAL model helps the testbench access and track the DUT registers.

25. RAL Model and DUT Register Updates

When a test writes to a DUT register through RAL:

1. The test accesses a register object.
2. The RAL model converts it into a register operation.
3. The bus agent performs the actual bus transaction.
4. The DUT register is updated.
5. The RAL mirror value is also updated.

This keeps the testbench view aligned with the DUT register state.

26. RAL Model Components Mentioned

The RAL-based testbench may include:

- Register model
- Register database
- Sequence adaptation layer
- Bus agent
- Adapter

These parts help convert high-level register operations into bus transactions.

Detailed implementation is discussed in later RAL topics.

27. RAL vs Direct Bus Access

Feature	Without RAL	With RAL
Test style	Bus-level	Register-level
Register access	Manual address + data	Register object methods
Address handling	Done in test	Stored in RAL model
Bit-field handling	Manual	Stored in field objects
Access checking	Manual	Built into RAL
Mirror tracking	Manual	Automatic mirror/shadow copy
Bus dependency	High	Lower
Reuse	Difficult	Better

28. Key Points to Remember

- RAL stands for Register Abstraction Layer.
- RAL models DUT registers inside the UVM testbench.
- IP blocks are configured using internal registers.
- CSR interface provides access to internal IP registers.
- Registers have offset addresses.
- Registers may be read-only, write-only, or read-write.
- Without RAL, tests must manually handle addresses, bit fields, and access rules.
- RAL allows tests to access registers as objects.
- RAL maintains a mirror or shadow copy of register values.
- RAL helps check reset values and unexpected register changes.
- RAL can catch illegal accesses, such as writing to read-only registers.
- RAL makes tests less dependent on the bus protocol.
- If the bus changes, usually only the adapter changes.
- RAL improves reuse, readability, and maintainability.

29. Final Summary

The UVM RAL model provides a clean abstraction between the UVM testbench and DUT registers.

Instead of manually writing bus transactions using register addresses and bit masks, the test can access register objects directly.

The RAL model stores register addresses, field information, access permissions, reset values, and mirror values.

This makes register verification easier, reusable, protocol-independent, and less error-prone.

UVM Register Model: Registers and Fields

Introduction to UVM Register Model: Registers and Fields

1. Topic Overview

This section explains the basic structure of a **UVM Register Abstraction Layer (RAL) model**.

The main focus is on:

- Register database
- Register fields
- UVM register class
- UVM register field class
- `new()` method
- `configure()` method
- Example register with multiple fields

2. Register Database in UVM RAL

In a UVM RAL model, the register database represents the DUT registers inside the testbench.

The register database is built using UVM built-in classes.

User-defined register and field classes are created by extending these built-in classes.

3. DUT Register Example

Assume a design contains three registers:

R0 register
R1 register
R2 register

Each register may contain one or more smaller bit groups called **fields**.

Example:

R0 register
├── Field 1
└── Field 2

4. What Is a Field?

A **field** is the smallest configurable unit inside a register.

A field represents a specific set of bits within a register.

A field can be:

- 1 bit wide
- Multiple bits wide
- Status field
- Control field
- Data field
- Configuration field

Example:

Register R3
└── Field [7:0]

Here, bits `[7:0]` form one field.

5. Purpose of a Field

A field is used to represent a particular meaning inside a register.

For example:

- A status bit may indicate whether an event occurred.
- A control bit may enable or disable a feature.
- A mode field may select the operating mode.
- A data field may store a value.

The exact meaning of each field is defined in the design specification.

6. What Is a Register?

A **register** is a collection of fields accessed as a single unit.

A register represents an actual hardware register inside the DUT.

Example:

Control Register
├── Enable field
├── Mode field
└── Interrupt field

The register value controls how the design behaves.

7. Register Access Types

A register or field can have different access permissions.

Common access types are:

Access Type	Meaning
Read-only	Can only be read
Write-only	Can only be written
Read-write	Can be read and written

The access type is defined during field configuration.

8. UVM Register Class

In UVM RAL, a user-defined register class extends the built-in class:

uvm_reg

Example:

class control_reg extends uvm_reg;

Since `uvm_reg` is part of the UVM object hierarchy, the register class is treated as a UVM object.

9. UVM Register Field Class

Register fields are created using the built-in class:

uvm_reg_field

Example:

uvm_reg_field enable;
uvm_reg_field mode;
uvm_reg_field status;

Each field handle represents one field inside the register.

10. `new()` Method in Register Class

The `new()` method is used to create the register object.

It defines:

- Register name
- Register size
- Coverage setting

General syntax:

```
function new(string name = "register_name");
  super.new(name, number_of_bits, coverage);
endfunction
```

Example:

```
function new(string name = "control_reg");
  super.new(name, 32, UVM_NO_COVERAGE);
endfunction
```

11. Purpose of `new()`

The `new()` method:

- Allocates memory for the register object
- Defines the register width
- Enables or disables coverage

At this stage:

- Fields are not created yet.
- Field positions are not configured yet.
- Register addressing is not assigned yet.

Only the basic register identity and size are defined.

12. `configure()` Method for Fields

The `configure()` method is used to configure a field inside a register.

It defines:

- Parent register
- Field width
- LSB position
- Access type
- Volatile property
- Reset value
- Whether reset exists
- Whether the field is randomizable

13. Field `configure()` Arguments

General form:

```
field.configure(
  parent_register,
  size,
```

```
  lsb_position,
  access_type,
  volatile,
  reset_value,
  has_reset,
  is_rand,
  individually_accessible
);
```

Common Argument Meaning

Argument	Meaning
`parent_register`	Register that contains the field
`size`	Width of the field in bits
`lsb_position`	Starting bit position of the field
`access_type`	Read/write permission
`volatile`	Whether value can change unexpectedly
`reset_value`	Field reset value
`has_reset`	Indicates whether reset value exists
`is_rand`	Indicates whether field can be randomized
`individually_accessible`	Indicates whether field can be accessed individually

14. Example Field Configuration

Example for an `enable` field:

```
enable.configure(
  this,
  1,
  0,
  "RW",
  0,
  0,
  1,
  0,
  0
);
```

Meaning

Value	Meaning

`this`	Field belongs to current register
`1`	Field width is 1 bit
`0`	LSB position is bit 0
`"RW"`	Field is read-write
`0`	Field is non-volatile
`0`	Reset value is 0
`1`	Reset value exists
`0`	Field is not randomizable
`0`	Not individually accessible

15. Example: Control Register

A control register can contain three fields:

```
control_reg
 ├── enable
 ├── mode
 └── status
```

Each field is created using `uvm_reg_field`.

Example declarations:

```
uvm_reg_field enable;
uvm_reg_field mode;
uvm_reg_field status;
```

16. Control Register Class

The control register is created by extending `uvm_reg`.

```
class control_reg extends uvm_reg;

  `uvm_object_utils(control_reg)

  uvm_reg_field enable;
  uvm_reg_field mode;
  uvm_reg_field status;

  function new(string name = "control_reg");
    super.new(name, 32, UVM_NO_COVERAGE);
```

```
  endfunction

endclass
```

17. Creating Field Objects

Inside the register build method, each field object is created using factory creation.

Example:

```
enable = uvm_reg_field::type_id::create("enable");
mode   = uvm_reg_field::type_id::create("mode");
status = uvm_reg_field::type_id::create("status");
```

18. Configuring Fields

After creating the fields, each field is configured.

Example:

```
enable.configure(this, 1, 0, "RW", 0, 0, 1, 0, 0);
mode.configure  (this, 3, 1, "RW", 0, 0, 1, 0, 0);
status.configure(this, 4, 4, "RO", 0, 0, 1, 0, 0);
```

Meaning

Field	Width	LSB Position	Access
`enable`	1 bit	0	Read-write
`mode`	3 bits	1	Read-write
`status`	4 bits	4	Read-only

The remaining bits of the 32-bit register may be reserved or used for future fields.

19. Complete Control Register Example

```
class control_reg extends uvm_reg;

  `uvm_object_utils(control_reg)

  uvm_reg_field enable;
  uvm_reg_field mode;
  uvm_reg_field status;
```

```
  function new(string name = "control_reg");
    super.new(name, 32, UVM_NO_COVERAGE);
  endfunction

  virtual function void build();

    enable = uvm_reg_field::type_id::create("enable");
    mode   = uvm_reg_field::type_id::create("mode");
    status = uvm_reg_field::type_id::create("status");

    enable.configure(this, 1, 0, "RW", 0, 0, 1, 0, 0);
    mode.configure  (this, 3, 1, "RW", 0, 0, 1, 0, 0);
    status.configure(this, 4, 4, "RO", 0, 0, 1, 0, 0);

  endfunction

endclass
```

20. Register and Field Relationship

The relationship is:

```
Register
 └── Field(s)
```

Example:

```
control_reg [31:0]
 ├── enable [0]
 ├── mode   [3:1]
 ├── status [7:4]
 └── reserved [31:8]
```

21. Important Notes

- A field is the smallest unit inside a register.
- A register is a collection of fields.
- A register class extends `uvm_reg`.
- A field is created using `uvm_reg_field`.
- The `new()` method defines register name, size, and coverage.
- The `configure()` method defines field properties.
- Field properties include width, position, access type, reset value, and randomization.
- Fields must be created before they are configured.
- Unused register bits can be kept as reserved bits.

22. Final Summary

In a UVM RAL model, DUT registers are represented using register classes derived from `uvm_reg`.

Each register contains one or more fields created using `uvm_reg_field`.

The `new()` method defines the register size and coverage settings.

The `configure()` method defines each field's width, bit position, access type, reset value, and other properties.

This forms the basic structure of a register model inside the UVM register database.